

BUILDTRUST

Da Start-up a Software Factory

Assessment AS-IS | Target Operating Model | Piano di Implementazione 180 giorni

Prepared by: Marco Scarpa

Versione 2.0 – Marzo 2026

Riservato – uso interno

Indice

- **1. Premessa e finalità del documento**
- **2. Executive Summary**
 - 2.1 Come leggere questo documento
- **3. Natura distintiva del modello BuildTrust**
- **4. Contesto di mercato**
- **5. Struttura organizzativa AS-IS**
 - 5.1 Governance e direzione
 - 5.2 Funzioni di staff
 - 5.3 Team operativo – Area BuildTrust Platform e Digital Plant
 - 5.4 Team operativo – Area The Safe Place (TSP)
 - 5.5 Partner tecnologici
 - 5.6 Criticità strutturali AS-IS
- **6. Pipeline commerciale e capacity planning**
 - 6.1 Pipeline 2026
 - 6.2 Vista Gantt – scenario a capacità infinita
 - 6.3 Scenario consequenziale governato
- **7. Principi guida del Target Operating Model**
- **8. Struttura organizzativa TO-BE**
 - 8.1 Schema generale
 - 8.2 CEO
 - 8.3 COO
 - 8.4 Business Development e Presales
 - 8.5 Delivery Manager e Product Manager
 - 8.6 Engineering
- **9. Profili tecnici – competenze, strumenti e responsabilità quotidiane**
 - 9.1 Cloud Engineer
 - 9.2 Data & AI Engineer
 - 9.3 Full Stack Developer
- **10. Modello di governance**
 - 10.1 Rituali operativi
 - 10.2 Artefatti minimi obbligatori
 - 10.3 Perimetri decisionali
 - 10.3.1 Guida pratica di lettura dei ruoli chiave
- **11. Mappa dei processi core**
- **12. Processo commerciale e presale**
 - 12.1 Obiettivo e responsabilità
 - 12.2 Fasi operative
 - 12.3 Gate obbligatorio pre-firma
 - 12.4 Handover commerciale verso delivery
- **14. Processo di delivery di progetto**
 - 14.1 Obiettivo e responsabilità
 - 14.2 Fasi operative
 - 14.3 Artefatti obbligatori per progetto
 - 14.3.1 Guida pratica all'uso del board di progetto
 - 14.4 Come leggere la salute reale di un progetto
 - 14.5 Comunicazione cliente, milestone e readiness
 - 14.6 Sequenza manageriale end-to-end del progetto
- **15. Processo di sviluppo prodotto**
 - 15.1 Obiettivo
 - 15.2 Classificazione delle richieste di sviluppo
 - 15.2.1 Quattro domande per distinguere prodotto da custom
 - 15.3 Modello milestone + Kanban

- 15.4 Fasi operative del ciclo di sviluppo
- 15.5 KPI di sviluppo e qualità
- 15.6 Ruolo del Data & AI Engineer nello sviluppo prodotto

■ 16. Processo di gestione partner e fornitori

- 16.1 Obiettivo
- 16.2 Cinque principi non negoziabili
- 16.3 Fasi operative
- 16.4 Checkpoint operativi e controllo dei deliverable partner

■ 17. Processo di supporto, customer success e continuità operativa

- 17.1 Obiettivo
- 17.2 Classificazione e SLA
- 17.3 Fasi operative
- 17.4 Go-live e stabilizzazione

■ 18. Processo di issue, escalation e decision management

- 18.1 Livelli di severità e regole di escalation
- 18.2 Decision Log – come si usa
- 18.3 Comportamento atteso nelle escalation

■ 19. Capacity model e criteri di scalabilità

- 19.1 Competenze critiche e fabbisogno per livello di parallelismo
- 19.2 Regola di parallelismo
- 19.3 Criteri di intake progetto
- 19.3.1 Guida pratica alle priorità quando la capacità è in tensione
- 19.4 Demand planning operativo a sei mesi
- 19.5 Piano di rafforzamento progressivo

■ 20. Toolchain operativa – Azure DevOps e ecosistema Microsoft

- 20.1 Principi di selezione
- 20.2 Stack completo BuildTrust
- 20.3 Configurazione Azure DevOps – struttura raccomandata
- 20.4 Integrazione HubSpot – Azure DevOps
- 20.5 Integrazione Teams – Azure DevOps
- 20.6 Uso operativo di Azure DevOps per prodotto, progetto ed engineering

■ 21. KPI, metriche e cruscotti di controllo

- 21.1 KPI commerciali – presidio BD e COO
- 21.2 KPI delivery – presidio DM e COO
- 21.3 KPI prodotto – presidio PM
- 21.4 KPI milestone, evidenze e acceptance – presidio COO e DM
- 21.5 Dashboard Power BI – struttura raccomandata
- 21.5.1 Routine di lettura manageriale della dashboard
- 21.5.2 Lettura integrata della dashboard per CEO, COO, PM e DM
- 21.6 Come leggere davvero velocity, flow e KPI senza usarli male

■ 22. Matrice RACI

■ 23. Template operativi

- 23.1 Template Opportunity Brief (HubSpot)
- 23.2 Template Milestone & Acceptance Matrix
- 23.3 Template Project Charter
- 23.4 Template Weekly Status Note
- 23.5 Template Closure Memo

■ 24. Rischi organizzativi e contromisure

■ 25. Change management e piano di adozione

- 25.1 Filosofia di adozione
- 25.2 Fattori critici di successo
- 25.3 Segnali di adozione riuscita a 90 giorni
- 25.4 Comportamenti attesi del nuovo modello operativo
- 25.5 Onboarding e knowledge capture
- 25.6 Il valore organizzativo della precisione
- 25.7 Piano suggerito di adozione del TOM

- 25.8 Cosa fare nei primi 180 giorni
- 25.9 Governance dell'adozione e rischi tipici
- 25.10 Segnali concreti di adozione riuscita e prime decisioni

■ PARTE IX – APPENDICE MANAGERIALE

■ PARTE X – APPENDICE OPERATIVA

■ 26. Conclusioni

1. Premessa e finalità del documento

Il presente documento costituisce la fusione organica tra l'assessment preliminare prodotto su BuildTrust e il Target Operating Model (TOM) sviluppato a partire da esso. La finalità è offrire al CEO, al COO e al team operativo un riferimento unico e coerente: un documento che descriva lo stato attuale dell'organizzazione, definisca il modello target verso cui evolvere e indichi, con precisione operativa, chi fa cosa, con quali strumenti e secondo quali processi.

Il focus non è sull'architettura tecnica della piattaforma né sulla roadmap di prodotto. Il perimetro di questo TOM è l'organizzazione: ruoli, responsabilità, flussi operativi, meccanismi decisionali, governo della pipeline, toolchain e processi che trasformano un contratto firmato in flusso esecutivo tracciabile e certificabile.

BuildTrust non è una software house generica. È un modello ibrido tra software company B2B, organizzazione di delivery per progetti complessi, piattaforma legal-tech / construction-tech e presidio operativo di evidenze e certificazioni digitali. Il TOM proposto riflette questa specificità e non è riducibile al modello standard di una startup SaaS.

Per quanto riguarda la toolchain operativa, il documento adotta Azure DevOps come scelta definitiva per la gestione di progetti, backlog e pipeline di sviluppo. Questa scelta è coerente con l'ecosistema Microsoft già presente nel gruppo Simem (Office 365, Azure) e consente di massimizzare l'integrazione tra CRM (HubSpot), collaboration (Teams/SharePoint) e project management senza introdurre stack tecnologici aggiuntivi.

2. Executive Summary

2.1 Come leggere questo documento

Questo TOM può essere letto su due livelli complementari. Per il management e una guida di governo: spiega come strutturare flussi, decisioni, capacity planning, demand planning e criteri di lettura delle performance. Per i team tecnici e operativi e una guida di esecuzione: chiarisce processi, artefatti, strumenti, regole di lavoro e comportamenti attesi lungo il ciclo di vita di prodotto e progetto.

Il documento è quindi pensato per essere usato, non solo consultato. Le sezioni iniziali descrivono contesto, assetto e modello target. Le sezioni centrali definiscono processi, ruoli, toolchain e KPI. Le sezioni finali trasformano il modello in guida operativa quotidiana, con approfondimenti su Azure DevOps, project delivery, product management, demand planning, lettura manageriale delle performance e piano di adozione del TOM.

BuildTrust si presenta al mercato con una proposizione di valore rara e concreta: la capacità di trasformare il ciclo di vita di un progetto complesso – dalla progettazione all'esecuzione – in un flusso digitale tracciabile, verificabile e certificabile. Questa capacità integra in un unico modello tecnologie che normalmente operano in silos separati: BIM per la modellazione, IoT per il monitoraggio in campo, AI per l'analisi predittiva e meccanismi di certificazione digitale applicati dalla piattaforma quando il caso d'uso cliente lo richiede.

Il mercato di riferimento – costruzioni e infrastrutture complesse – è storicamente resistente alla digitalizzazione, caratterizzato da cicli di vendita lunghi, elevata complessità contrattuale e forte domanda di credibilità operativa. In questo contesto, BuildTrust opera con un vantaggio competitivo reale e difficilmente replicabile nel breve termine da attori generici.

L'analisi della struttura attuale evidenzia tuttavia una discontinuità significativa tra il potenziale della pipeline commerciale e la capacità organizzativa disponibile. L'organizzazione è attualmente dimensionata per gestire un solo progetto complesso alla volta. La pipeline 2026 prevede fino a cinque progetti attivi in parallelo nel secondo semestre, con un fabbisogno teorico di cinque risorse per competenza critica. Anche nello scenario consequenziale governato – che limita il parallelismo a due o tre progetti core – il gap organizzativo è strutturale e richiede un intervento deliberato.

Le cinque leve del Target Operating Model proposto sono:

Leva strategica	Descrizione sintetica
1. COO come centro di gravità operativo	Unica figura responsabile dell'esecuzione end-to-end, della capacità e della governance
2. Separazione Product / Delivery	Product Manager protegge la coerenza della piattaforma; Delivery Manager governa l'esecuzione progetto
3. Funzione commerciale qualificata	Sales e Presales operano in modo integrato; nessuna offerta senza validazione di capacità
4. Governo strutturato dei partner	Owner interno per ogni partner, work package formali, knowledge capture sistematico
5. Readiness di progetto e governo delle evidenze	Processo distintivo che traduce il venduto in milestone leggibili, evidenze verificabili e criteri di acceptance sostenibili

L'implementazione del TOM è pianificata in 180 giorni, con output verificabili per ogni fase. Il documento fornisce al team operativo un riferimento pratico su ogni processo: chi avvia, chi esegue, chi approva, con quale strumento e con quale output atteso.

3. Natura distintiva del modello BuildTrust

Comprendere la natura del modello è il prerequisito per applicare correttamente il TOM. BuildTrust non è posizionabile in una sola categoria:

Dimensione	Descrizione
Software company B2B	Esiste un prodotto proprietario (la piattaforma) che evolve come asset e deve essere protetto da derive custom
Delivery organization	Ogni cliente richiede configurazione, integrazione, raccolta dati e coordinamento stakeholder: non è un deploy SaaS standard
Legal-tech / Construction-tech	Il valore differenziante è rendere opponibili, certificabili e interpretabili gli eventi di progetto
Presidio di evidenze digitali	La piattaforma non solo monitora: trasforma condizioni contrattuali in stati verificabili con fondo probatorio

Il vero differenziale competitivo di BuildTrust non è la capacità di raccogliere dati, ma la capacità di trasformare quei dati in evidenze opponibili che rendono l'avanzamento leggibile, difendibile e verificabile. Questo deve essere il filo conduttore di ogni processo operativo.

PARTE II – ANALISI AS-IS

4. Contesto di mercato

BuildTrust opera nel mercato delle costruzioni e delle infrastrutture complesse, un contesto caratterizzato da tre dinamiche strutturali che definiscono sia le opportunità che le sfide operative.

Dinamica	Impatto su BuildTrust
Cicli di vendita lunghi (6-18 mesi)	Richiede funzione commerciale paziente, pipeline strutturata e forecast affidabile. Non si chiude in fretta ma si chiude bene.
Elevata complessità contrattuale	Clausole di SAL, penali, milestone e obblighi informativi richiedono un forte presidio di handover, readiness, evidenze e acceptance.
Forte domanda di credibilità	Il cliente non acquista software: acquista affidabilità. Referenze operative, evidence pack e capacità di rendere verificabile l'avanzamento sono asset commerciali prima ancora che tecnici.

Nel breve periodo BuildTrust è focalizzata su progetti pilota e prime iniziative strutturate con l'obiettivo di costruire referenze solide. Nel medio termine queste referenze saranno il principale driver di accesso a progetti di dimensione maggiore, dove la credibilità operativa è condizione necessaria per partecipare alle gare.

5. Struttura organizzativa AS-IS

5.1 Governance e direzione

Il Consiglio di Amministrazione è composto da Federico Furlani e Stefano Gelain. Federico Furlani ricopre il ruolo di Amministratore Delegato con responsabilità su strategia, sviluppo commerciale e potere di firma. Andrea Arrigoni presidia lo Sviluppo Business, la pianificazione del budget e il cash flow. Stefano Gelain ricopre il ruolo di COO ed è attualmente l'unica figura che concentra su di sé quattro responsabilità critiche e distinte: Sviluppo Prodotto, Delivery, Project Management operativo e governo dei partner tecnologici.

Rischio attivo: raccomandazione prioritaria. La concentrazione di questi quattro ruoli su Gelain non è un rischio futuro: è il vincolo operativo più critico dell'organizzazione nel momento in cui questo documento viene adottato. Ogni progetto che entra in esecuzione aggrava il carico su un'unica persona che non può essere sostituita su nessuno dei quattro fronti. Finché la separazione tra Product Manager e Delivery Manager non avviene con persone distinte e perimetri formali, il TOM rischia di produrre benefici solo parziali, redistribuendo più le etichette che il carico reale. Per questo motivo si raccomanda al CEO e al COO di trattare questa separazione come una priorità molto alta del piano di adozione, idealmente entro i primi 60 giorni o secondo una tempistica formalmente concordata.

5.2 Funzioni di staff

Le funzioni di staff sono mutate dal gruppo Simem e non presidiano direttamente le esigenze specifiche di una software company in scaling. I referenti attuali sono:

Funzione	Referente	Nota
Amministrazione	Matteo Dal Bosco, Sara Nicolato	Gestione contabile e amministrativa
HR	Elena Cream	Selezione e gestione del personale
Marketing	Chiara Schenato	Comunicazione e presenza di mercato
IT	Michael Ambroso	Infrastruttura IT di gruppo
Supporto fiscale	Studio S.L.T.	Consulenza esterna

5.3 Team operativo – Area BuildTrust Platform e Digital Plant

Ruolo	Persona	Ambito principale
Digital Transformation & Automation Manager	Robert Armellini	Riferimento tecnico operativo, coordinamento partner
Model-Based Systems Engineer	Mario Libro	Ingegneria di sistema, modellazione

Ruolo	Persona	Ambito principale
Legal & Smart Contract	Veronica Paternolli	Competenza legale-blockchain, smart contract
Sistemista	Eros Cason	Infrastruttura, sistemi, ambienti
Data Scientist	Anna Marcazzan	Analisi dati, modelli predittivi
CDE Developer	VACANTE	Common Data Environment – competenza critica mancante
DB Specialist	VACANTE	Database e persistenza – competenza critica mancante
PLC / SCADA Developer	VACANTE	Integrazione sistemi di campo – competenza critica mancante

5.4 Team operativo – Area The Safe Place (TSP)

Ruolo	Persona
AI Developer – Responsabile di Progetto	Michele Boldo
AI Developer – Nuovi Sviluppi	Enrico Martini
AI Developer – Industrializzazione	Stefano Aldegheri
AI Subject Matter Expert	Nicola Bombieri

5.5 Partner tecnologici

Una quota rilevante del lavoro tecnico è oggi affidata a partner esterni. La gestione di tutti i partner è accentrata su Stefano Gelain, senza owner interni formalmente nominati per ciascuna relazione.

Partner	Ambito	Owner interno attuale
ID-Group	Elaborazioni BIM	Robert Armellini
Blockchain Italia	Blockchain e smart contract	Veronica Paternolli
Maxfone	Digital Plant	Robert Armellini
Orienta + Trium	Sviluppi CDE	Non nominato
StatWolf	Modelli AI e database	Anna Marcazzan
Factoryal	Data integration e factory management	Non nominato
Ipsum Lab	Sviluppo web	Non nominato
XCF	UX models	Non nominato

5.6 Criticità strutturali AS-IS

- Accentramento decisionale: più ruoli critici su un'unica figura rende l'organizzazione fragile e poco scalabile.
- Confusione prodotto / progetto: non esiste separazione formale tra evoluzione della piattaforma e delivery dei singoli progetti cliente.
- Partner non governati: 4 partner su 8 non hanno un owner interno nominato, con rischio di dipendenza e dispersione del know-how.
- Vacancies critiche: tre ruoli tecnici fondamentali (CDE Developer, DB Specialist, PLC/SCADA) sono ancora vuoti.
- Assenza di processi standard: ogni progetto viene gestito in modo custom, senza template, board comuni o rituali definiti.
- Toolchain non unificata: collaborazione, documentazione e tracking del lavoro avvengono su strumenti non integrati.

6. Pipeline commerciale e capacity planning

6.1 Pipeline 2026

L'analisi considera esclusivamente i progetti con avvio stimato nel 2026, distinguendo tra progetto con ordine firmato (certo) e progetti in pipeline previsionale. Nel perimetro analizzato: 1 progetto firmato (Inerteco), 10 in pipeline previsionale, per un totale di 11 iniziative. I cluster di appartenenza sono Digital Plant (DP), BuildTrust Platform e The Safe Place (TSP), con diverse iniziative ibride.

6.2 Vista Gantt – scenario a capacità infinita

Durata standard assunta: 3 mesi per progetto. Avvii secondo le date commerciali stimate.

Progetto	Stato	Cluster	Gen	Feb	Mar	Apr	Mag	Giu	Lug	Ago	Set	Ott	Nov	Dic
Inerteco	Firmato	DP+TSP		X	X	X								
Heidelberg Materials	Pipeline	DP	X	X	X									
Veneto Strade	Pipeline	Platform	X	X	X									
Colabeton	Pipeline	DP				X	X	X						

Progetto	Stato	Cluster	Gen	Feb	Mar	Apr	Mag	Giu	Lug	Ago	Set	Ott	Nov	Dic
Zanardi Fondrie	Pipeline	TSP				X	X	X						
Immobiliare Europea	Pipeline	Plt+ TSP						X	X	X				
Ecomar Italia	Pipeline	DP+ TSP							X	X	X			
Holcim Australia	Pipeline	DP+ TSP							X	X	X			
CAV	Pipeline	TSP							X	X	X			
Savills/Barrings	Pipeline	DP+ TSP							X	X	X			
Transgulf	Pipeline	Platform							X	X	X			

Picco di parallelismo: luglio-settembre 2026 con fino a 5 progetti non-DP attivi contemporaneamente. Fabbisogno teorico: 5 risorse per ciascuna competenza critica. Struttura attuale: dimensionata per 1 progetto complesso alla volta. Il gap è strutturale, non contingente.

6.3 Scenario consequenziale governato

Limitando il parallelismo dei progetti core (Platform e TSP) a un massimo di 2-3 e utilizzando i progetti Digital Plant come leva di continuità commerciale, il fabbisogno per competenza critica scende da 5 a 2-3 risorse. Questo rende il gap progressivo e governabile, a condizione che la consequenzialità diventi una policy formale di intake e non resti una semplice ipotesi.

Progetto	Stato	Cluster	Avvio Governato	Note
Inerteco	Firmato	DP+TSP	feb-26	Progetto certo, avvio immediato
Heidelberg Materials	Pipeline	DP	gen-26	DP: basso assorbimento risorse core

Progetto	Stato	Cluster	Avvio Governato	Note
Veneto Strade	Pipeline	Platform	mar-26	Slittato di 2 mesi per spaziatura
Colabeton	Pipeline	DP	apr-26	DP: in parallelo con Platform
Zanardi Fonderie	Pipeline	TSP	mag-26	Avvio dopo chiusura Veneto Strade
ImmobiliarEuropea	Pipeline	Plt+TSP	lug-26	Avvio dopo Zanardi Fonderie
Ecomar Italia	Pipeline	DP+TSP	set-26	Avvio dopo ImmobilierEuropea
Holcim Australia	Pipeline	DP+TSP	ott-26	Primo progetto internazionale
CAV	Pipeline	TSP	nov-26	
Savills/Barings	Pipeline	DP+TSP	gen-27	Slittato al 2027
Transgulf	Pipeline	Platform	feb-27	Slittato al 2027

Lo slittamento di Savills/Barings e Transgulf è una scelta di sequenziamento coerente con i limiti di capacità, ma introduce un rischio commerciale che il processo di pipeline management (sezione 12) deve presidiare attivamente: opportunità posticipate di oltre sei mesi senza un piano di ingaggio continuativo tendono a raffreddarsi. La gestione di questo tipo di situazione rientra nella responsabilità del BD e va tracciata in HubSpot come pipeline attiva, non archiviata.

PARTE III – PRINCIPI E STRUTTURA TO-BE

7. Principi guida del Target Operating Model

I seguenti principi non sono aspirazioni astratte. Sono vincoli operativi che devono guidare ogni decisione di organizzazione, processo e strumento. Quando sorge un dubbio su come agire in una situazione non prevista, questi principi sono il riferimento.

- Una sola ownership per ogni decisione critica. Ogni processo, progetto e relazione con un partner deve avere un unico responsabile interno identificato e noto a tutti. Non esistono responsabilità condivise senza un owner primario.
- Product e Delivery devono collaborare ma non confondersi. Le roadmap di prodotto non sono guidate dall'urgenza dei singoli progetti. I progetti non vengono gestiti come fossero sprint di sviluppo prodotto.
- Il COO e il sistema nervoso operativo. Traduce la strategia in esecuzione, governa la capacità, coordina le funzioni, intercetta i colli di bottiglia prima che diventino crisi.

- I partner esterni ampliano la capacita, non sostituiscono la regia. BuildTrust non puo esternalizzare la comprensione del problema ne la responsabilita del risultato. Il partner esegue; BuildTrust dirige.
- Ogni progetto ha un flusso di evidenze, non solo un flusso di attivita. La verificabilita del lavoro svolto e il nucleo del valore differenziante. Senza evidenze strutturate, la delivery perde credibilita e capacita di controllo.
- Standard minimo comune, adattamento controllato. Tutti i progetti seguono gli stessi processi base. Le personalizzazioni sono eccezioni documentate, non prassi divergenti.
- Gating prima del parallelismo. Nessun progetto entra in esecuzione senza aver superato una verifica di fattibilita operativa: owner nominato, capacita disponibile, dipendenze chiarite.
- Documentare il necessario, non il superfluo. La burocrazia rallenta. L'assenza di documentazione crea dipendenze personali e fragilita. Il giusto mezzo e definito dai template standard.
- Visibilita radicale del lavoro in corso. Chiunque nel team deve poter capire in qualsiasi momento lo stato dei progetti, le priorita aperte e i blocchi attivi. Azure DevOps Boards e il cruscotto primario.
- L'eccezione deve essere gestita, non assorbita in silenzio. Ogni deviazione dal piano deve essere dichiarata, valutata e decisa. Non ignorata finche non diventa emergenza.

8. Struttura organizzativa TO-BE

8.1 Schema generale

Livello	Funzione	Responsabilita primaria	Strumento principale
Executive	CEO	Strategia, relazioni istituzionali, fundraising, closing strategico	HubSpot, Office 365
Executive	COO	Operations end-to-end, capacita, governance, escalation	Azure DevOps, HubSpot, Power BI
Commercial	Business Development / Sales	Pipeline, qualificazione, presidio mercato	HubSpot
Commercial	Presales / Account Management	Proposta, discovery, fattibilita, handover delivery	HubSpot, SharePoint
Delivery & Product	Delivery Manager	Esecuzione progetti, milestone, stakeholder, evidenze	Azure DevOps Boards, SharePoint
Delivery & Product	Product Manager	Evoluzione piattaforma, backlog, release, coerenza	Azure DevOps Backlog, GitHub/Azure Repos
Engineering	Cloud Engineer	Infrastruttura Azure, ambienti, deploy, sicurezza, monitoring	Azure DevOps Pipelines, Azure Monitor

Livello	Funzione	Responsabilità primaria	Strumento principale
Engineering	Data & AI Engineer	Pipeline dati, qualità, tracciabilità, ingestion	Azure Data Factory, Azure DevOps
Engineering	Full Stack Developer	Sviluppo applicativo, integrazioni, configurazioni progetto	Azure Repos, Azure DevOps, VS Code

8.2 CEO

Il CEO presidia visione strategica, relazioni istituzionali, partnership di alto livello e closing su opportunità ad alto impatto strategico. Non è coinvolto nella gestione operativa corrente. Delega esplicitamente al COO ogni decisione operativa, intervenendo solo su: contratti sopra soglia definita, partnership strategiche, comunicazione istituzionale, fundraising. Monitora la performance aziendale attraverso la dashboard Power BI condivisa con il COO.

8.3 COO

Il ruolo di COO è ricoperto da Stefano Gelain. Le sue responsabilità operative concrete includono: condurre il Weekly Operations Review e il Biweekly Delivery Deep Dive; validare ogni intake di progetto prima che il contratto venga firmato; mantenere aggiornato il capacity board in Azure DevOps; gestire le escalation operative; assicurarsi che ogni progetto abbia un Delivery Manager nominato, un project board attivo e criteri di milestone, evidenze e acceptance chiariti prima del kickoff.

Come opera il COO ogni settimana: Lunedì: review del capacity board in Azure DevOps e aggiornamento priorità. Mercoledì: Weekly Operations Review con DM e team Engineering (30 min). Giovedì: Weekly Sales & Pipeline Review con BD (30 min). A fine mese: Product & Capacity Council con CEO e PM. In ogni momento: decision log aggiornato, escalation S1 gestite entro 2h.

Rischio attivo: continuità operativa sul ruolo COO. Il modello assegna al COO la governance dell'intera esecuzione, incluse le escalation S1 con SLA di risposta entro 2 ore. Non esiste tuttavia una figura designata a coprire questo ruolo in caso di indisponibilità di Gelain. In un'organizzazione con progetti attivi e obblighi contrattuali di milestone, un vuoto operativo anche di pochi giorni può generare blocchi a cascata su delivery, intake e partner. Per questo motivo è fortemente raccomandabile che CEO e COO definiscano e comunichino al team una copertura di continuità operativa, chiarendo chi può agire da deputy COO in caso di assenza prolungata, quali decisioni può prendere autonomamente e quali invece richiedono escalation. Nella fase attuale, una delle opzioni più naturali può essere il Delivery Manager senior oppure, limitatamente ai temi di intake e firma, il CEO stesso. È inoltre consigliabile che questa scelta venga resa esplicita anche nel Decision Log.

Rischio di transizione: Gelain come COO e ancora unico presidio tecnico. Nella fase attuale Gelain ricopre formalmente il ruolo di COO ma continua di fatto a essere il principale punto di riferimento tecnico su prodotto, delivery e partner. Finché il Delivery Manager e il Product Manager non saranno operativi con piena autonomia nei rispettivi perimetri, il COO tenderà a rimanere sovraccarico e il modello di governance rischierà di non attivarsi pienamente. Ogni ritardo nel completare questa separazione prolunga il periodo in cui una quota rilevante dell'operatività resta concentrata su una sola persona.

8.4 Business Development e Presales

Il BD gestisce la pipeline in HubSpot con stage aggiornati in tempo reale: Lead, Qualified, Discovery, Fit Assessment, Proposal, Negotiation, Closed Won/Lost. Il Presales/Account conduce le sessioni di discovery, produce la Discovery Note e verifica con COO e Delivery Manager che l'offerta sia eseguibile prima di inviarla. Nessuna offerta esce senza il sign-off del COO sul piano di capacità.

8.5 Delivery Manager e Product Manager

Questi due ruoli dovrebbero essere distinti e, nel modello target, preferibilmente ricoperti da persone diverse. Una configurazione stabile in cui la stessa persona svolge entrambi i ruoli tende infatti a generare un conflitto strutturale tra roadmap di prodotto e urgenze di progetto, difficilmente governabile da una sola figura, indipendentemente dalla sua capacità individuale. Se nella fase di avvio del TOM i due ruoli risultano ancora sovrapposti, è raccomandabile che CEO e COO definiscano una data obiettivo di separazione formale e la trattino come una priorità del piano di adozione, non come un obiettivo aspirazionale a data aperta.

Il Delivery Manager guida l'esecuzione del progetto cliente: è responsabile di milestone, RAID log, evidence pack e comunicazione con il cliente. Il Product Manager protegge la piattaforma come asset: gestisce il backlog in Azure DevOps, decide cosa è prodotto e cosa è custom, coordina le release.

Implicazione organizzativa da monitorare. Con un solo progetto attivo la sovrapposizione può apparire gestibile. Con due o più progetti in parallelo - scenario previsto già nel secondo trimestre 2026 - tende però a diventare una delle principali cause di ritardo su entrambi i fronti: il prodotto rischia di essere distorto dall'urgenza del cliente, mentre la delivery tende a essere gestita in modo più reattivo. Il rischio non è teorico: è una dinamica già visibile nell'AS-IS e che questo TOM prova a ridurre.

Quando un progetto richiede uno sviluppo potenzialmente riusabile, il Delivery Manager apre una 'Feature Request' nel backlog di prodotto in Azure DevOps con tag 'da progetto'. Il Product Manager decide in sede di triage se includerla nella roadmap o classificarla come custom. Questa traccia nel tool è obbligatoria: non si discute a voce senza una issue aperta.

8.6 Engineering

Il Cloud Engineer lavora su Azure (già adottato nel gruppo) con Azure DevOps Pipelines per CI/CD, Azure Monitor per l'observability e Terraform per l'infrastruttura as code. È responsabile della qualità degli ambienti di sviluppo, staging e produzione e dei quality gate con SonarCloud.

Il Data Engineer presidia le pipeline di ingestione dati (Azure Data Factory), la qualità del dato e la disponibilità delle evidenze per le milestone. Lavora in stretto coordinamento con il Delivery Manager e aggiorna il Milestone & Evidence Register a ogni passaggio rilevante.

Il Full Stack Developer implementa componenti applicative, integrazioni UI/API/servizi e configurazioni progetto. Usa Azure Repos per il versioning, Azure DevOps per il tracking dei task e segue il workflow di branch definito dal team (main protetto, feature branch per ogni task, PR review obbligatoria).

Nel modello BuildTrust il presidio di milestone, evidenze e acceptance non è trattato come un processo separato o come una funzione autonoma dedicata. Fa parte della governance ordinaria di progetto: il Delivery Manager legge il contratto dal punto di vista esecutivo, il Data & AI Engineer presidia la qualità e la disponibilità delle evidenze, il COO verifica la readiness e il Product Manager interviene quando il progetto richiede impatti di piattaforma.

Posizionamento del team The Safe Place (TSP) nel modello TO-BE. Il team TSP – composto da Michele Boldo (AI Developer, Responsabile di Progetto), Enrico Martini (AI Developer, Nuovi Sviluppi), Stefano Aldegheri (AI Developer, Industrializzazione) e Nicola Bombieri (AI Subject Matter Expert) – è parte integrante dell'area Engineering nel modello TO-BE. Dal punto di vista funzionale, i profili AI Developer contribuiscono alle stesse aree presidiate dal Data & AI Engineer: sviluppo di modelli predittivi, pipeline dati, industrializzazione delle componenti AI. Nicola Bombieri opera come SME di validazione scientifica dei modelli, in coordinamento con il Product Manager e il Data & AI Engineer. In termini di governance e toolchain, il team TSP è previsto che segua gli stessi processi dell'area Engineering: backlog in Azure DevOps, branch strategy standard, quality gate SonarCloud, documentazione tecnica su SharePoint.

Questa descrizione ha carattere orientativo: il mapping completo dei ruoli TSP nel capacity model, nella RACI e nelle tabelle di staffing non è stato aggiornato in questa versione del TOM e richiede una sessione di allineamento esplicita tra COO e i referenti del team TSP durante le prime fasi di adozione. L'obiettivo è che ogni persona abbia un ruolo assegnato nel capacity board e che le responsabilità operative non restino implicite.

9. Profili tecnici – competenze, strumenti e responsabilità quotidiane

I profili tecnici che compongono l'area Engineering sono tre. Ciascuno ha un perimetro di responsabilità distinto, un set di strumenti primari e un insieme di attività ricorrenti che ne definiscono il contributo operativo quotidiano. Questa sezione costituisce il riferimento per la valutazione delle risorse esistenti, l'onboarding di nuove figure e la definizione dei criteri di hiring.

9.1 Cloud Engineer

Il Cloud Engineer è il presidio dell'affidabilità operativa dell'intera infrastruttura BuildTrust. È responsabile della disponibilità degli ambienti, della qualità del processo di deployment e della sicurezza operativa della piattaforma. Lavora in stretto coordinamento con il Full Stack Developer per le pipeline CI/CD e con il Data & AI Engineer per l'infrastruttura dati.

Area	Dettaglio
Responsabilit� primarie	Gestione ambienti Azure (dev, staging, prod); pipeline CI/CD in Azure DevOps Pipelines; infrastruttura as code con Terraform; monitoraggio e alerting con Azure Monitor; gestione secrets con Azure Key Vault; sicurezza operativa e compliance infrastrutturale
Attivit� quotidiane	Verifica stato pipeline e alert di monitoring al mattino; revisione PR che impattano infrastruttura; aggiornamento Terraform se necessario; supporto al team Engineering su issue di ambiente; partecipazione al daily async standup in Teams
Strumenti principali	Azure Portal, Azure DevOps Pipelines, Terraform, Azure Monitor, Azure Key Vault, Azure Container Registry, SonarCloud (quality gate integration), Teams
Output attesi	Pipeline CI/CD attive e stabili per ogni ambiente; zero deploy in prod senza approvazione esplicita; infrastruttura documentata come codice su Azure Repos; alert configurati per ogni servizio critico; runbook di incident su SharePoint
Competenze chiave	Azure cloud architecture (IaaS/PaaS), Terraform/IaC, CI/CD pipelines, container (Docker/ACI), networking Azure, sicurezza cloud, monitoring e observability

9.2 Data & AI Engineer

Il Data & AI Engineer presidia due domini distinti ma complementari: le pipeline di dati operative a supporto dei progetti cliente e lo sviluppo dei modelli di intelligenza artificiale integrati nella piattaforma. E la figura che trasforma dati grezzi di campo in evidenze strutturate e certificabili, e che porta capacit  predittiva e analitica all'interno del prodotto. Collabora con Nicola Bombieri (AI Subject Matter Expert) per la validazione scientifica dei modelli e con il Full Stack Developer per l'integrazione applicativa.

Area	Dettaglio
Responsabilit� primarie	Progettazione e gestione pipeline dati (ingestione, normalizzazione, trasformazione); sviluppo e manutenzione modelli AI/ML integrati nella piattaforma; qualit� e tracciabilit� dei dati di progetto per la certificazione milestone; sperimentazione di nuovi approcci algoritmici su backlog AI Platform; supporto al Delivery Manager nella raccolta evidenze
Attivit� quotidiane	Monitoraggio pipeline dati attive (Azure Data Factory); verifica qualit� dati in ingestione per progetti in corso; sviluppo su backlog AI in Azure DevOps; code review su PR di componenti dati/AI; aggiornamento documentazione modelli su SharePoint
Strumenti principali	Azure Data Factory, Azure Machine Learning, Azure Databricks (se adottato), Python (pandas, scikit-learn, pytorch/tensorflow secondo stack), Azure Blob Storage / Azure SQL, Azure DevOps Boards, Jupyter Notebooks, MLflow per experiment tracking, Teams
Output attesi	Pipeline dati operative e monitorate per ogni progetto attivo; modelli AI versionati e documentati su Azure Repos; evidence pack completi alla scadenza di ogni milestone; backlog AI Platform aggiornato con esperimenti e risultati; documentazione tecnica dei modelli su SharePoint
Competenze chiave	Data engineering (ETL/ELT, pipeline orchestration), machine learning (supervised/unsupervised, NLP se rilevante), Azure ML stack, Python avanzato, qualit� del dato, MLOps, integrazione API modelli

9.3 Full Stack Developer

Il Full Stack Developer   responsabile dello sviluppo delle componenti applicative della piattaforma e delle integrazioni tecniche richieste dai progetti cliente. Opera sia sul backlog di prodotto che sulle configurazioni e customizzazioni di progetto, sempre secondo la distinzione definita dal Product Manager nel processo di triage.   il principale esecutore tecnico del ciclo di sviluppo prodotto descritto nel capitolo 14.

Area	Dettaglio
Responsabilit� primarie	Sviluppo funzionalit� di piattaforma secondo backlog Platform in Azure DevOps; implementazione integrazioni con sistemi terzi (BIM, IoT, ERP cliente); configurazioni tecniche per i progetti cliente; code review su PR del team; contributo alla documentazione tecnica e agli ADR; supporto al Cloud Engineer per i deploy
Attivit� quotidiane	Pull dal backlog Kanban (item in stato Ready); sviluppo su feature branch; apertura e gestione PR con almeno un reviewer; aggiornamento task in Azure DevOps Boards; partecipazione al daily async standup; segnalazione blocchi tecnici come RAID Issue
Strumenti principali	Azure Repos / GitHub, Azure DevOps Boards, VS Code, Azure DevOps Pipelines, SonarCloud, Postman (API testing), Docker, Teams, SharePoint (ADR e documentazione tecnica)
Output attesi	Feature completate secondo criteri di accettazione definiti nella User Story; PR con quality gate SonarCloud verde; documentazione tecnica aggiornata per ogni feature rilasciata; zero deploy in prod senza approvazione PM/COO; contributo attivo alla retrospettiva di milestone
Competenze chiave	Sviluppo full stack (React/Angular frontend, Node.js/.NET/Python backend secondo stack), REST API design e integrazione, database relazionali e NoSQL, Docker/container, Git workflow, testing automatizzato

I tre profili tecnici non operano in isolamento. Il flusso tipico di un ciclo di sviluppo vede il Full Stack Developer e il Data & AI Engineer sviluppare in parallelo su branch distinti, mentre il Cloud Engineer garantisce la stabilit  degli ambienti e il processo di deploy. Il coordinamento avviene quotidianamente tramite Azure DevOps Boards e il canale Teams Engineering.

PARTE IV – GOVERNANCE E DECISIONI

10. Modello di governance

10.1 Rituali operativi

La governance si articola su sei rituali con cadenze, partecipanti e scopi distinti. Non sono riunioni di aggiornamento: sono momenti decisionali con output definiti.

Rituale	Cadenza	Chi	Scopo	Output atteso
Daily async standup	Giornaliera	Team operativo	Aggiornamento stato attività, blocchi	Task aggiornati in Azure DevOps Boards
Weekly Operations Review	Settimanale	COO + DM + Engineering	Stato progetti, issue aperte, priorità settimana	Action items nel decision log
Weekly Sales & Pipeline Review	Settimanale	COO + Sales + Presales	Pipeline, nuove opportunità, forecast	HubSpot aggiornato, intake assessment se necessario
Biweekly Delivery Deep Dive	Bisettimanale	COO + DM + PM	Milestone, rischi, partner, evidenze	RAID log aggiornato, eventuali escalation
Monthly Product & Capacity Council	Mensile	CEO + COO + PM + DM	Backlog, roadmap, capacità, intake nuovi progetti	Backlog prioritizzato, capacity plan aggiornato
Quarterly Strategic Review	Trimestrale	CEO + COO + BD	Strategia, pipeline, struttura, investimenti	Decisioni strategiche documentate

Come si gestisce un blocco operativo Il DM crea una issue tipo 'Issue' in Azure DevOps Boards con severità S1/S2/S3/S4, la assegna al COO e aggiorna il RAID log. Per S1 e S2 il COO risponde entro le tempistiche definite (vedi capitolo 19). Non si gestiscono blocchi critici in chat Teams senza una issue tracciata.

10.1.1 Disciplina di conduzione dei rituali

I rituali hanno valore solo se sono costruiti attorno a una domanda di governo. Quando una riunione esiste solo "per aggiornarci", tende a riempirsi di informazioni eterogenee e a produrre pochi esiti. In BuildTrust ogni rituale dovrebbe invece avere un oggetto chiaro: decidere intake, leggere capacità, governare backlog, proteggere una milestone, sbloccare un progetto o fare triage delle richieste di prodotto.

Questo significa che ogni rituale deve partire da artefatti già preparati. Il board è aggiornato prima della riunione, non durante. Le issue aperte hanno già severità e owner. Le richieste candidate al backlog prodotto arrivano con contesto minimo. Le milestone da discutere hanno già un primo stato di evidenze e readiness. Solo così il tempo delle persone più senior viene speso per leggere e decidere, non per inseguire dati mancanti.

Momento	Regola operativa	Strumento di riferimento
Prima del rituale	Board, RAID log, capacity plan e note devono essere già aggiornati. La riunione non serve a ricostruire dati mancanti.	Azure DevOps, SharePoint, HubSpot
Durante il rituale	Si navigano gli strumenti live. Le slide sono eccezioni; il board e la fonte primaria di verità.	Teams con condivisione schermo + Azure DevOps/HubSpot
Chiusura del rituale	Ogni decisione rilevante ha owner, scadenza e luogo di tracciamento. Se manca uno dei tre elementi, la decisione non è chiusa.	Decision Log o work item Azure DevOps

10.2 Artefatti minimi obbligatori

Artefatto	Strumento	Owner	Aggiornamento
Pipeline Board	HubSpot	BD / COO	Continuo
Capacity Board	Azure DevOps – vista Team	COO	Settimanale
Decision Log	SharePoint – cartella Operations	COO	Ad ogni decisione rilevante
Project Status One-Pager	SharePoint – cartella progetto	Delivery Manager	Settimanale
Product Backlog	Azure DevOps – Product Backlog	Product Manager	Ad ogni triage
RAID Log	Azure DevOps – progetto cliente	Delivery Manager	Settimanale

10.2.1 Funzione reale degli artefatti

Uno dei rischi tipici di crescita è moltiplicare documenti senza chiarirne la funzione. In BuildTrust conviene invece avere pochi artefatti, ma molto chiari. Il Project Charter serve per allineare l'impianto del progetto. Il RAID serve per leggere rischio e dipendenze. Il backlog di prodotto serve per ordinare la domanda di piattaforma. Il board di progetto serve per governare l'esecuzione. Il runbook serve per supporto e rilascio. La Weekly Status Note serve per la narrativa verso cliente. Quando ogni artefatto ha un compito preciso, le persone smettono di duplicare contenuti in luoghi diversi.

La regola pratica è semplice: se una informazione viene aggiornata spesso per guidare il lavoro, deve vivere in uno strumento operativo. Se deve restare come riferimento stabile, deve vivere in un documento. Se serve per formalizzare una decisione verso il cliente, deve avere una forma condivisibile e recuperabile.

10.3 Perimetri decisionali

Tipo di decisione	Chi decide	Soglia / Condizione
Decisioni operative quotidiane (risorse, task, configurazioni)	COO	Autonomo, nessuna escalation
Variazioni scope o timeline di progetto	COO + DM	Notifica CEO se impatto > 15% sul valore contratto
Intake nuovo progetto / avvio delivery	COO	Validazione CEO per progetti > 150k EUR
Evoluzione roadmap prodotto	PM con approvazione COO	Priorita alto impatto approvate in Product & Capacity Council
Partnership, investimenti, nuovi mercati	CEO	Con advisory del COO

10.3.1 Guida pratica di lettura dei ruoli chiave

Nel lavoro quotidiano i ruoli chiave non dovrebbero essere letti solo come caselle organizzative, ma come punti di vista complementari sullo stesso sistema. Il CEO legge coerenza strategica, credibilità del posizionamento e sostenibilità delle eccezioni. Il COO legge capacità, rischio operativo, tenuta dei flussi e necessità di intervento. Il Product Manager legge protezione della piattaforma, qualità del backlog e confine tra prodotto e custom. Il Delivery Manager legge esecuzione reale, milestone, readiness ed equilibrio della relazione col cliente.

La regola pratica è che ogni problema importante venga letto almeno da due prospettive: una di piattaforma e una di esecuzione. Quando questo non accade, BuildTrust tende a reagire a urgenze locali. Quando invece le letture restano compatibili, il modello operativo funziona come una software factory e non come una somma di interventi indipendenti.

In ottica manageriale, questi ruoli dovrebbero poter leggere lo stesso sistema con prospettive diverse ma compatibili. Il CEO guarda coerenza strategica e credibilità. Il COO guarda capacità, rischio operativo e stabilità del sistema. Il PM guarda coerenza della piattaforma. Il DM guarda execution reale, milestone e cliente. Quando questi sguardi si allineano, BuildTrust si comporta come una software factory. Quando si scollegano, l'organizzazione torna a reagire per urgenze locali.

PARTE V – PROCESSI OPERATIVI CORE

11. Mappa dei processi core

I processi operativi core coprono l'intero ciclo di vita di un'opportunità commerciale: dal primo contatto con il mercato alla chiusura del progetto e alla capitalizzazione degli apprendimenti. Per ogni processo sono descritti gli attori coinvolti, le fasi operative, gli strumenti utilizzati e gli output attesi.

#	Processo	Trigger	Output principale
1	Commerciale e presale	Lead identificato o opportunità inbound	Offerta tecnico-economica validata
2	Contrattualizzazione	Offerta accettata	Contratto firmato + pacchetto di handover
3	Setup e readiness di progetto	Contratto firmato	Project Charter, board Azure DevOps attivo, milestone e acceptance chiarite
4	Delivery ed esecuzione	Kickoff completato	Avanzamento verificabile per milestone
5	Verifica milestone	Evidence pack completo	Milestone verificata e accettata dal cliente
6	Gestione partner	Need identificato	Deliverable partner accettato, knowledge catturata
7	Supporto e customer success	Progetto in go-live o post-delivery	Ticket risolti, relazione cliente mantenuta
8	Issue e escalation management	Issue identificata	Decisione presa e documentata entro SLA

12. Processo commerciale e presale

12.1 Obiettivo e responsabilità

Trasformare lead e opportunità in proposte realistiche, profittevoli e operativamente sostenibili. Il BD è il responsabile del processo; il Presales/Account è il responsabile della qualità tecnica della proposta; il COO è il validatore della fattibilità.

12.2 Fasi operative

- Opportunity Identification. Il BD identifica e registra il lead in HubSpot, compila il campo 'Opportunity Brief' (problema del cliente, valore indicativo, cluster DP/Platform/TSP, probabilità realistica, next action). Stage HubSpot: Lead.

- **Qualification.** Il BD conduce una prima call esplorativa e verifica tre condizioni: problema reale e urgente, budget presente o in definizione, stakeholder con potere decisionale identificato. Se le tre condizioni sono soddisfatte, l'opportunità avanza. Stage: Qualified.
- **Discovery / Presale.** Il Presales/Account conduce una o più sessioni di discovery con il cliente. Produce la Discovery Note su SharePoint: contesto cliente, stakeholder rilevanti, processi coinvolti, fonti dati disponibili, milestone e vincoli contrattuali, ipotesi di soluzione, domande aperte. Stage: Discovery.
- **Fit Assessment.** Sessione interna con COO, DM e PM. Si valutano: disponibilità di capacità nel periodo di avvio, fit con la piattaforma (prodotto vs custom), fattibilità tecnica delle integrazioni. Output: go/no-go scritto nel decision log. Stage: Fit Assessment.
- **Solution Outline e Scoping.** Il Presales elabora la bozza di soluzione con decomposizione in componenti (prodotto, custom, partner), stima ore per competenza, dipendenze cliente. Il DM rivede la fattibilità operativa. Stage: Proposal.
- **Bid Review Interna.** Il COO valida la sostenibilità del piano di capacità. Il CEO approva per opportunità di valore strategico elevato. Nessuna offerta esce senza questo step.
- **Offerta e Negoziazione.** Invio offerta, negoziazione, documentazione delle variazioni. Ogni modifica di scope viene tracciata in HubSpot come nota. Stage: Negotiation → Closed Won/Lost.

12.2.1 Presales tecnico e discovery eseguibile

Nelle opportunità BuildTrust più complesse il presales non può limitarsi a descrivere la soluzione desiderata o a raccogliere requisiti in senso generico. Deve invece costruire una lettura eseguibile del problema: quali attori sono coinvolti, quali fonti dati esistono davvero, quali milestone contrattuali sono rilevanti, quali dipendenze tecniche devono essere risolte prima dell'avvio e quali elementi della piattaforma coprono già il bisogno rispetto a quelli che richiedono configurazione o sviluppo.

Una discovery di qualità deve ridurre l'ambiguità, non solo produrre documentazione. Questo significa che il presales dovrebbe uscire dalle sessioni con tre risultati minimi: una mappa del problema del cliente formulata in termini operativi, una prima lettura di fattibilità rispetto alla piattaforma BuildTrust e una lista di rischi da sottoporre al COO e al Delivery Manager nel gate pre-firma.

La qualità del presales ha un impatto diretto sulla salute della delivery. Una proposta con scope vago, dati assunti ma non verificati o ruoli lato cliente non chiariti genera quasi sempre un progetto che parte con ritardo invisibile. Per questo BuildTrust dovrebbe trattare la discovery come un investimento operativo e non come una fase commerciale leggera.

Domanda di discovery	Perche e critica per BuildTrust
Quale decisione operativa del cliente vogliamo rendere piu oggettiva?	Aiuta a collegare la soluzione al valore reale e a definire meglio milestone ed evidenze.
Quali dati esistono oggi e con quale affidabilita?	Riduce il rischio di vendere integrazioni che poi non partono.
Chi approvera davvero l'avanzamento e con quale forma?	Anticipa il modello di acceptance che il progetto dovra sostenere.
Quali eccezioni o personalizzazioni sono gia state chieste?	Aiuta a distinguere presto tra prodotto, configurazione e custom.
Quale partner o competenza esterna potrebbe servire?	Permette di leggere il vero assorbimento di capacita e il rischio di dipendenza.

12.3 Gate obbligatorio pre-firma

Prima di firmare qualsiasi contratto, il COO verifica: scope minimo chiaramente documentato, owner delivery nominato, stima di capacita validata, dipendenze cliente note, piano di avvio coerente con la pipeline esistente. Senza questi elementi il contratto non si firma.

12.4 Handover commerciale verso delivery

Al momento della firma il BD produce il pacchetto di handover su SharePoint (cartella progetto): riepilogo dell'opportunita vinta, scope promesso, assunzioni fondanti, rischi noti, elementi fuori scope ma sensibili, allegati (offerta definitiva, mail critiche, note di meeting rilevanti). Questo pacchetto e il punto di ingresso del setup di progetto e della verifica di readiness.

L'handover commerciale non e completo quando la proposta e firmata, ma quando il Delivery Manager puo iniziare il setup senza dover reinterpretare il venduto. Per questo il passaggio tra BD, Presales e Delivery deve essere trattato come un mini-gate operativo.

Il punto piu delicato del modello BuildTrust non e solo l'esistenza delle funzioni, ma la qualita del passaggio di informazioni tra di esse. Le vendite aprono la relazione e qualificano il valore dell'opportunita. Il presales traduce il bisogno in un'ipotesi di soluzione eseguibile. Il prodotto chiarisce che cosa appartiene alla piattaforma e che cosa e richiesta specifica. La delivery prepara il terreno per l'esecuzione reale. L'engineering realizza, misura, documenta e rende ripetibile. Ognuno eredita il lavoro del passaggio precedente ma non dovrebbe mai doverlo reinventare.

Per far funzionare davvero questo flusso servono tre cose: informazioni minime obbligatorie nel passaggio, uno strumento chiaro in cui vengono registrate e un gate che dichiara il passaggio sufficientemente maturo per procedere. Se manca uno di questi tre elementi, il progetto tende a sembrare veloce nelle prime settimane ma paga poi il prezzo in riaperture, aspettative mal allineate e lavoro tecnico che parte senza le premesse giuste.

Area	Contenuto minimo obbligatorio
Scope	Scope IN, scope OUT, ipotesi di soluzione proposta, elementi custom vs prodotto, dipendenze da partner.
Stakeholder	Sponsor, referente operativo, referente tecnico, referente contrattuale, soggetti con potere di approvazione milestone.
Dati e accessi	Fonti dati previste, disponibilit� reale, formati attesi, accessi tecnici promessi, eventuali prerequisiti del cliente.
Governance contrattuale	Milestone, criteri di accettazione, documenti richiesti dal cliente, SLA o vincoli temporali gi� negoziati.
Red flags	Punti non chiusi, ambiguit� emerse, richieste sensibili fuori scope, rischi politici o di allineamento lato cliente.

Da -> A	Informazioni da scambiare	Strumento / artefatto primario
Vendite -> Presales	Contesto cliente, obiettivo, urgenza, attori e primi vincoli.	HubSpot + opportunity brief
Presales -> PM	Lettura piattaforma, gap, richieste candidate a backlog prodotto.	Discovery Note + item backlog
Presales -> DM	Prerequisiti, milestone ipotizzate, rischi di esecuzione, dipendenze.	Handover Pack
PM -> COO	Impatto su roadmap, trade-off di piattaforma e decisioni di priorit�.	Review roadmap / note di triage
DM -> Cliente	Stato, decisioni, impatti, prossimi passi e richieste pendenti.	Weekly Status Note / call di stato
Engineering -> DM e PM	Avanzamento reale, blocchi, impatti di qualit� e scelte tecniche che cambiano scope o tempi.	Azure DevOps + review dedicata

Dove trovare le procedure Template Opportunity Brief: SharePoint > Operations > Template > Opportunity_Brief_v1.docx Discovery Note: SharePoint > Operations > Template > Discovery_Note_v1.docx Handover Pack: SharePoint > [Nome Progetto] > 01_Commerciale > Handover_Pack.docx Pipeline: HubSpot > Sales > Pipeline

12.4.1 Dal primo contatto commerciale alla qualificazione interna

La funzione commerciale non deve limitarsi a capire se esiste un budget o un interesse. Deve arrivare alla prima review interna con una lettura minima della sostanza dell'opportunit . Questo significa chiarire quale problema operativo del cliente si vuole risolvere, quale decisione aziendale il cliente vorrebbe rendere pi  affidabile, quanto il tema tocca il perimetro core di BuildTrust e quali attori interni del cliente influenzeranno davvero l'esito del progetto.

Nel modello futuro la vendita non dovrebbe promettere in modo definitivo una soluzione tecnica, una tempistica o una personalizzazione senza un passaggio strutturato con presales e, quando serve, con prodotto e delivery. Il valore della funzione commerciale non diminuisce per questo; al contrario aumenta, perch  la promessa fatta al cliente diventa pi  credibile e sostenibile. La maturit 

commerciale di una software factory si misura anche dalla sua capacita di dire "questo lo verifichiamo prima di impegnarci".

Informazione minima raccolta da vendite	Perche serve	Dove viene registrata
Problema cliente e obiettivo atteso	Consente di evitare offerte impostate sulla soluzione e non sul valore.	Scheda opportunity in HubSpot
Stakeholder chiave e sponsor	Permette di capire chi decide davvero, chi approva e chi potra bloccare il progetto.	Scheda opportunity + note discovery
Urgenza reale e driver temporale	Serve a distinguere deadline reali da pressioni solo percepite.	Scheda opportunity
Vincoli gia dichiarati dal cliente	Riduce il rischio di scoprire tardi dipendenze critiche di compliance, IT o procurement.	Note discovery / checklist presales
Ipotesi di perimetro	Aiuta presales a impostare la qualificazione senza ripartire da zero.	Opportunity brief

12.4.2 Il ruolo del presales come traduttore esecutivo

Il presales in BuildTrust non dovrebbe essere solo un supporto tecnico alla vendita, ma la funzione che converte una conversazione commerciale in un'ipotesi operabile. Questo passaggio e cruciale perche crea la prima versione strutturata del futuro progetto: perimetro iniziale, ipotesi di soluzione, prerequisiti, dipendenze, possibili componenti di piattaforma gia disponibili e punti che richiederanno scelta manageriale.

La qualita del presales si vede nella capacita di eliminare ambiguita. Un buon presales non produce necessariamente un documento lungo, ma un documento che rende visibili le aree ancora aperte. Se il cliente non ha ancora chiarito disponibilita dati, ownership dei processi o criteri di accettazione, questo deve comparire come rischio o assunzione e non essere nascosto dentro un linguaggio genericamente positivo.

Nel nuovo modello il presales dovrebbe uscire da ogni opportunity qualificata con un pacchetto leggibile da management, product e delivery. Questo pacchetto deve rendere possibili tre decisioni: vale la pena perseguire l'opportunità, la piattaforma e adeguata rispetto al bisogno e il progetto e avviabile nelle condizioni date. Se uno di questi tre punti non e chiaro, il gate non e maturo.

Output del presales	Contenuto atteso	Destinatari principali
Opportunity brief qualificato	Problema, obiettivi, attori, vincoli, perimetro iniziale, ipotesi di soluzione.	Vendite, COO, PM, DM
Mappa dei prerequisiti	Dati, accessi, integrazioni, disponibilit� cliente, eventuali partner.	DM, engineering, COO
Lettura prodotto vs custom	Cosa � gi� coperto dalla piattaforma, cosa richiede configurazione, cosa richiede sviluppo.	PM, COO, vendite
Rischi e assunzioni	Elementi ancora non verificati che possono impattare tempi, costi o sostenibilit�.	COO, CEO, DM

12.4.3 Come deve avvenire il confronto tra presales, prodotto e delivery prima della firma

Uno degli errori pi  costosi per una realt  come BuildTrust   trattare il pre-firma come momento solo commerciale. In realt  la fase pre-firma   gi  un pezzo di governance operativa. Prima di impegnarsi col cliente occorre una discussione mirata tra presales, prodotto e delivery per capire se l'offerta proposta   realmente sostenibile e coerente con la direzione della piattaforma.

Il Product Manager deve leggere l'opportunit  con una domanda di piattaforma: quello che stiamo promettendo rafforza una capability che vorremmo avere, oppure   un adattamento specifico che potrebbe trascinarci in manutenzione non desiderata. Il Delivery Manager legge invece la stessa opportunit  con una domanda di esecuzione: il progetto parte con i prerequisiti giusti, con un modello milestone sensato e con un livello di ambiguit  gestibile. Il COO deve infine comporre le due letture e trasformarle in una decisione di priorit  e sostenibilit .

Il punto non   rallentare il commerciale, ma impedire che la firma di oggi diventi il disordine operativo di domani. Quando questo gate   ben fatto, l'organizzazione pu  accelerare dopo la firma. Quando viene saltato, la velocit  commerciale iniziale viene pagata con settimane di riallineamenti successivi.

Domanda del gate pre-firma	Chi deve rispondere	Criterio di uscita
Il bisogno � allineato al posizionamento BuildTrust?	CEO / vendite / PM	S�, oppure si decide esplicitamente l'eccezione.
La piattaforma copre davvero il cuore del bisogno?	PM / presales	Copertura chiara o piano di gap esplicito.
I prerequisiti di esecuzione sono ragionevoli?	DM / engineering / presales	Rischi noti, owner chiari, dipendenze esplicitate.
La capacit� interna � disponibile?	COO	Allocazione realistica o decisione di sequencing/partner.
Il modello di milestone � difendibile col cliente?	DM / vendite	Milestone comprensibili, verificabili e sostenibili.

12.4.4 Il passaggio dalla firma all'avvio progetto

Dopo la firma non dovrebbe esistere un periodo grigio in cui "ci organizziamo". Il modello BuildTrust di domani dovrebbe prevedere un passaggio ordinato in massimo pochi giorni lavorativi, con owner chiari e un kit di avvio standard. Questo kit non serve a fare documentazione fine a se stessa, ma a permettere al Delivery Manager e al team tecnico di entrare rapidamente in esecuzione senza dover rincorrere informazioni commerciali.

Il passaggio corretto include sempre un kickoff interno, precedente al kickoff cliente. Nel kickoff interno il team BuildTrust allinea scope, milestone, rischi, ruoli, decisioni aperte, asset di piattaforma coinvolti, eventuali componenti custom e dipendenze da cliente o partner. Il kickoff cliente avviene solo quando questa lettura è pronta. In questo modo il primo incontro con il cliente comunica controllo e non ricerca di informazioni che avremmo dovuto già avere.

La regola pratica dovrebbe essere semplice: nulla parte solo perché il contratto è firmato; parte quando il progetto è stato preparato. Questa differenza culturale è fondamentale per passare da startup a software factory.

Artefatto di avvio	Scopo	Owner BuildTrust
Project Charter	Definire perimetro, outcome, ruoli, milestone, governance.	DM
Project board attivo	Tracciare lavoro, issue, dipendenze e milestone.	DM con engineering lead
RAID log iniziale	Rendere subito visibili rischi, assunzioni e blocchi noti.	DM
Backlog iniziale ordinato	Separare subito attività di setup, discovery residua, sviluppo e dipendenze esterne.	DM + engineering lead
Repository e convenzioni di lavoro	Abilitare sviluppo coerente e tracciato fin dal primo giorno.	Engineering lead

12.4.5 Come si scambiano le informazioni tra le funzioni nel lavoro quotidiano

Nel nuovo assetto ogni funzione dovrebbe sapere non solo con chi parlare, ma cosa deve trasferire e in quale momento. Il problema delle organizzazioni in crescita è che spesso il dialogo esiste, ma è troppo informale. Le persone si aggiornano, si aiutano e si mandano messaggi, ma le informazioni essenziali restano distribuite e nessuno è certo di quale sia l'ultima versione affidabile.

Per BuildTrust la disciplina giusta non è moltiplicare i meeting, ma stabilire punti di passaggio chiari. Vendite aggiorna presales e COO quando cambia sostanza dell'opportunità. Presales aggiorna PM e DM quando emergono implicazioni di piattaforma o di delivery. Il PM restituisce una lettura di standardizzazione e priorità. Il DM restituisce una lettura di fattibilità operativa, stato e rischio. L'engineering non parla solo di task completati, ma di impatti, decisioni richieste e qualità del flusso.

Da → A	Informazioni da scambiare	Strumento/artefatto primario
Vendite → Presales	Contesto cliente, obiettivo, urgenza, attori e primi vincoli.	HubSpot + opportunity brief
Presales → PM	Lettura piattaforma, gap, richieste candidate a backlog prodotto.	Scheda analisi + item backlog discovery
Presales → DM	Prerequisiti, milestone ipotizzate, rischi di esecuzione, dipendenze.	Handover pack
PM → COO	Impatto su roadmap, trade-off con piattaforma, decisioni di priorit�.	Review roadmap / note di triage
DM → Cliente	Stato, decisioni, impatti, prossimi passi, richieste pendenti.	Weekly Status Note / call di stato
Engineering → DM e PM	Avanzamento reale, blocchi, impatti di qualita, scelte tecniche che cambiano scope o tempi.	Azure DevOps + review dedicata

Il modo di comunicare conta quanto il contenuto. In una software factory matura le funzioni non si limitano a passarsi informazioni: le passano in modo leggibile, sintetico e azionabile. Questo significa che ogni comunicazione operativa importante dovrebbe contenere almeno quattro elementi: fatto osservato, impatto, decisione o richiesta, owner del prossimo passo. Un messaggio privo di impatto o privo di owner genera rumore invece che coordinamento.

Teams puo essere usato per accelerare il confronto, ma non deve diventare il luogo finale di verita. Se una decisione cambia perimetro, priorit , readiness o responsabilit , allora deve rientrare nello strumento giusto: work item, RAID, charter, note di decisione o documento di governance. Questo   il modo concreto con cui BuildTrust puo restare veloce senza tornare opaca.

- Le chat servono per coordinare rapidamente, non per custodire decisioni importanti.
- Le mail verso il cliente formalizzano, ma la preparazione dello stato avviene sugli strumenti interni.
- Ogni cambio rilevante di scope, milestone o dipendenza deve lasciare traccia in un artefatto stabile.
- Ogni escalation deve essere anticipata internamente prima di essere percepita dal cliente.

14. Processo di delivery di progetto

14.1 Obiettivo e responsabilit 

Eseguire il progetto in modo prevedibile, con controllo continuo su milestone, dipendenze, evidenze e qualita dell'avanzamento. Il Delivery Manager   il responsabile end-to-end. Il COO   il supervisor operativo. Il team Engineering esegue.

Il Delivery Manager in BuildTrust non governa solo task, ma coerenza tra contratto, execution tecnica, stakeholder, evidenze e narrativa verso il cliente. Questo significa che la lettura dello stato non puo dipendere dalla memoria individuale o da chat disperse.

Il progetto è sano quando il board riflette davvero il lavoro in corso, le dipendenze esterne sono visibili, le milestone hanno criteri di accettazione leggibili e i blocchi vengono dichiarati prima di trasformarsi in emergenza.

14.2 Fasi operative

- **Project Kickoff Interno.** Prima dell'avvio formale con il cliente il DM convoca il team BuildTrust coinvolto. Obiettivo: tutti hanno letto il pacchetto di handover, i ruoli sono chiari, milestone e criteri di acceptance sono leggibili, il board Azure DevOps è configurato, il RAID log è aperto. Output: Project Charter firmato internamente.
- **Kickoff con il Cliente.** Prima riunione formale con stakeholder del cliente. Presentazione team, piano di lavoro, milestone e acceptance flow. Definizione referenti, frequenza degli aggiornamenti, canale di comunicazione primario (Teams channel dedicato). Verbale su SharePoint.
- **Detailed Setup.** Il Cloud Engineer configura gli ambienti tecnici (dev, staging, prod su Azure). Il Data Engineer avvia l'onboarding delle fonti dati e configura le pipeline di ingestione. I partner vengono invitati al progetto in Azure DevOps con perimetro definito. Il DM verifica che tutti gli accessi siano attivi.
- **Execution Cycle.** Ciclo iterativo organizzato in sprint di 2 settimane o in milestone track secondo il contratto. I task sono tracciati in Azure DevOps Boards. Il DM aggiorna il Weekly Status Note e lo invia al cliente ogni lunedì. Il COO partecipa al Biweekly Delivery Deep Dive.
- **Milestone Verification.** Al raggiungimento di ogni milestone il Data Engineer chiude la raccolta evidenze. Il Delivery Manager verifica la completezza del pacchetto secondo il Milestone & Evidence Register, con supporto del team tecnico sui contenuti specialistici. Il DM presenta il pacchetto al cliente e avvia il processo di accettazione.
- **Formal Acceptance.** Accettazione formale secondo l'Acceptance Flow concordato con il cliente. Output minimo: documento o evidenza di accettazione firmata o confermata attraverso il canale previsto, aggiornamento milestone in Azure DevOps a 'Closed/Accepted', eventuale attivazione di funzionalità di certificazione della piattaforma solo se previste dal caso d'uso cliente.
- **Closure / Handover.** Closure Memo su SharePoint: obiettivi iniziali vs risultati, milestone raggiunte, problemi incontrati, learnings, debiti tecnici aperti, input per il backlog prodotto. Passaggio al team di supporto se previsto dal contratto.

14.3 Artefatti obbligatori per progetto

Artefatto	Owner	Strumento	Frequenza aggiornamento
Project Charter	Delivery Manager	SharePoint > [Progetto] > 03_Delivery	Avvio progetto
Work Plan / Sprint Board	Delivery Manager	Azure DevOps Boards	Continuo
Stakeholder Map	DM + Presales	SharePoint > [Progetto] > 03_Delivery	Avvio + aggiornamento ad ogni cambio
RAID Log	Delivery Manager	Azure DevOps – Issues RAID	Settimanale
Milestone & Evidence Register	DM + Data & AI Engineer	Azure DevOps + SharePoint	Ad ogni milestone
Weekly Status Note	Delivery Manager	SharePoint + Teams (invio cliente)	Lunedì ogni settimana
Closure Memo	Delivery Manager	SharePoint > [Progetto] > 03_Delivery	Fine progetto

Configurazione Azure DevOps per ogni progetto Ogni progetto cliente ha un proprio Project in Azure DevOps con: Board Kanban (colonne: Backlog | In Progress | Review | Done), Work item types attivi: Task, Bug, Issue (RAID), Milestone, Evidence Item. La naming convention e: [Anno]-[Sigla cliente]-[Tipo] (es: 2026-INET-Platform). Il DM e il Project Administrator del board. Il COO ha accesso in sola lettura a tutti i board.

14.3.1 Guida pratica all'uso del board di progetto

Il board di progetto deve permettere di capire lo stato senza ricostruzioni verbali. Per questo il Delivery Manager dovrebbe usarlo con quattro regole semplici. Primo: ogni item rilevante per milestone, issue, dipendenza o evidenza deve essere visibile. Secondo: lo stato dell'item deve riflettere il lavoro reale e non l'intenzione. Terzo: le dipendenze esterne non devono vivere in chat o mail isolate, ma comparire come item o issue con owner e data attesa. Quarto: tutto ciò che può cambiare il messaggio al cliente deve essere leggibile dal board o dai suoi collegamenti.

La guida pratica per il team è questa: il board non sostituisce il giudizio del DM, ma lo rende verificabile. Se il board è pieno di task aperti ma non si capisce quale milestone sia a rischio, il board è usato male. Se il board mostra pochi item ma le riunioni sono piene di sorprese, il board è troppo povero. Il punto corretto è avere abbastanza dettaglio da leggere avanzamento, blocchi ed evidenze, senza trasformarlo in un elenco burocratico di micro-attività.

14.4 Come leggere la salute reale di un progetto

Un progetto può apparire molto attivo, con molte call, molti task aperti e molte persone coinvolte, ma essere in realtà in peggioramento. La vera salute si vede nella leggibilità del board, nella prevedibilità delle milestone, nella chiarezza delle dipendenze esterne e nella qualità della narrativa verso il cliente.

Segnale	Cosa indica	Azione consigliata
Troppi item fermi in In Progress	Multitasking eccessivo o blocchi non dichiarati.	Ridurre WIP, chiarire priorit�, aprire issue dove serve.
Milestone discusse solo a ridosso della scadenza	Assenza di previsione reale e di readiness progressiva.	Introdurre review pre-milestone e ownership esplicita delle evidenze.
Dipendenze cliente senza owner	Il progetto dipende da fattori esterni non governati.	Rendere la dipendenza visibile in RAID e riallinearla con il cliente.
Partner silenziosi o opachi	Rischio di slittamento nascosto e perdita di controllo.	Fare checkpoint formale e verificare output intermedi.
Feature request da progetto non instradate	Confusione crescente tra delivery e prodotto.	Aprire subito work item nel backlog Platform o classificare come custom.
Weekly Status Note poco chiaro	Il DM non ha una narrativa stabile dello stato.	Ricostruire lo stato a partire da board, issue e milestone.

14.5 Comunicazione cliente, milestone e readiness

BuildTrust non vende solo tecnologia: vende anche affidabilit  operativa. Questo significa che il modo in cui comunica con il cliente durante il progetto   parte integrante del valore percepito. Un progetto tecnicamente solido ma comunicato male puo perdere fiducia piu rapidamente di un progetto con difficolt  dichiarate in modo trasparente e governato.

La narrativa di progetto non dovrebbe nascere improvvisata il giorno della call col cliente. Dovrebbe essere il risultato di una lettura ordinata di board, issue, milestone, evidenze e decisioni aperte. Ogni Weekly Status Note dovrebbe consentire al cliente di capire quattro cose: cosa   stato fatto, cosa cambia per il progetto, cosa serve da lui e quali rischi o decisioni meritano attenzione.

Un principio utile   distinguere sempre tra informazione, interpretazione e decisione. Informazione: stato dei task, evidenze raccolte, issue aperte. Interpretazione: cosa questo significa per milestone, tempi, qualita o dipendenze. Decisione: cosa BuildTrust propone di fare ora. Questa distinzione rende la comunicazione pi  adulta, pi  chiara e pi  credibile.

- Non comunicare al cliente un blocco senza proporre anche una lettura dell'impatto e una strada di gestione.
- Non nascondere scostamenti piccoli nella speranza di recuperarli senza parlarne.
- Non usare linguaggio tecnico per coprire un problema di chiarezza manageriale.

Momento	Verifiche minime obbligatorie
Kickoff interno	Project Charter letto dal team; ruoli chiari; board attivo; RAID aperto; milestone nominate; stakeholder cliente identificati; ipotesi critiche esplicitate.
Kickoff cliente	Referenti confermati; canale di comunicazione definito; frequenza status concordata; piano di accessi approvato; logica di acceptance spiegata.
Pre-milestone	Task critici chiusi o chiaramente residui; evidence item raccolti; criteri di accettazione verificati; narrativa di presentazione preparata; issue aperte con impatto noto.
Go-live tecnico	Ambiente monitorato; rollback o piano di rientro definito; ownership supporto chiara; release note e runbook aggiornati; stakeholder informati.
Go-live operativo	Utenti o referenti cliente allineati; canale supporto attivo; backlog di ottimizzazioni separato dai bug; DM informato degli impatti del rilascio.

14.6 Sequenza manageriale end-to-end del progetto

Per essere davvero utile ai manager e ai tecnici, il TOM deve spiegare non solo chi fa cosa, ma come il progetto si muove nel tempo. In BuildTrust il progetto tecnico dovrebbe essere governato come una sequenza di fasi leggibili, ciascuna con un obiettivo preciso, un set minimo di artefatti, un criterio di uscita e un tipo di attenzione manageriale.

Fase del progetto	Domanda manageriale corretta	Segnale da leggere
Mobilizzazione	Siamo davvero pronti a partire o stiamo ancora inseguendo il venduto?	Board, ruoli e prerequisiti iniziali sono chiari.
Discovery	Le ambiguità stanno diminuendo o si stanno spostando avanti?	Le assunzioni si chiudono con owner e date.
Design	La soluzione è sostenibile per piattaforma e delivery?	Scelte tecniche leggibili e criteri di done chiari.
Build	Il flusso è sano o stiamo solo accumulando lavoro in corso?	WIP, cycle time, blocchi e review in coda.
Verifica	Siamo pronti a dimostrare il valore o solo a dichiararlo?	Evidenze complete e acceptance preparata.
Go-live	L'organizzazione sa assorbire il rilascio?	Runbook, osservazione post-release e owner del supporto.
Closure	Abbiamo chiuso solo l'attività o anche l'apprendimento?	Closure memo e learnings riutilizzabili.

14.6.1 Fase di mobilitazione e setup

La mobilitazione comincia appena il progetto entra realmente nel perimetro operativo. In questa fase il Delivery Manager deve trasformare il materiale commerciale e presales in una macchina di lavoro pronta a partire. Gli obiettivi non sono ancora sviluppare, ma allineare ruoli, creare il board, chiarire milestone, aprire il RAID iniziale, verificare accessi, validare dipendenze e costruire una prima narrativa condivisa all'interno del team.

Se questa fase è fatta bene, il progetto parte con un linguaggio comune. Se è fatta male, i problemi emergono subito sotto forma di task mal definiti, domande già risolte che riemergono, attività urgenti che non trovano owner e continue richieste di chiarimento verso commerciale o presales. La mobilitazione ha quindi una funzione protettiva: abbassa il debito di ambiguità con cui il progetto entra nella delivery.

Fase	Cosa deve succedere	Segnale di completamento
Mobilitazione	Ruoli, scope, milestone e board iniziale sono allineati internamente.	Kickoff interno svolto con action item chiusi.
Setup strumenti	Repository, board, cartelle documentali e template sono attivi.	Il team sa dove guardare e dove scrivere.
RAID iniziale	Rischi, assunzioni, issue e dipendenze già note sono esplicitate.	Esiste una vista iniziale dei punti di attenzione.
Preparazione kickoff cliente	Narrativa di avvio, stakeholder e prerequisiti sono pronti.	Il kickoff cliente non viene usato per improvvisare.

14.6.2 Fase di discovery operativa e chiarimento requisiti

Anche quando la vendita è stata fatta bene, quasi tutti i progetti tecnici hanno una quota di discovery residua. BuildTrust dovrebbe trattarla come fase esplicita e non come attività nascosta dentro i primi task di sviluppo. La discovery operativa serve a trasformare ipotesi in dati utilizzabili: quali fonti esistono davvero, quali regole di business sono stabili, quali utenti o stakeholder prenderanno parte alla validazione, quali componenti di piattaforma saranno usate e quali no.

La cosa importante è che la discovery non resti un esercizio indefinito. Ogni tema aperto dovrebbe avere owner, domanda da chiudere, data attesa e impatto sull'esecuzione. Se manca questo rigore, la discovery si trasforma in un sottofondo continuo che erode capacità e impedisce di distinguere tra lavoro di chiarimento e lavoro di costruzione.

Per i manager questa fase richiede una vigilanza particolare: non va accelerata in modo cieco, ma nemmeno lasciata aperta senza disciplina. Il criterio corretto non è avere tutte le risposte, ma avere abbastanza chiarezza per iniziare il lavoro successivo in modo governato.

14.6.3 Fase di progettazione della soluzione

Una volta chiariti i punti essenziali, il progetto entra nella progettazione della soluzione. Qui il team traduce il problema in un disegno tecnico e operativo: architettura coinvolta, integrazioni, configurazioni, mapping dati, regole di deployment, criteri di test e forma delle evidenze. La progettazione non deve diventare documentazione eccessiva, ma nemmeno restare tutta nella testa degli specialisti. Per BuildTrust è sufficiente un livello di progettazione che renda chiaro cosa verrà costruito, con quali dipendenze e con quali principali scelte.

In questa fase il Product Manager, se coinvolto, ha un ruolo di protezione della piattaforma. Deve verificare che la soluzione non introduca scorciatoie tecniche che sembrano utili sul progetto ma che aumentano complessità futura per il prodotto. Il Delivery Manager invece deve assicurarsi che la soluzione sia traducibile in task, milestone e aspettative leggibili per il cliente.

Per i tecnici la progettazione è anche il momento in cui chiarire i criteri di done. Se il progetto richiede non solo codice ma anche configurazioni, test di integrazione, evidenze documentali o aggiornamento di runbook, tutto questo va reso esplicito prima di entrare pienamente in sviluppo.

14.6.4 Fase di costruzione e integrazione

La fase di costruzione e integrazione è quella in cui il team tecnico produce effettivamente software, configurazioni, mapping e collegamenti con i sistemi esterni. Il rischio qui è considerarla una corsa individuale alla chiusura dei task. In BuildTrust invece dovrebbe essere letta come fase di flusso: il lavoro deve restare piccolo, visibile, prioritizzato e costantemente collegato all'esito di progetto o di prodotto a cui contribuisce.

Il Delivery Manager non deve entrare nel merito di ogni scelta tecnica, ma deve poter leggere se il flusso è sano: quanti item sono aperti, quali sono bloccati, quanto invecchiano le issue, quali task impattano la milestone e quali dipendenze esterne stanno rallentando il team. Il PM osserva invece se lo sviluppo sta assorbendo correttamente il lavoro prodotto o se il backlog viene continuamente destabilizzato da richieste reattive.

Per il team tecnico la qualità del lavoro in questa fase dipende soprattutto da tre discipline: task ben definiti, branch brevi con PR leggibili e feedback rapido su bug o problemi di integrazione. Se una di queste tre discipline salta, aumenta subito il debito di coordinamento.

14.6.5 Fase di verifica, UAT e preparazione evidenze

Molti progetti sembrano vicini alla chiusura tecnica ma arrivano impreparati alla verifica. Questo succede quando test, UAT ed evidenze vengono lasciati alla fine come passaggio amministrativo. In BuildTrust dovrebbero invece essere preparati progressivamente. Ogni milestone dovrebbe avere sin dall'inizio una lista di prove attese e un piano minimo di validazione. In questo modo, mentre il lavoro procede, il team raccoglie già ciò che servirà per dimostrare il risultato.

Il Delivery Manager ha qui una responsabilità chiave: evitare che la milestone venga raccontata solo in termini di task chiusi. Il cliente e il management non comprano task; leggono avanzamento, risultato e affidabilità. Per questo il pacchetto di verifica deve spiegare cosa è stato realizzato, quali test o controlli sono stati eseguiti, quali limiti o prerequisiti restano e quale forma di accettazione si chiede.

Per i tecnici questa fase significa anche preparare l'organizzazione al rilascio. Se una soluzione funziona ma richiede monitoraggio, interventi manuali o configurazioni sensibili, questi aspetti devono

essere resi visibili prima della produzione.

14.6.6 Fase di go-live e stabilizzazione

Il go-live è un momento breve ma ad alta densità di rischio. In BuildTrust non dovrebbe essere trattato come passaggio solo del team tecnico. Il rilascio coinvolge delivery, supporto, eventuali partner e talvolta il cliente stesso. Questo significa che oltre alla readiness tecnica occorre verificare readiness informativa: chi sa che cosa sta per essere rilasciato, chi osserva il comportamento nelle ore successive, come si gestisce un rollback o una escalation.

La stabilizzazione successiva al go-live serve a evitare che il progetto dichiari successo troppo presto. Le prime 24-72 ore dovrebbero essere lette come finestra di osservazione: bug emersi, comportamenti inattesi, necessità di supporto, adozione lato cliente, tenuta dei dati o delle integrazioni. Solo dopo questa lettura il team può veramente dire di aver portato il progetto oltre il semplice deployment.

14.6.7 Chiusura, apprendimento e passaggio a supporto o continuità

La chiusura di progetto non coincide con l'ultimo task chiuso. In BuildTrust dovrebbe comprendere una verifica formale di cosa è stato consegnato, un riepilogo di milestone ed evidenze, la raccolta dei principali learning, il trasferimento di informazioni a supporto o al team interno che erediterà la soluzione e una lettura onesta di cosa ha funzionato e cosa no nel processo.

Questa fase è essenziale perché trasforma l'esperienza del singolo progetto in patrimonio organizzativo. Se ogni progetto si chiude senza lasciare un miglioramento a template, checklist, toolchain o regole di lavoro, l'organizzazione continuerà a pagare gli stessi errori. Una software factory matura cresce anche così: non solo consegnando, ma imparando in modo esplicito.

15. Processo di sviluppo prodotto

15.1 Obiettivo

Far evolvere la piattaforma BuildTrust come prodotto coerente, modulare e difendibile nel tempo, senza che la roadmap sia guidata dall'urgenza dei singoli progetti. Lo sviluppo prodotto segue un modello ibrido: milestone strategiche per scandire le release principali, Kanban board per la gestione operativa del flusso di lavoro quotidiano. Il Product Manager è il custode di questa coerenza e risponde a Stefano Gelain (COO).

In chiave manageriale, il Product Manager dovrebbe proteggere la piattaforma da due rischi opposti: la deriva custom, che fa entrare ogni bisogno cliente nella roadmap, e l'astrazione eccessiva, che ignora i pattern reali emersi nei progetti. Ogni richiesta nata da un progetto deve quindi essere letta come segnale e non come comando: la domanda corretta non è solo se serve al cliente attuale, ma se rafforza una capability riusabile, sostenibile e coerente con l'architettura di BuildTrust.

Il backlog di prodotto deve restare comprensibile nel tempo. Ogni feature dovrebbe spiegare problema, valore atteso, ipotesi di riuso, impatto tecnico e motivo della priorità. Quando questo livello di chiarezza manca, il management perde la capacità di capire se sta finanziando prodotto, supportando delivery o assorbendo custom travestito da roadmap.

15.2 Classificazione delle richieste di sviluppo

Ogni richiesta che arriva al team di prodotto deve essere classificata prima di essere presa in carico. La classificazione determina il routing e impedisce che i progetti cliente snaturino la roadmap.

Tipo	Descrizione	Routing	Board di destinazione
Bug / Correzione	Anomalia rispetto al comportamento atteso della piattaforma	PM – priorità critica o alta	Azure DevOps > Platform > Bug backlog
Configurazione / Parametrizzazione	Adattamento di una istanza cliente che non richiede sviluppo	DM – gestisce direttamente	Azure DevOps > [Progetto cliente] > Task
Estensione prodotto (riuso atteso)	Nuova funzionalità nata da un progetto ma riutilizzabile su più clienti	PM – valutata per roadmap e milestone	Azure DevOps > Platform > Backlog, tag: da-progetto
Custom non productizzabile	Sviluppo specifico per un cliente, senza valore di generalizzazione	DM – componente di progetto, non di prodotto	Azure DevOps > [Progetto cliente] > Backlog
Innovazione AI / modelli	Nuovi modelli predittivi, pipeline AI, sperimentazione algoritmica	Data & AI Engineer + PM – valutata in Product Council	Azure DevOps > Platform > AI Backlog

15.2.1 Quattro domande per distinguere prodotto da custom

Questo punto merita particolare attenzione perché da qui passano molte inefficienze. Una richiesta nata in un progetto non deve entrare automaticamente nel backlog di prodotto, ma nemmeno sparire in una soluzione ad hoc decisa dal solo team tecnico. Il passaggio corretto è registrarla come richiesta, chiarire il contesto in cui nasce, valutarne il potenziale di riuso e decidere se resta custom, se entra in roadmap o se va scartata.

Domanda	Se la risposta è SÌ	Se la risposta è NO
Serve a più clienti o a una capability core di BuildTrust?	Tende a prodotto.	Tende a custom di progetto.
Riduce complessità futura o evita rework ricorrente?	Ha valore di piattaforma e merita valutazione roadmap.	Resta nel perimetro del progetto.
Dipende da dati, regole o processi troppo specifici di un cliente?	Rischia di diventare custom travestito da prodotto.	Può essere standardizzato più facilmente.
Siamo pronti a mantenerla nel tempo come parte della piattaforma?	Può entrare in backlog prodotto.	Meglio trattarla come custom o prototipo.

15.3 Modello milestone + Kanban

Lo sviluppo prodotto è organizzato su due livelli sovrapposti: le milestone di prodotto definiscono i traguardi strategici della piattaforma su orizzonte trimestrale; la Kanban board gestisce il flusso quotidiano di lavoro del team Engineering senza interruzioni di contesto.

15.3.1 Milestone di prodotto

Le milestone di prodotto sono definite dal Product Manager in accordo con il COO nel Monthly Product & Capacity Council. Ogni milestone ha: una data target, un insieme di feature che la compongono, criteri di accettazione verificabili, un responsabile tecnico per ogni feature. Le milestone sono visibili in Azure DevOps come work item di tipo Milestone nel progetto Platform, collegate alle Feature e alle User Story che le compongono tramite la gerarchia Epic > Feature > User Story > Task.

Livello gerarchia	Tipo work item Azure DevOps	Chi lo gestisce	Orizzonte
Obiettivo strategico	Epic	Product Manager	Semestrale / annuale
Traguardo di release	Milestone	Product Manager + COO	Trimestrale
Funzionalità	Feature	Product Manager	6-8 settimane
Requisito implementabile	User Story	PM + Developer	1-2 sprint (2-4 settimane)
Unità di lavoro	Task	Developer / AI Engineer	1-5 giorni
Anomalia	Bug	Chiunque la rileva – triage PM	Priorità immediata

15.3.2 Kanban board operativa

Il lavoro quotidiano del team Engineering è gestito su una Kanban board dedicata nel progetto Platform di Azure DevOps. La board ha colonne standard che riflettono il flusso reale di sviluppo:

Colonna	Significato	Limite WIP raccomandato
Backlog	Item prioritizzati, pronti per essere presi in carico	Illimitato
Ready	Item con criteri di accettazione chiari, dipendenze risolte	4 per developer
In Progress	Sviluppo attivo	1-2 per developer (focus)
Code Review / PR	Pull Request aperta, in attesa di review	2 per developer
Testing / QA	Test funzionali e quality gate SonarCloud	2 per developer
Done	Merge completato, deploy in staging	–

I limiti WIP (Work In Progress) sono la regola più importante della Kanban board: impediscono l'accumulo di lavoro in corso non finito, rendono visibili i colli di bottiglia e migliorano il cycle time. Il PM è responsabile di far rispettare i limiti WIP. Se una colonna supera il limite, il team risolve prima di

iniziare nuovi item.

15.3.3 Guida pratica manageriale all'uso del board di prodotto

Il board di prodotto non dovrebbe essere usato come lista indifferenziata di desideri. Deve rendere leggibile almeno quattro cose: quali item sono davvero pronti, quale lavoro è in corso, dove si stanno formando i colli di bottiglia e quanto il flusso è disturbato da richieste non pianificate. Il PM dovrebbe poter leggere dal board non solo il volume del lavoro, ma la qualità del focus del team.

La regola pratica è distinguere chiaramente tre piani. Il backlog raccoglie la domanda ordinata ma non ancora in esecuzione. Il board mostra solo ciò che il team sta davvero assorbendo. Le milestone di prodotto danno il riferimento strategico di rilascio. Se questi tre piani si confondono, il management non capisce più se il problema sia di priorità, di capacità o di disciplina del flusso.

15.3.4 Come usare davvero il board di prodotto

Il board di prodotto dovrebbe rappresentare il lavoro della piattaforma e non il lavoro di ogni cliente. Qui trovano posto epiche, feature, capability trasversali, bug di piattaforma, attività di hardening, miglioramenti tecnici e debito. Il Product Manager lo usa per ordinare domanda e capacità. L'engineering lo usa per rendere visibile il flusso reale. Il COO lo usa per capire se la roadmap è governata o continuamente interrotta.

Perché il board di prodotto resti leggibile, ogni item dovrebbe avere una formulazione che spiega il problema e non solo l'intervento. Anche quando il lavoro è molto tecnico, il board deve restare interpretabile da chi governa prodotto e delivery.

15.3.5 Gestione parallela prodotto e progetti pipeline

Quando i progetti cliente sono attivi in parallelo, la capacità del team Engineering è divisa tra sviluppo prodotto e delivery di progetto. Il COO (Stefano Gelain) presidia questa allocazione nel capacity board di Azure DevOps con vista settimanale per risorsa.

Scenario	Allocazione Engineering raccomandata	Presidio
Nessun progetto attivo	80% prodotto, 20% manutenzione e tech debt	PM
1 progetto attivo in setup	60% prodotto, 40% progetto	COO + PM + DM
1 progetto in esecuzione piena	50% prodotto, 50% progetto	COO + PM + DM
2 progetti attivi in parallelo	30% prodotto, 70% progetti	COO – revisione settimanale obbligatoria
3 progetti attivi (max governato)	Product backlog congelato tranne bug critici	COO – alert capacità

Con 3 progetti attivi in parallelo lo sviluppo prodotto si ferma tranne i bug critici. Questa non è una scelta da subire: è una policy esplicita che il COO comunica al PM e al team prima che la situazione si verifichi, non dopo.

15.4 Fasi operative del ciclo di sviluppo

- **Intake richieste.** Le richieste arrivano da tre fonti: DM (durante i progetti), BD/Presales (dal mercato), team Engineering (osservazioni tecniche e AI). Ogni richiesta viene aperta come work item in Azure DevOps nel progetto Platform. Non si accettano richieste verbali senza issue aperta.
- **Triage settimanale.** Il PM rivede le nuove richieste ogni lunedì (30 min). Classifica ogni item secondo la tassonomia, assegna il tipo corretto, verifica la completezza della descrizione e instrada nel backlog appropriato. Le richieste custom vengono restituite al DM entro 24h.
- **Prioritization e milestone planning.** Il PM mantiene il backlog Platform ordinato per priorit . Le User Story vengono associate alla milestone di destinazione. La prioritizzazione viene rivista e validata nel Monthly Product & Capacity Council con il COO.
- **Sprint planning (se sprint) o pull dal backlog (se Kanban).** Per le milestone principali il team puo lavorare in sprint da 2 settimane con planning esplicito. Per il flusso ordinario si usa il pull Kanban: il developer prende il prossimo item Ready quando ha capacita disponibile, senza cerimonie di planning settimanale.
- **Sviluppo e review.** Il developer crea un feature branch da develop (naming: feature/[task-id]-[descrizione-breve]), sviluppa, apre una Pull Request. La PR richiede almeno un reviewer tecnico e il superamento del quality gate SonarCloud. Nessuna PR viene mergiata con quality gate rosso.
- **Release readiness.** Quando una milestone e completa: tutti i task Done, test di regressione passati, changelog aggiornato, documentazione tecnica su SharePoint aggiornata, DM attivi informati sulle nuove funzionalita. Il deploy in produzione richiede approvazione esplicita del PM o del COO in Azure DevOps Pipelines.
- **Retrospettiva di milestone.** Sessione di 45 minuti al termine di ogni milestone. Input: metriche del ciclo (velocity, cycle time, bug rate, interrupt work). Output: learnings nel decision log, aggiustamenti al processo, input per la milestone successiva.

15.5 KPI di sviluppo e qualita

Il monitoraggio della salute del processo di sviluppo si basa su metriche concrete, estratte automaticamente da Azure DevOps e SonarCloud e visualizzate nella dashboard Power BI del COO.

KPI	Definizione	Fonte	Target orientativo	Frequenza
Velocity	Story point completati per sprint o per settimana Kanban	Azure DevOps – sprint report	Stabile o in crescita	Per sprint / settimanale
Cycle time	Giorni medi da In Progress a Done per User Story	Azure DevOps – Analytics	< 5 giorni per User Story media	Settimanale
Lead time	Giorni medi da apertura issue a Done	Azure DevOps – Analytics	Riduzione progressiva	Settimanale

KPI	Definizione	Fonte	Target orientativo	Frequenza
WIP medio	N. medio di item In Progress per developer	Azure DevOps – Kanban metrics	<= 2 per developer	Settimanale
Bug rate post-release	N. bug aperti nelle 2 settimane dopo ogni deploy in prod	Azure DevOps – Bug items	< 2 per release	Per release
Code coverage	% copertura test automatici	SonarCloud	Soglia minima definita dal PM (es. 70%)	Per PR / per release
Technical debt ratio	% debito tecnico stimato da SonarCloud	SonarCloud	< 5% (grade A o B)	Per release
Interrupt work %	% task non pianificati sul totale completato	Azure DevOps – tag interrupt	< 20%	Per sprint / mensile
Milestone on time %	% milestone di prodotto chiuse entro la data target	Azure DevOps – Milestone items	> 80%	Per milestone

Come leggere le metriche in Azure DevOps Azure DevOps > Platform > Analytics > Overview: velocity chart, cycle time chart, cumulative flow diagram (CFD). Il CFD è il più utile per vedere i colli di bottiglia Kanban: se una colonna si espande nel tempo, il WIP limit non viene rispettato o c'è un blocco non segnalato. SonarCloud: [sonarcloud.io/organizations/\[buildtrust-org-key\]](https://sonarcloud.io/organizations/[buildtrust-org-key]) > Platform > Dashboard per code coverage e technical debt in tempo reale. (L'URL esatto va configurato al momento del provisioning dell'organizzazione su SonarCloud.)

15.6 Ruolo del Data & AI Engineer nello sviluppo prodotto

Il Data & AI Engineer non è solo una risorsa di supporto ai progetti cliente. È una figura attiva nello sviluppo della piattaforma, responsabile della componente di intelligenza artificiale e data pipeline del prodotto. Le sue aree di contributo specifiche sono:

- Sviluppo e manutenzione dei modelli AI integrati nella piattaforma (modelli predittivi per rischio progetto, anomaly detection su dati di campo, analisi avanzamento).
- Progettazione e implementazione delle pipeline dati interne alla piattaforma: ingestione, normalizzazione, trasformazione e disponibilità dei dati per i moduli applicativi.
- Sperimentazione e validazione di nuovi approcci AI/ML da integrare nel roadmap prodotto, in collaborazione con il PM e con Nicola Bombieri (AI Subject Matter Expert).
- Gestione della qualità e della tracciabilità dei dati di progetto ai fini della certificazione delle milestone, coordinandosi con il Delivery Manager.

Il lavoro AI/ML del Data & AI Engineer è tracciato nel backlog Platform di Azure DevOps in un'area dedicata (AI Backlog), con gli stessi standard di qualità del resto del prodotto: PR review, quality gate, documentazione tecnica su SharePoint.

Come aprire una Feature Request da un progetto 1. Azure DevOps > Platform > Backlog. 2. Nuovo work item: tipo User Story o Feature. 3. Titolo: [SIGLA-CLIENTE] - Descrizione sintetica del bisogno. 4. Descrizione: problema da risolvere (non la soluzione tecnica). 5. Tag: da-progetto, [sigla-cliente]. 6. Assegna al PM. 7. Notifica in Teams con link al work item. Non via mail, non verbalmente.

16. Processo di gestione partner e fornitori

16.1 Obiettivo

Utilizzare i partner come leva di capacita e specializzazione senza perdere il controllo sul risultato e senza cedere know-how strategico all'esterno. BuildTrust rimane la regia di ogni progetto, anche quando la maggior parte del lavoro tecnico e eseguita da partner.

16.2 Cinque principi non negoziabili

- Ogni partner ha sempre un owner interno BuildTrust, unico responsabile della relazione e dei risultati.
- Il backlog assegnato al partner nasce da priorita BuildTrust, non e definito dal partner stesso.
- Artefatti e documentazione prodotti dal partner rientrano in casa BuildTrust al termine di ogni fase.
- Le review dei deliverable partner sono sempre condotte internamente prima dell'accettazione formale.
- Il partner non definisce autonomamente il perimetro della soluzione: questa e responsabilita esclusiva BuildTrust.

16.3 Fasi operative

- Need Definition. Il COO o il DM identifica il gap di capacita che giustifica il ricorso al partner. Si verifica se esiste un partner gia attivo e idoneo o se e necessaria una nuova selezione.
- Work Package Definition. L'owner interno formalizza il Work Package: scope preciso, deliverable attesi con criteri di accettazione, timeline, modalita di reporting (checkpoint bisettimanali), accessi richiesti (Azure DevOps, SharePoint), formato della documentazione finale.
- Onboarding Partner. Il partner viene invitato nel progetto Azure DevOps con ruolo 'Contributor' limitato al proprio board. Riceve il briefing del Project Charter e della parte di scope, milestone, dipendenze ed evidenze rilevante per il suo perimetro. L'owner interno conduce il briefing e verifica la comprensione degli obiettivi.
- Execution e Checkpoints. Il partner lavora secondo il work package. Checkpoint bisettimanale obbligatorio con l'owner interno: aggiornamento stato, blocchi, issue. I checkpoint vengono registrati come commenti sulla issue del work package in Azure DevOps.
- Review and Acceptance. L'owner interno conduce la review di ogni deliverable prima dell'accettazione. Per i deliverable tecnici la review e condotta con il Cloud Engineer o il Full Stack Developer. Per i deliverable di analisi o documentazione con il DM o il PM.
- Knowledge Capture. Al termine di ogni fase o progetto il partner produce la documentazione tecnica del lavoro svolto secondo il template BuildTrust. L'owner interno verifica la completezza e carica la documentazione su SharePoint nella cartella del progetto.

Partner	Ambito	Owner Interno	Azione prioritaria
ID-Group	BIM	Robert Armellini	Owner già nominato – formalizzare work package
Blockchain Italia	Blockchain / smart contract	Veronica Paternolli	Owner già nominato – allineare a regole progetto e handover standard
Maxfone	Digital Plant	Robert Armellini	Owner già nominato – formalizzare work package
Orienta + Trium	CDE	Da nominare	Priorità: nominare owner entro 30 giorni
StatWolf	AI e database	Anna Marazzan	Owner già nominato – formalizzare knowledge capture
Factoryal	Data integration	Data Engineer TO-BE	Dipende da hiring
Ipsium Lab	Web	Full Stack Developer	Assegnare a FSD TO-BE
XCF	UX	Product Manager	Assegnare a PM TO-BE

Dove trovare le procedure Work Package template: SharePoint > Operations > Template > Partner_WorkPackage_v1.docx Partner Register: SharePoint > Operations > Partner_Register.xlsx (aggiornato dal COO) Checkpoint report: Azure DevOps > [Progetto] > Issues > [Work Package Issue] > Commenti

16.4 Checkpoint operativi e controllo dei deliverable partner

Nel modello BuildTrust il partner non deve mai diventare una seconda regia. Anche quando esegue una parte tecnica importante, il partner resta una capacità coordinata da BuildTrust e non un soggetto che ridefinisce scope, priorità o criteri di completamento. Questo principio è essenziale per difendere know-how, ritmo operativo e narrativa verso il cliente.

Il rischio più comune non è il partner che fallisce apertamente, ma il partner che procede in autonomia su un perimetro poco chiarito e restituisce tardi un deliverable difficile da integrare. In questi casi la perdita di controllo non è immediatamente visibile ma si traduce in debito di coordinamento, ritardo, rilavorazione interna e minore credibilità verso il cliente.

Checkpoint partner	Cosa verificare	Owner BuildTrust
Kickoff del work package	Scope, output atteso, vincoli, accessi, formato della consegna.	Owner interno
Checkpoint intermedio 1	Avanzamento reale, blocchi, qualità degli output prodotti finora.	Owner interno + referente tecnico
Checkpoint intermedio 2	Coerenza con milestone di progetto e impatto su altre dipendenze.	DM + owner interno
Review finale	Qualità del deliverable, documentazione, learnings, passaggio di conoscenza.	Owner interno + reviewer tecnico

17. Processo di supporto, customer success e continuità operativa

17.1 Obiettivo

Gestire in modo disciplinato il post go-live e il supporto continuativo, garantendo continuità operativa, raccolta strutturata dei feedback e capitalizzazione degli apprendimenti nel backlog di prodotto.

17.2 Classificazione e SLA

Tipo	Descrizione	SLA risposta	Routing
Incident	Blocco operativo che impatta il cliente in produzione	2h	Escalation immediata a COO + DM
Service Request	Assistenza su funzionalità esistenti, configurazioni	24h lavorative	DM o referente supporto
Bug	Comportamento non conforme, non bloccante	48h (inserito in sprint)	PM per triage e inserimento backlog
Enhancement	Miglioramento funzionalità esistente	Valutazione entro 1 settimana	PM per valutazione roadmap
New Scope	Richiesta fuori contratto	Qualificazione entro 5 giorni	BD per valutazione opportunità commerciale

17.3 Fasi operative

- Ticket Intake. Il cliente apre il ticket attraverso il canale definito a contratto (Teams channel dedicato, email a helpdesk@buildtrust.it o portale se disponibile). Il ticket viene registrato in Azure DevOps del progetto come Issue con tipo corrispondente alla classificazione.
- Classification e Prioritization. Il DM o il referente supporto classifica il ticket entro le tempistiche SLA e lo assegna alla risorsa appropriata. Gli Incident vengono marcati S1 in Azure DevOps e il

COO viene notificato automaticamente via Teams.

- Resolution o Backlog Transfer. Risoluzione diretta con documentazione della soluzione nella issue. Se non risolvibile nel breve: apertura di una issue collegata nel backlog con priorità appropriata e comunicazione al cliente dei tempi.
- Closure e Feedback. Chiusura del ticket con link alla soluzione o al work item di risoluzione. Per clienti in customer success attivo: report mensile su SharePoint con stato del servizio, ticket risolti, trend, prossime attività.

Configurazione Azure DevOps per il supporto Ogni progetto in go-live ha un'area 'Support' nel board con colonne: New | In Progress | Pending Client | Resolved | Closed. La notifica automatica al COO per S1 è configurata tramite Azure DevOps Service Hooks > Microsoft Teams. Il DM è responsabile che tutti i ticket abbiano una risposta entro SLA anche nei periodi di minor presidio.

17.4 Go-live e stabilizzazione

Nel contesto BuildTrust il rilascio non dovrebbe mai essere considerato solo un evento tecnico. Ogni release impatta processi, milestone, supporto, aspettative dei Delivery Manager e in alcuni casi la logica con cui il cliente interpreta stati e avanzamenti. Questo rende la release readiness una verifica di sistema e non solo di pipeline.

Il passaggio da staging a produzione dovrebbe essere sostenuto da un set minimo di verifiche leggibili: quality gate superato, smoke test eseguiti, effetti su dati e configurazioni noti, eventuali dipendenze di progetto comunicate, runbook aggiornato, finestra di osservazione post-rilascio definita e ownership chiara in caso di incident. Se uno di questi elementi manca, il rischio non è solo tecnico ma organizzativo.

La stabilizzazione successiva al go-live serve a evitare che il progetto dichiari successo troppo presto. Le prime 24-72 ore dovrebbero essere lette come finestra di osservazione: bug emersi, comportamenti inattesi, necessità di supporto, adozione lato cliente, tenuta dei dati o delle integrazioni.

Aspetto	Domanda di readiness
Qualità tecnica	I test rilevanti sono passati e il quality gate è verde?
Dati e configurazioni	Sappiamo se la release tocca mapping, pipeline, secrets o ambienti?
Impatto sui progetti	I DM coinvolti sono stati informati prima del deploy?
Runbook	Esiste una guida minima per supporto, recovery o verifiche post-release?
Osservazione post go-live	Chi guarda alert e bug nelle prime 24-48 ore?

18. Processo di issue, escalation e decision management

18.1 Livelli di severita e regole di escalation

Severita	Descrizione	Escalation	Timeframe risposta
S1 – Critica	Blocco operativo su progetto attivo, rischio contrattuale immediato, cliente fermo	Immediata a COO – COO avvisa CEO se > 2h senza soluzione	Risposta entro 2h, piano entro 4h
S2 – Alta	Ritardo milestone probabile, dipendenza cliente bloccata, partner fermo senza sblocco	Review con COO entro 24h	Piano di azione entro 24h
S3 – Media	Problema tecnico non bloccante, friction di processo, partner in leggero ritardo	Weekly Operations Review	Piano entro 1 settimana
S4 – Bassa	Miglioramento, inefficienza minore, suggerimento operativo	Backlog o monitoraggio	Valutazione nel prossimo ciclo

18.2 Decision Log – come si usa

Il Decision Log è un documento condiviso su SharePoint (Operations > Decision_Log.xlsx) con le seguenti colonne: Data, ID decisione, Contesto, Opzioni valutate, Decisione presa, Rationale, Owner, Scadenza implementazione, Stato (Open/Done/Cancelled).

Quando si usa: ogni volta che si prende una decisione che impatta scope, timeline, budget, struttura organizzativa, scelta tecnologica rilevante o relazione con un cliente o partner. Non serve per decisioni di dettaglio operativo giornaliero.

Regola pratica sul Decision Log Se sei in riunione e si decide qualcosa di rilevante, prima di chiudere la call qualcuno apre il Decision Log e inserisce la decisione in real time. Non si demanda 'a dopo'. Il COO è responsabile che il log sia aggiornato entro 24h da ogni decisione presa nei rituali operativi.

18.3 Comportamento atteso nelle escalation

Ogni modello operativo viene realmente testato quando le cose non vanno come previsto. Per questo BuildTrust deve chiarire non solo il processo ideale, ma anche il comportamento atteso in caso di blocchi, ritardi, bug critici, dipendenze cliente non rispettate o richieste fuori perimetro. Una software factory matura non si riconosce dall'assenza di problemi, ma dalla velocità e dalla qualità con cui li rende leggibili e li governa.

Il primo principio è l'emersione precoce. Quando un rischio comincia a minacciare una milestone o una release, non dovrebbe restare in un canale tecnico ristretto. Va classificato, tracciato e portato rapidamente al livello giusto. Il secondo principio è la responsabilità di proposta: chi porta un

problema non dovrebbe limitarsi a segnalarlo, ma dovrebbe già formulare una prima ipotesi di impatto e una possibile strada di gestione.

18.3.1 Escalation tecnica

Un'escalation tecnica nasce quando il team incontra un problema che non riesce a risolvere nei tempi necessari o che cambia in modo rilevante il rischio del lavoro. Può trattarsi di un'integrazione che non funziona come previsto, di un bug bloccante, di un limite architetturale emerso tardi o di una dipendenza tra componenti che mette a rischio il rilascio. In questi casi l'engineering dovrebbe aggiornare il work item o l'issue, chiarire impatto e severità e coinvolgere rapidamente chi può aiutare a decidere: engineering lead, PM, DM o COO a seconda del tipo di problema.

La qualità dell'escalation tecnica si misura dalla chiarezza. Una buona escalation dice cosa è successo, quale parte del sistema è coinvolta, che impatto ha sul piano, quali opzioni sono sul tavolo e quale decisione serve. Senza questa chiarezza il management perde tempo a ricostruire il problema invece di scioglierlo.

18.3.2 Escalation di progetto e verso il cliente

L'escalation di progetto riguarda milestone a rischio, dipendenze esterne non rispettate, assenze di risposta del cliente, disallineamenti sull'accettazione o richieste fuori scope che stanno destabilizzando il lavoro. In questi casi il DM ha la responsabilità di costruire una lettura manageriale del problema: che cosa rischiamo, quando lo vedremo, che opzioni abbiamo e quale messaggio va portato al cliente. L'errore più comune è aspettare troppo per paura di 'disturbare' il cliente o il management; così facendo si spreca il tempo in cui la situazione sarebbe ancora recuperabile con interventi relativamente piccoli.

La comunicazione al cliente dovrebbe essere preparata internamente prima della call o della mail formale. BuildTrust dovrebbe presentarsi con una proposta, non solo con un allarme. Anche quando il problema nasce dal cliente, la forma migliore di escalation resta quella che combina trasparenza e governo: spiegare il fatto, mostrare l'impatto, proporre una strada e chiedere una decisione o una collaborazione puntuale.

18.3.3 Quando deve intervenire il COO

Il COO dovrebbe entrare quando il problema non è più solo locale: capacità insufficiente, conflitto di priorità tra progetto e prodotto, rischio reputazionale con il cliente, necessità di partner, slittamento che impatta altri impegni, problema sistemico di qualità o di processo. Il suo ruolo non è sostituirsi ai team, ma creare le condizioni di risoluzione: riallocare capacità, bloccare lavori non essenziali, decidere trade-off, cambiare sequencing, intervenire nella governance con il cliente o rafforzare un gate operativo.

Perché l'intervento del COO sia efficace, il problema deve arrivare con un minimo di preparazione. Questo significa che DM, PM o engineering dovrebbero presentare non solo il problema ma anche il quadro delle conseguenze e delle alternative. In questo modo la decisione del COO è più rapida e più utile.

Tipo di problema	Chi guida la prima gestione	Quando si scala oltre
Bug o blocco tecnico locale	Engineering lead / team tecnico	Se impatta milestone, release o richiede scelta di priorit�.
Milestone a rischio	Delivery Manager	Se serve riallocare capacit� o riallineare il cliente.
Richiesta che impatta piattaforma	Product Manager	Se altera roadmap o richiede trade-off strategico.
Conflitto tra piu stream di lavoro	COO	Quando il team locale non puo piu risolvere da solo.
Rischio reputazionale/commerciale	COO con CEO e vendite	Quando il tema puo cambiare relazione o fiducia del cliente.

PARTE VI – CAPACITY MODEL E TOOLCHAIN

19. Capacity model e criteri di scalabilit 

19.1 Competenze critiche e fabbisogno per livello di parallelismo

Competenza	1 progetto	2 progetti	3 progetti (max governato)	Note
Delivery / PM	1 DM a tempo pieno	1 DM + supporto DM	2 DM	Limite critico: con 3 progetti serve il secondo DM
Data & AI Engineer	Parziale (setup + milestone)	1 dedicato	1-2	Con piu fonti eterogenee il carico cresce rapidamente
Cloud Engineer	Parziale (setup, deploy)	Parziale condiviso	1 dedicato	Carico concentrato in setup e milestone critiche
Full Stack Developer	1 FSD (prodotto + progetto)	1 FSD + partner	1-2 FSD	Partner gestiti da FSD come owner interno

19.2 Regola di parallelismo

2 progetti complessi (Platform o TSP) in parallelo: scenario standard sano. 3 progetti: massimo governato, temporaneo, previa verifica esplicita COO. Oltre 3: solo con rinforzo strutturale gia attivato. I progetti Digital Plant possono coesistere senza saturare le competenze core.

19.3 Criteri di intake progetto

Nessun progetto può essere avviato senza che il COO verifichi:

- Owner delivery nominato e disponibile per tutta la durata stimata.
- Piano di capacità per competenza critica validato: nessuna risorsa > 100% per più di 2 settimane consecutive.
- Dipendenze cliente chiarite: accessi tecnici, referenti, disponibilità per kickoff e milestone.
- Readiness minima dati e infrastruttura verificata dal Data & AI Engineer.
- Spaziatura sostenibile rispetto ai progetti attivi: almeno 2 settimane di overlap massimo tra fine setup di un progetto e inizio setup del successivo.

19.3.1 Guida pratica alle priorità quando la capacità è in tensione

Quando la capacità entra in tensione, BuildTrust non dovrebbe reagire all'ultima urgenza emersa ma usare una gerarchia di decisione stabile. La prima priorità è proteggere ciò che evita perdita di affidabilità: milestone vicine, issue gravi, qualità di rilascio, dipendenze critiche e continuità di piattaforma. La seconda è distinguere il lavoro che genera valore riusabile dal lavoro che risponde solo a una pressione locale. La terza è dichiarare con chiarezza ciò che non può entrare subito, invece di farlo partire in modo implicito.

La guida pratica per COO, PM e DM è chiedersi sempre cinque cose: quale lavoro protegge meglio il valore della piattaforma; quale sblocca una milestone contrattuale; quale riduce un rischio sistemico; quale può aspettare senza creare debito operativo; quale andrebbe esplicitamente rifiutato o sequenziato. Questa disciplina è più importante del dettaglio numerico delle allocazioni, perché impedisce che il modello perda coerenza proprio nel momento di maggiore pressione.

19.4 Demand planning operativo a sei mesi

Il capacity model non è sufficiente se BuildTrust legge il carico solo dopo la firma dei contratti. Per questo il TOM raccomanda di introdurre un processo stabile di demand planning a sei mesi che colleghi pipeline commerciale, presales, tipologia di progetto, capacità disponibile e decisioni di hiring o sequencing. L'obiettivo non è prevedere con certezza il futuro, ma trasformare la pipeline in una vista credibile di domanda operativa.

Il processo operativo dovrebbe essere semplice e ripetibile. Per ogni opportunità matura il commerciale e il presales aggiornano almeno quattro dati: stadio dell'opportunità, finestra attesa di avvio, classe di progetto e prima lettura delle competenze richieste. Il COO traduce questi dati in domanda ponderata, confrontandola con capacità interna, partner disponibili, carico già impegnato e protezione minima della roadmap di prodotto.

La logica di base è calcolare, per ciascuna opportunità, un effort ponderato dato da effort medio stimato per probabilità di ingresso nella finestra temporale considerata. La somma degli effort ponderati non produce una promessa commerciale, ma un'indicazione di saturazione attesa per ruolo. Questo consente di anticipare decisioni su hiring, partner, no-go o sequenziamento, prima che il sistema entri in crisi.

Per rendere il modello utile nel tempo, BuildTrust dovrebbe leggere il demand planning per classi ricorrenti di progetto e non solo opportunità isolate. Classificare le iniziative in archetipi leggibili

consente di confrontare effort previsto ed effort consuntivato, migliorare progressivamente le stime e capire quali pattern di domanda stanno diventando strutturali.

Dato minimo da raccogliere	Uso nel planning	Owner primario
Stadio presales e probabilit� associata	Trasforma la pipeline in domanda ponderata e non in semplice elenco opportunit�.	Commerciale + COO
Finestra attesa di avvio	Distribuisce il carico sui prossimi sei mesi e rende leggibili i picchi.	Commerciale / presales
Classe di progetto	Permette di applicare staffing tipico ed effort medio coerenti.	COO + DM + PM
Ruoli/competenze coinvolte	Fa emergere i colli di bottiglia reali per ruolo e non solo per totale aggregato.	COO + engineering / DM
Dipendenze e complessit�	Corregge le stime base e segnala rischio di slittamento o maggior coordinamento.	Presales + DM

Il modello pratico pi  utile per BuildTrust   leggere la pipeline su tre livelli: volume di opportunit  per finestra temporale, probabilit  di ingresso per stadio della presales ed effort medio atteso per tipologia di progetto. La formula di base   semplice: per ogni opportunit  si calcola un effort ponderato dato da effort medio stimato moltiplicato per probabilit  di ingresso nella finestra considerata. Sommando gli effort ponderati si ottiene una domanda attesa da confrontare con capacit  interna, partner disponibili e protezione minima della roadmap di prodotto.

Per non trasformare il planning in opinione, BuildTrust dovrebbe associare a ogni stadio di presales una probabilit  standard di ingresso, da affinare nel tempo con lo storico reale. L'obiettivo non   la precisione matematica al primo colpo, ma una convenzione comune che permetta a commerciale e COO di parlare la stessa lingua e di aggiornare il modello ogni trimestre in base a quanto accaduto davvero.

Stadio opportunit�	Lettura manageriale	Uso nel demand planning
Discovery iniziale	Interesse reale ma ancora molto variabile.	Carico solo orientativo, utile per segnale precoce.
Presales qualificata	Problema e fit tecnico iniziano a essere chiari.	Carico ponderato da inserire nel piano a sei mesi.
Offerta in definizione	Perimetro pi� leggibile, discussione economica in corso.	Carico con probabilit� medio-alta.
Offerta finale / negoziazione	Possibile ingresso a breve con perimetro pi� stabile.	Carico quasi certo da confrontare con capacit� disponibile.
Commit imminente	Partenza molto probabile o gi� pianificata.	Carico da trasformare in staffing operativo.

Il demand planning diventa davvero utile quando BuildTrust non ragiona opportunit  per opportunit  in modo completamente artigianale, ma costruisce classi tipiche di progetto. L'idea non   semplificare troppo la realt , ma creare una base comparabile da cui derivare staffing tipico, durata indicativa ed

effort medio per ruolo. Questo rende più veloce la stima di nuove opportunità e consente di confrontare effort previsto ed effort consuntivato, migliorando progressivamente il modello.

Classe progetto	Staffing tipico	Lettura utile per il COO
Configurazione / onboarding	DM leggero, supporto tecnico limitato, basso peso di prodotto.	Utile per capire saturazione di delivery e supporto.
Integrazione dati	DM, Data/AI Engineer, Cloud/Integration support, eventuale QA.	Segnala fabbisogno di competenze tecniche specialistiche.
Evoluzione piattaforma guidata da cliente	PM coinvolto, engineering più strutturato, DM con forte raccordo cliente.	Aiuta a vedere impatto su roadmap prodotto.
Progetto enterprise complesso	DM senior, PM o product advisor, engineering multi-ruolo, maggior governance stakeholder.	Assorbe capacità manageriale oltre che tecnica.
Custom ad alta intensità	Engineering focalizzato, DM forte, rischio alto di erosione roadmap.	Richiede decisione esplicita su convenienza e sostenibilità.

Una volta definite probabilità di ingresso e staffing tipico, il COO può costruire una vista a sei mesi con tre scenari: base, upside e stress. Lo scenario base considera la conversione più probabile delle opportunità. L'upside considera un tasso di ingresso migliore del previsto sulle opportunità mature. Lo stress considera ingressi ravvicinati o combinazioni di progetti ad alta intensità. Questo aiuta a non pianificare in modo ingenuamente lineare e a preparare risposte diverse a seconda di come evolve la pipeline.

Le policy di hiring dovrebbero nascere da questo quadro e non da percezioni episodiche. Se per più cicli consecutivi il carico ponderato a sei mesi supera una soglia di saturazione su un ruolo chiave, si apre un hiring plan. Se la tensione è breve o ancora troppo incerta, si privilegia una risposta tramite partner governati o tramite migliore sequencing delle opportunità. Le competenze core legate a piattaforma, architettura, product thinking e delivery governance dovrebbero essere rafforzate soprattutto in-house; le capacità più variabili possono invece essere assorbite con partner finché il pattern di domanda non dimostra che conviene internalizzarle.

La review di demand planning dovrebbe avvenire almeno una volta al mese tra COO e commerciale, con il coinvolgimento di PM e DM quando opportuno. L'uscita attesa non è solo un forecast, ma una decisione manageriale: aprire o no un hiring plan, attivare partner, proteggere la roadmap di prodotto, rifiutare eccezioni o riallineare la sequenza di avvio dei progetti.

19.4.1 Perché il demand planning deve partire dalle presales

In molte organizzazioni il planning della capacità parte dai contratti firmati. Questo approccio è troppo tardo per una realtà come BuildTrust, in cui competenze specialistiche, onboarding e hiring richiedono tempo. Il punto di partenza corretto è invece la pipeline qualificata di presales, perché è il momento in cui l'opportunità smette di essere solo commerciale e inizia a mostrare forma tecnica, tipologia di progetto, intensità di lavoro e probabilità di concretizzarsi.

Il commerciale e il presales devono quindi produrre non solo previsione di valore economico, ma anche previsione di assorbimento operativo. Per ogni opportunità rilevante serve almeno una classificazione per tipologia, una probabilità di ingresso, una finestra attesa di partenza e una prima

lettura delle competenze richieste. Senza queste informazioni il COO non può fare un vero piano di capacità, ma solo reagire quando la firma è già avvenuta.

La precisione qui non significa predire con certezza il futuro, ma trasformare la pipeline in scenari credibili di carico. Ogni opportunità in presales genera una probabilità di ingresso, una finestra temporale attesa e un assorbimento indicativo di capacità. Il punto non è sapere esattamente quale progetto entrerà il giorno X, ma capire quanta domanda tecnica e di delivery ha probabilità ragionevole di materializzarsi nei prossimi sei mesi, in quali finestre e con quale mix di competenze.

19.4.2 Come si leggono i risultati del demand planning

Un buon piano di domanda non serve a produrre un foglio di calcolo elegante, ma a orientare decisioni concrete. Se il carico ponderato a sei mesi supera in modo stabile la capacità disponibile di un ruolo chiave, il management non deve attendere la firma di tutti i contratti per reagire. Deve capire se conviene aprire hiring, attivare partner, rifiutare o sequenziare opportunità, oppure proteggere maggiormente il backlog prodotto.

La lettura corretta è sempre doppia. Da un lato quantitativa: quante giornate o FTE attesi per ruolo. Dall'altro qualitativa: che tipo di progetti stanno entrando, quanto coordinamento richiedono, quanto incidono su piattaforma, quanto rischio di dipendenza cliente o partner introducono. Due opportunità con effort simile possono avere impatto organizzativo molto diverso. Per questo il demand planning deve restare vicino alla realtà della delivery e non essere lasciato solo a una funzione amministrativa.

Segnale emerso dal planning	Lettura manageriale	Decisione possibile
Saturazione forte su un ruolo specializzato	Il collo di bottiglia non è generale ma concentrato.	Hiring mirato, partner qualificato o sequencing.
Crescita di progetti simili	Sta emergendo una domanda ripetibile.	Valutare hiring strutturale e standardizzazione del delivery model.
Molto carico probabile ma poca certezza	Rischio di over-reaction se si assume troppo presto.	Usare partner o pipeline review più frequenti.
Molti progetti che erodono prodotto	Il business sta spingendo custom o delivery urgente.	Ridefinire policy commerciali e triage prodotto.
Picchi ravvicinati di avvio	Serve capacità di mobilitazione e governance oltre che sviluppo.	Rinforzare DM, PM o supporto al setup.

19.4.3 Policy di hiring derivate dal demand planning

Le policy di hiring BuildTrust dovrebbero nascere da questo quadro e non da percezioni episodiche. Il COO e il management dovrebbero definire soglie e trigger: se per due o tre cicli consecutivi il carico ponderato a sei mesi supera una soglia di saturazione su un ruolo chiave, si apre un hiring plan. Se la tensione è breve o ancora troppo incerta, si privilegia una risposta tramite partner o tramite migliore sequencing.

È inoltre utile distinguere il perché dell'hiring: aumentare throughput tecnico, aumentare capacità di governance progettuale, proteggere prodotto da interruzioni, ridurre dipendenza da partner su competenze core. Le capacità core legate a piattaforma, architettura, product thinking e delivery governance dovrebbero essere rafforzate principalmente in-house. Le capacità più variabili possono

invece essere assorbite tramite partner governati, finché il pattern di domanda non dimostra che conviene internalizzarle.

Situazione osservata	Policy suggerita	Perché è coerente
Domanda stabile e ripetibile su competenza core	Hiring interno.	Conviene accumulare know-how e ridurre dipendenza.
Picco temporaneo su competenza non core	Partner governato o staffing flessibile.	Evita sovrastruttura permanente.
Domanda crescente ma ancora incerta	Pipeline watch ravvicinata + opzione hiring preparata.	Riduce il rischio di assumere troppo presto.
Ruolo manageriale sotto stress	Hiring o rinforzo su DM/COO support.	La saturazione non è solo tecnica ma di coordinamento.
Prodotto disturbato da delivery	Hiring/allocazione dedicata per proteggere roadmap.	Serve separare meglio capacità di piattaforma e progetto.

19.4.4 Ruoli di COO e commerciale nel processo

Questo processo funziona solo se COO e commerciale si vedono come co-owner della previsione. Il commerciale porta visibilità sulle opportunità, sulla maturità delle relazioni e sulla probabilità reale di chiusura. Il COO porta la lettura di capacità, la sensibilità sulle competenze scarse, la conoscenza dei progetti già in corso e la responsabilità di trasformare la previsione in staffing, sequencing e hiring. Se uno dei due lavora da solo, il modello si spezza.

BuildTrust dovrebbe introdurre una review periodica di demand planning, almeno mensile, con una struttura semplice: pipeline presales aggiornata, probabilità condivise, tassonomia di staffing tipico, confronto carico-capacità a sei mesi, decisioni finali (hiring, partner, protezione roadmap, riallineamento commerciale). Il valore della review sta nel far convergere commerciale e operations su una stessa rappresentazione del futuro prossimo.

19.4.5 Come spiegare il demand planning al management e ai tecnici

Ai manager il demand planning va presentato come strumento per prendere decisioni in anticipo e non come previsione perfetta. Ai tecnici va spiegato come meccanismo che protegge il lavoro da sovraccarichi e promesse irrealistiche. Se la squadra comprende che la lettura della pipeline serve anche a evitare partite contemporanee ingestibili, sarà più naturale collaborare con dati di effort consuntivo, pattern di staffing e feedback sul reale assorbimento dei progetti.

Nel medio periodo questo processo produce anche un altro vantaggio: rende BuildTrust più capace di imparare dai propri numeri. Ogni progetto chiuso migliora il catalogo delle classi progetto, affina l'effort medio, corregge le probabilità di ingresso e rende più robuste le future decisioni di hiring. In questo senso il demand planning non è solo uno strumento di previsione; è una delle pratiche con cui l'azienda diventa davvero una software factory governata.

19.5 Piano di rafforzamento progressivo

Orizzonte	Azione	Trigger	Priorita
0-3 mesi	Formalizzare separazione ruoli PM e DM, nominare owner per tutti i partner attivi	Immediata – indipendente dalla pipeline	Critica
3-6 mesi	Filling vacancies critiche: CDE Developer, DB Specialist	Pipeline > 2 progetti attivi	Alta
6-12 mesi	Aggiungere secondo Delivery Manager	Pipeline stabile > 2 progetti complessi	Alta
6-12 mesi	Rafforzare Data & AI Engineering (secondo profilo o partner con owner dedicato)	Progetti con piu di 3 fonti dati eterogenee	Media
12-18 mesi	Cloud Engineer dedicato full time	Pipeline stabile > 3 progetti	Media
18-24 mesi	Head of Product dedicato, strutturazione Customer Success	Piattaforma > 5 clienti in go-live	Pianificazione

20. Toolchain operativa – Azure DevOps e ecosistema Microsoft

20.1 Principi di selezione

La toolchain di BuildTrust è stata selezionata seguendo tre criteri: integrazione nativa con l'ecosistema Microsoft già presente nel gruppo Simem (Office 365, Azure AD, Azure cloud), semplicità di adozione per un team piccolo che non può permettersi di gestire molti sistemi disconnessi, visibilità end-to-end dal backlog di prodotto alla delivery di progetto alla dashboard direzionale.

20.2 Stack completo BuildTrust

Area	Strumento	Utilizzo specifico BuildTrust
CRM e pipeline	HubSpot (già adottato)	Gestione opportunità commerciali, pipeline per stage, forecast, template email, log attività commerciale. Il BD aggiorna ogni opportunità dopo ogni interazione cliente.
Collaboration	Office 365 – Teams, Outlook, SharePoint	Teams: comunicazione interna e canali cliente. SharePoint: documentation hub (template, handover, artefatti progetto, decision log). Outlook: comunicazione formale esterna.

Area	Strumento	Utilizzo specifico BuildTrust
Project & Delivery Management	Azure DevOps – Boards	Un Project per ogni progetto cliente + Project 'Platform' per il backlog prodotto + Project 'Operations' per capacity board, decision log, RAID aziendale.
Sviluppo e versioning	Azure Repos	Repository per piattaforma e per ogni modulo. Branch strategy: main protetto, feature/[task-id], PR con almeno 1 reviewer. Scelta definitiva: Azure Repos, coerente con l'ecosistema Microsoft e con l'integrazione nativa in Azure DevOps Pipelines.
CI/CD	Azure DevOps Pipelines	Pipeline per ogni ambiente: dev (trigger su merge in develop), staging (trigger manuale o su tag), prod (approvazione PM o COO). Template pipeline su SharePoint.
Qualità codice	SonarCloud (integrato con Azure DevOps)	Quality gate configurato: coverage minimo, zero critical issues, complexity threshold. PR bloccata se gate non superato.
Infrastruttura cloud	Azure (già adottato nel gruppo)	App Service o Container Apps per la piattaforma, Azure Data Factory per pipeline dati, Azure Monitor per observability, Key Vault per secrets, Terraform per IaC.
BI e dashboard	Power BI (incluso in Office 365)	Dashboard COO: stato progetti, capacity per competenza, KPI pipeline commerciale. Aggiornamento automatico da Azure DevOps e HubSpot.
IaC e configuration	Terraform	Infrastruttura Azure gestita come codice. Repository dedicato su Azure Repos. Ogni ambiente ha il proprio stato Terraform.

20.3 Configurazione Azure DevOps – struttura raccomandata

La struttura organizzativa di Azure DevOps per BuildTrust è la seguente:

Organization	Projects	Boards / Areas	Chi vi accede
buildtrust.visualstudio.com	Platform	Product Backlog, Sprint Board, Release Board	PM, Engineering, COO (read)
buildtrust.visualstudio.com	Operations	Capacity Board, Decision Log, Risk Register, Partner Tracker	COO, DM, PM
buildtrust.visualstudio.com	[Sigla cliente]-[Anno]	Project Board, RAID Log, Milestone Tracker, Evidence Tracker, Support	DM owner, Engineering coinvolto, COO (read), Partner (contributor limitato)

20.4 Integrazione HubSpot – Azure DevOps

Quando un'opportunità passa a Closed Won in HubSpot, il BD crea manualmente (o tramite automazione HubSpot Workflow) un ticket di handover che avvia la creazione del progetto in Azure DevOps. Il link tra opportunità HubSpot e project Azure DevOps viene registrato nel campo 'External Reference' del project Azure DevOps e nel campo 'Project ID' dell'opportunità in HubSpot. Questo garantisce tracciabilità end-to-end dalla lead al progetto.

20.5 Integrazione Teams – Azure DevOps

Ogni progetto Azure DevOps ha un canale Teams dedicato. Le notifiche di Azure DevOps (nuove issue, aggiornamenti milestone, fallimento pipeline, alert qualità) vengono inviate al canale Teams del progetto tramite Azure DevOps Service Hooks. Il DM è responsabile della configurazione iniziale delle notifiche al momento del setup del progetto.

Setup rapido notifiche Azure DevOps in Teams Azure DevOps > [Progetto] > Project Settings > Service Hooks > Microsoft Teams Connector. Configura notifiche per: Work item created/updated (filtro tipo: Issue, Milestone), Build completed (filtro status: Failed), Pipeline run state changed. Incolla il webhook del canale Teams del progetto.

20.6 Uso operativo di Azure DevOps per prodotto, progetto ed engineering

Azure DevOps deve essere introdotto in BuildTrust non come ennesimo strumento tecnico, ma come il sistema che rende visibile il lavoro e lo collega a decisioni, qualità, rilasci e responsabilità. La sua utilità non sta nella completezza della piattaforma, ma nel fatto che può unire backlog, board, repository, pull request e pipeline dentro un unico filo di tracciabilità. Se usato bene permette a manager e tecnici di leggere lo stesso lavoro con profondità diverse ma coerenti.

La struttura ufficiale della toolchain e le regole di riferimento su Azure DevOps sono già definite nel capitolo 20. Questa sezione le riprende in forma operativa e sintetica, come quick reference per l'uso quotidiano di board, repo, PR e tracciabilità.

Azure DevOps deve essere introdotto in BuildTrust non come ennesimo strumento tecnico, ma come il sistema che rende visibile il lavoro e lo collega a decisioni, qualità, rilasci e responsabilità. La sua utilità non sta nella completezza della piattaforma, ma nel fatto che può unire backlog, board, repository, pull request e pipeline dentro un unico filo di tracciabilità. Se usato bene permette a

manager e tecnici di leggere lo stesso lavoro con profondità diverse ma coerenti.

La guida che segue non vuole essere un manuale di Azure DevOps. L'obiettivo è chiarire come BuildTrust dovrebbe impostarlo, quali regole di utilizzo sono davvero utili e quali comportamenti quotidiani lo trasformano in uno strumento di governo e non in un archivio aggiornato male.

20.6.1 Principio guida: un solo flusso, viste diverse

La prima regola è evitare di usare Azure DevOps come contenitore generico di task scollegati. In BuildTrust il lavoro deve potersi leggere come flusso: dalla richiesta alla prioritizzazione, dall'esecuzione al rilascio, dal bug al fix, dalla milestone alla evidenza. Questo non significa che prodotto e progetti debbano stare nello stesso backlog indistinto, ma che devono vivere nello stesso sistema con strutture leggibili e collegate.

Una buona configurazione dovrebbe prevedere almeno due viste principali. La prima è la vista Prodotto, orientata a capability, epiche, feature, bug di piattaforma e debito tecnico. La seconda è la vista Progetto, orientata a milestone, task, issue, evidenze e dipendenze. Le due viste devono poter dialogare tramite link espliciti: una richiesta nata in progetto e candidata al backlog di prodotto; una release di prodotto impatta specifici progetti; un bug di piattaforma può bloccare una milestone cliente.

20.6.2 Come usare Boards per il prodotto

Il board di prodotto dovrebbe rappresentare il lavoro della piattaforma e non il lavoro di ogni cliente. Qui trovano posto epiche, feature, capability trasversali, bug di piattaforma, attività di hardening, miglioramenti tecnici e debito. Il Product Manager lo usa per ordinare domanda e capacità. L'engineering lo usa per rendere visibile il flusso reale. Il COO lo usa per capire se la roadmap è governata o continuamente interrotta.

Perché il board di prodotto resti leggibile, ogni item dovrebbe avere una formulazione che spiega il problema e non solo l'intervento. Un titolo come 'aggiungere endpoint X' dice poco al management. Un item come 'rendere tracciabile la validazione documentale multi-fase per supportare onboarding enterprise' permette invece di leggere valore, impatto e coerenza di piattaforma. Anche quando il lavoro è molto tecnico, il board deve restare interpretabile da chi governa prodotto e delivery.

La distinzione tra epic, feature e task non deve essere dogmatica, ma funzionale. L'epic raccoglie una capability o un filone di cambiamento rilevante. La feature descrive un outcome rilasciabile o almeno valutabile. Il task rappresenta un pezzo di lavoro eseguibile e assegnabile. Se il team usa troppe feature minuscole, il board si frammenta. Se usa task troppo grandi o generici, il board perde capacità di lettura. La regola pratica dovrebbe essere: una feature abbastanza chiara da essere prioritizzabile, un task abbastanza piccolo da essere governabile.

Livello board prodotto	Come usarlo in BuildTrust	Errore da evitare
Epic	Capability o area di evoluzione con impatto reale su piattaforma o roadmap.	Usarla come contenitore vago senza outcome misurabile.
Feature	Unità di valore da discutere in triage e verificare in rilascio.	Spezzarla troppo o descriverla solo in termini tecnici.
Task	Lavoro eseguibile da sviluppatore o piccolo gruppo in pochi giorni.	Aprire task senza legame a feature o senza chiaro criterio di done.
Bug	Difetto che altera comportamento atteso della piattaforma.	Nascondere bug dentro task generici o backlog non visibile.

20.6.3 Come usare Boards per i progetti

Il board di progetto non deve essere una copia del piano economico, né una to-do list disordinata. Deve aiutare il Delivery Manager a capire dove si trova il progetto, cosa rischia la milestone successiva e quali dipendenze stanno consumando attenzione del team. Per questo nel board di progetto dovrebbero comparire attività di setup, attività tecniche, task del cliente, issue, checkpoint e raccolta evidenze.

Una pratica utile è separare con chiarezza il lavoro di BuildTrust dal lavoro atteso dal cliente o dal partner. Se il board contiene solo task interni, il rischio è illudersi che il progetto avanzi mentre dipende da accessi, dati o approvazioni ancora in sospeso. Le dipendenze esterne non devono stare in una chat o in un verbale disperso: devono comparire come item o issue visibili, con owner e scadenza.

Il Delivery Manager dovrebbe entrare nel board ogni giorno non per 'controllare le persone', ma per capire flusso, blocchi e prossime decisioni. Se vede molti item in corso e pochi chiusi, deve aprire un confronto sul focus. Se vede item chiusi ma milestone senza evidenze, deve intervenire sulla preparedness. Se vede blocchi esterni senza owner, deve lavorare sulla governance con il cliente. Il board quindi non sostituisce il giudizio del DM, ma lo rende più fondato.

Tipo di item di progetto	Uso corretto	Owner tipico
Milestone	Traguardo di avanzamento leggibile anche dal cliente.	DM
Task BuildTrust	Attività operative o tecniche interne collegate a milestone o issue.	Engineering / DM
Dependency item	Input o azione richiesta a cliente o partner che condiziona l'avanzamento.	DM con owner esterno nominato
Issue/Risk	Problema o rischio da monitorare con severità e decisione attesa.	DM / COO
Evidence item	Prova da raccogliere per acceptance o avanzamento.	DM / referente tecnico

20.6.4 Distinguere bene backlog di prodotto e backlog di progetto

Questo punto merita particolare attenzione perche da qui passano molte inefficienze. Una richiesta nata in un progetto non deve entrare automaticamente nel backlog di prodotto, ma nemmeno sparire in una soluzione ad hoc decisa dal solo team tecnico. Il passaggio corretto e registrarla come richiesta, chiarire il contesto in cui nasce, valutarne il potenziale di riuso e decidere se resta custom, se entra in roadmap o se va scartata.

Per BuildTrust e utile adottare una regola semplice. Se il bisogno riguarda una capability che rafforza il posizionamento della piattaforma, la richiesta deve essere discussa come candidato prodotto. Se invece risponde a una specificita di processo, dati, reporting o governance di un singolo cliente, dovrebbe restare nel backlog di progetto o nel perimetro custom. Il valore di questa distinzione e enorme: protegge la piattaforma e permette al commerciale di fare promesse piu oneste in futuro.

Quando nasce una richiesta dal progetto	Passo corretto	Decisione attesa
Richiesta evidentemente specifica	Registarla nel progetto come custom o change request.	Resta fuori dalla roadmap prodotto.
Richiesta potenzialmente riusabile	Aprire item candidato nel backlog prodotto con link al progetto sorgente.	PM valuta priorit� e sostenibilit�.
Richiesta urgente ma non ancora chiara	Aprire item di discovery/analisi, non una feature gi� promessa.	Decisione dopo approfondimento.
Richiesta fuori strategia	Registrarne l'esito e motivare il no-go.	Vendite e delivery allineano aspettative col cliente.

20.6.5 Repos come fondamento della tracciabilit  tecnica

Azure Repos in BuildTrust non dovrebbe essere letto solo come luogo dove risiede il codice, ma come archivio della storia tecnica del prodotto e dei cambiamenti di progetto. Ogni repository dovrebbe avere un confine chiaro, una ownership leggibile, regole minime di branch, convenzioni di naming e una relazione diretta con i work item. Il vantaggio organizzativo e immediato: quando un manager chiede cosa e cambiato e perche, la risposta puo essere trovata senza passare dalla memoria degli sviluppatori.

L'errore piu frequente in team piccoli e lasciare che il repository si organizzi secondo abitudini individuali. All'inizio sembra efficiente, ma con l'aumento dei progetti il risultato e una cronologia poco leggibile, branch aperti troppo a lungo, fix fatti d'urgenza non collegati a ticket e PR che non raccontano piu il cambiamento. In BuildTrust il repository deve sostenere un'organizzazione che vuole crescere, non solo uno sviluppo che vuole correre.

20.6.6 Strategia di branching consigliata

La strategia di branching dovrebbe essere semplice, coerente e disciplinata. BuildTrust non ha bisogno di una tassonomia complessa di rami, ma di pochi rami con regole chiare. In molti casi il modello piu sano per il team e avere un branch protetto principale, uno o piu branch di rilascio solo quando davvero servono, e branch brevi di lavoro collegati a work item o bug specifici. Il valore di questa scelta e ridurre al minimo il tempo in cui codice e contesto restano separati dal ramo principale.

La regola più importante non è il nome dei branch, ma la loro durata. Un branch che resta aperto troppo a lungo aumenta divergenza, rischio di conflitti, review superficiali e incertezza su cosa è davvero pronto. Per questo in BuildTrust conviene preferire branch piccoli, focalizzati e chiusi velocemente. Se una iniziativa è grande, va spezzata in incrementi o protetta da feature flag, non tenuta settimane fuori dal main.

Una convenzione concreta può essere la seguente: `main` come ramo protetto di integrazione; `release/x.y` solo quando un rilascio richiede stabilizzazione separata; branch di lavoro come `feature/BT-123-nome-breve`, `bugfix/BT-456-nome-breve`, `hotfix/BT-789-nome-breve`. L'ID del work item nel nome non serve per burocrazia, ma per permettere a chiunque di collegare rapidamente codice, decisione e valore atteso.

Tipo branch	Quando si usa	Regola pratica
main	Integrazione stabile e base delle release ordinarie.	Protetto, niente commit diretti.
release/x.y	Solo se serve stabilizzare un rilascio importante o gestire QA separato.	Durata breve, chiusura dopo il rilascio.
feature/ID	Nuova capability o incremento funzionale.	Un branch per change set coerente, non per settimane di lavoro indifferenziato.
bugfix/ID	Correzione di difetti non bloccanti ma tracciati.	Sempre collegato a bug o task.
hotfix/ID	Correzione urgente su comportamento in produzione.	PR rapida ma tracciata e con successivo allineamento su main/release.

20.6.7 Regole di branch protection e permessi

Le branch policies non vanno vissute come sfiducia verso gli sviluppatori, ma come protezione della qualità e della prevedibilità. In BuildTrust il ramo principale dovrebbe impedire merge senza pull request, richiedere almeno un reviewer, verificare il collegamento al work item quando il cambiamento è rilevante, eseguire i controlli automatici minimi e bloccare merge in presenza di check falliti.

Le policy vanno calibrate sul contesto. Se sono troppo leggere, non proteggono nulla. Se sono troppo pesanti, rallentano il team e vengono aggirate. Il criterio giusto è questo: introdurre solo le regole che prevengono errori tipici già osservati o molto probabili. Nel caso BuildTrust, le policy che danno subito valore sono review minima, build di validazione, divieto di commit diretto su `main`, obbligo di commento in PR sui test eseguiti e gestione esplicita delle eccezioni per hotfix.

Policy consigliata	Perche serve	Nota di utilizzo
Nessun push diretto su main	Evita bypass del processo e perdita di tracciabilità.	Le eccezioni devono essere rarissime.
Almeno un reviewer	Aumenta qualità, condivisione e responsabilità tecnica.	Su change rischiosi meglio due reviewer.
Build/check obbligatorio	Riduce merge di codice non compilabile o non validato.	Tenere il check veloce, altrimenti viene percepito come ostacolo.
Collegamento work item	Permette di leggere il perche del cambiamento.	Utile soprattutto su feature, bug e hotfix.
Commento PR su test e impatto	Migliora review e comprensione del rischio.	Formato breve ma sempre presente.

20.6.8 Come fare pull request utili e non solo formali

Una buona pull request in BuildTrust deve aiutare il reviewer a capire cosa cambia, perché cambia e quale rischio introduce. Se la PR contiene solo codice senza contesto, la review tende a ridursi a un controllo rapido di stile. Se invece la PR spiega il problema di partenza, il comportamento atteso, i test eseguiti e l'impatto su ambienti o configurazioni, allora la review diventa un vero passaggio di qualità e apprendimento.

Le PR troppo grandi dovrebbero essere considerate un segnale di processo da correggere. Non è realistico aspettarsi review profonde su centinaia di righe che toccano più aree funzionali. Quando una modifica cresce troppo, il team dovrebbe chiedersi se il lavoro era stato spezzato male, se il branch è rimasto aperto troppo a lungo o se si sta cercando di far passare in un colpo solo un cambiamento che sarebbe meglio introdurre per incrementi.

La review non deve concentrarsi solo sul 'funziona'. In BuildTrust dovrebbe verificare anche leggibilità del codice, impatto su monitoraggio e supporto, coerenza con il dominio, necessità di aggiornare documentazione o runbook e backward compatibility. Una review che non guarda il dopo-rilascio protegge poco una software factory.

Elemento della PR	Cosa dovrebbe contenere	Perche aiuta
Descrizione iniziale	Problema, obiettivo e collegamento a work item.	Rende chiaro il contesto del cambiamento.
Sintesi delle modifiche	Cosa è stato toccato e con quale logica.	Accelera la comprensione del reviewer.
Test eseguiti	Test automatici, smoke test, verifiche manuali mirate.	Aiuta a valutare rischio residuo.
Impatto operativo	Effetti su configurazioni, ambienti, dati, monitoraggio o supporto.	Allinea review tecnica e qualità di delivery.
Note di documentazione	Runbook, commenti, guide o changelog aggiornati se necessario.	Evita che il sapere resti solo nel codice.

20.6.9 Collegare work item, commit, PR e release

La vera forza di Azure DevOps emerge quando gli oggetti non restano isolati. Un work item genera un branch, il branch genera commit, i commit confluiscono in una PR, la PR viene integrata in una release, la release impatta un ambiente e talvolta una milestone di progetto. Quando questa catena è visibile, BuildTrust può fare una cosa molto preziosa: spiegare con precisione cosa è stato rilasciato, per chi, perché e con quale qualità.

Questa tracciabilità è utile non solo ai tecnici. Il DM può capire se una milestone dipende davvero da una feature ancora non rilasciata. Il PM può leggere quali richieste hanno consumato capacità. Il COO può valutare se l'organizzazione sta spendendo energia in hotfix, backlog prodotto o custom di progetto. Anche in ottica di audit e di rapporto con clienti enterprise, la capacità di raccontare il filo di una modifica aumenta molto la credibilità operativa.

20.6.10 Routine quotidiana di utilizzo per sviluppatori, PM e DM

Per gli sviluppatori la routine corretta dovrebbe essere semplice e ripetibile: entrare sul board, verificare priorità e blocchi, prendere in carico il work item giusto, aprire o aggiornare il branch collegato, mantenere lo stato del ticket coerente con il lavoro reale, aprire la PR con sufficiente contesto e chiudere il ticket solo quando il cambiamento è integrato e il done definito e stato realmente raggiunto.

Per il Product Manager la routine non è seguire ogni task, ma guardare flow e salute del backlog. Ogni settimana il PM dovrebbe rivedere item candidati, richieste nate dai progetti, feature invecchiate, bug aperti, trend di throughput e lavoro interrotto. In questo modo il board di prodotto resta una vista di governo e non si trasforma in un deposito di idee mai veramente ordinate.

Per il Delivery Manager la routine è diversa. Il DM usa Azure DevOps per leggere stato, issue e readiness, non per fare micro-management del codice. Ogni giorno verifica milestone, dipendenze esterne, item bloccati, evidenze raccolte e work item che impattano il cliente. Nelle review settimanali usa il board come base per la narrativa verso management e cliente. Se il board non sostiene questa narrativa, va migliorato l'uso del board, non sostituito con slide manuali.

20.6.11 Esempio di flusso end-to-end in Azure DevOps

Un esempio tipico può chiarire il modello. Durante un progetto emerge la richiesta di tracciare una nuova evidenza documentale. Il DM registra la richiesta nel backlog di progetto e la collega alla milestone impattata. Nella review con il PM si capisce che il bisogno compare in più contesti e merita valutazione di piattaforma. Viene quindi aperto un item candidato nel backlog prodotto con link al progetto sorgente. Il PM lo prioritizza, l'engineering lo prende in carico, apre branch dedicato, sviluppa il cambiamento, apre PR, supera i check, rilascia e collega la release alla milestone che ne beneficiava. A quel punto il DM può comunicare al cliente non solo che la richiesta è stata soddisfatta, ma con quale perimetro, in quale release e con quali prerequisiti di utilizzo.

Questo esempio mostra bene perché Azure DevOps non dovrebbe essere visto solo come strumento per sviluppatori. Se impostato correttamente diventa il luogo in cui prodotto, progetto e tecnologia si incontrano in modo leggibile.

PARTE VII – KPI, RACI E TEMPLATE OPERATIVI

21. KPI, metriche e cruscotti di controllo

21.1 KPI commerciali – presidio BD e COO

KPI	Come si misura	Fonte	Frequenza	Target orientativo
Opportunita qualificate aperte	N. opportunita in stage Discovery+	HubSpot	Settimanale	> 5 attive
Conversion rate Discovery -> Proposal	% avanzamento tra stage	HubSpot	Mensile	> 60%
Conversion rate Proposal -> Closed Won	% offerte accettate sul totale inviate	HubSpot	Mensile	> 40%
Valore pipeline weighted	Somma valore x probabilita per stage	HubSpot	Settimanale	Da definire su base piano
Durata media ciclo vendita	Giorni da Qualified a Closed Won	HubSpot	Mensile	Riduzione progressiva
% opportunita respinte per non fit	Rifiuti proattivi pre-offerta	HubSpot – tag 'No Fit'	Mensile	< 20% del totale

21.2 KPI delivery – presidio DM e COO

KPI	Come si misura	Fonte	Frequenza
Progetti attivi	N. progetti in stato 'In Execution'	Azure DevOps – Operations	Settimanale
Milestone completate on time	% milestone chiuse entro data pianificata	Azure DevOps – Milestone issues	Per milestone
Slittamento medio milestone	Giorni medi di ritardo su milestone chiuse	Azure DevOps	Mensile
Issue S1/S2 aperte	N. issue critiche aperte	Azure DevOps	Settimanale
Evidence pack completeness	% milestone con evidence pack completo alla verifica	Azure DevOps – Evidence items	Per milestone
Change request per progetto	N. variazioni scope gestite per progetto	Azure DevOps – CR issues	Per progetto

21.3 KPI prodotto – presidio PM

KPI	Come si misura	Fonte	Frequenza
Backlog health	% item con descrizione completa e priorit� assegnata	Azure DevOps – Platform Backlog	Mensile
Lead time prioritizzazione -> rilascio	Giorni medi dalla prioritizzazione al deploy in prod	Azure DevOps – cycle time	Per sprint
Interrupt work percentage	% capacit� assorbita da richieste non pianificate	Azure DevOps – sprint report	Per sprint
Bug post-release	N. bug emersi in prod dopo ogni release	Azure DevOps – Bug items	Per release
Custom vs Product ratio	% richieste classificate custom vs prodotto	Azure DevOps – Feature Request tags	Mensile

21.4 KPI milestone, evidenze e acceptance – presidio COO e DM

KPI	Come si misura	Fonte	Frequenza
Milestone ready on time	% milestone con evidence pack consegnato entro la data di readiness concordata	Azure DevOps	Per milestone
Acceptance al primo passaggio	% milestone accettate dal cliente alla prima presentazione	Azure DevOps / verbale cliente	Per milestone
Tempo medio validazione evidenze	Ore dalla chiusura raccolta alla validazione completata	Azure DevOps – timestamps	Per milestone
Riaperture post-verifica	N. milestone o pacchetti evidenze riaperti per incompletezza o chiarimenti	Azure DevOps – issue / note acceptance	Per milestone

21.5 Dashboard Power BI – struttura raccomandata

Dashboard	Contenuto	Aggiornamento	Destinatario
Executive Overview	Pipeline HubSpot, revenue atteso, progetti attivi, capacity overview	Settimanale automatico	CEO, COO
Delivery Status	Stato per progetto (RAG), milestone completate/in ritardo, issue aperte	Giornaliero da Azure DevOps	COO, DM
Product Health	Backlog size, sprint velocity, bug trend, interrupt work	Per sprint	COO, PM
Milestone & Acceptance	Milestone pronte, acceptance al primo passaggio, tempo validazione evidenze	Per milestone	COO, DM

21.5.1 Routine di lettura manageriale della dashboard

La dashboard non dovrebbe essere usata come cruscotto da osservare passivamente, ma come ordine di lettura comune. Per il CEO la vista iniziale corretta è pipeline, saturazione complessiva, stato dei progetti più esposti e tenuta della piattaforma. Per il COO la sequenza dovrebbe essere più operativa: capacity per ruolo, domanda ponderata in ingresso, milestone a rischio, issue aging, interrupt work e trend di qualità. Il PM guarda backlog health, bug, predictability e rapporto tra richieste di progetto e domanda di piattaforma. Il DM guarda milestone, evidenze, dipendenze aperte e stato delle escalation.

La regola pratica è non saltare subito al sintomo più rumoroso. Se una milestone è in ritardo, prima si guarda se il problema nasce da dipendenze, capacità, qualità del flusso o chiarezza di acceptance. Se il backlog prodotto peggiora, prima si verifica quanto interrupt work è entrato dal lato delivery. In questo modo la dashboard diventa uno strumento di governo trasversale e non un insieme di cruscotti separati.

21.5.2 Lettura integrata della dashboard per CEO, COO, PM e DM

Perché le metriche producano davvero governance, BuildTrust dovrebbe renderle leggibili dentro una dashboard integrata che unisca pipeline commerciale, backlog prodotto, stato dei progetti, capacità disponibile, qualità della piattaforma e andamento delle issue critiche. Il valore della dashboard non è mostrare molti numeri, ma ridurre il tempo necessario per capire dove intervenire. Una dashboard utile permette al CEO, al COO, al PM e ai DM di leggere la stessa realtà da prospettive diverse ma coerenti.

La routine di lettura dovrebbe essere semplice e stabile. Il COO apre ogni settimana la vista su capacity, milestone a rischio, issue aging e interrupt work. Il PM guarda trend di velocity, cycle time, bug post-release e stato del backlog. Il DM guarda evidenze, dipendenze esterne e readiness delle milestone. Il CEO guarda segnali aggregati: progetti realmente in controllo, stabilità della piattaforma, saturazione organizzativa e affidabilità della pipeline. Quando questa lettura è condivisa, le decisioni si spostano da opinioni sparse a priorità governate.

Il principio chiave è che nessuna dashboard sostituisce il giudizio manageriale, ma una buona dashboard rende più difficile ignorare i pattern. Se il sistema mostra insieme calo di predictability, crescita dell'interrupt work e peggioramento della milestone acceptance rate, non servono molte altre prove per capire che il modello operativo è sotto pressione. In questo senso la dashboard è il cruscotto della software factory: non guida da sola, ma permette di capire prima dove il sistema sta surriscaldandosi.

21.6 Come leggere davvero velocity, flow e KPI senza usarli male

Le metriche di sviluppo sono spesso fraintese perché vengono usate come numeri da mostrare e non come segnali da interpretare. In BuildTrust è particolarmente importante evitare questo errore: il team lavorerà in parallelo su prodotto, delivery, supporto e integrazioni, quindi quasi nessun indicatore ha significato se letto da solo. La vera maturità sta nel collegare il dato al contesto e al tipo di decisione che deve produrre.

21.6.1 Come leggere velocity nel backlog di prodotto

Nel backlog di prodotto la velocity aiuta soprattutto il PM a capire quanto lavoro è realistico mettere in roadmap senza creare promesse impossibili. Non dice quante persone lavorano bene, ma quanta domanda il sistema riesce ad assorbire con un certo livello di qualità. Quando la velocity è stabile e il bug rate resta sotto controllo, il PM può ragionare su finestre di rilascio e sequencing delle feature. Quando invece oscilla molto, il punto non è fare più pressione sul team, ma capire quali fattori esterni o interruzioni stanno disturbando il flusso: supporto, richieste da progetto, backlog poco pronto, review lente o dipendenze tecniche non risolte.

Velocity, throughput, lead time e WIP vanno letti insieme. Se il throughput sale ma il lead time peggiora, è possibile che il team stia chiudendo item piccoli lasciando invecchiare il backlog. Se il WIP medio cresce, il sistema tende a sentirsi occupato ma diventa meno prevedibile, con maggiore fatica cognitiva e più probabilità di errori o rilavorazioni. In BuildTrust questa lettura è particolarmente importante perché prodotto, supporto e delivery convivono sullo stesso motore operativo.

Indicatore	Cosa misura	Perche e utile	Segnale di attenzione
Velocity	Volume di lavoro completato in sprint o in finestra temporale omogenea.	Aiuta a leggere la capacita media del team nel tempo.	Oscillazioni forti o crescita apparente non accompagnata da qualita e prevedibilita.
Throughput	Numero di item completati in un periodo.	Utile in Kanban per misurare il flusso reale.	Molti item piccoli chiusi ma poca sostanza rilasciata.
Lead time	Tempo da apertura richiesta a completamento.	Misura l'esperienza complessiva del sistema, non solo dello sviluppo puro.	Backlog che invecchia o richieste urgenti che assorbono troppo il team.
WIP medio	Numero medio di item contemporaneamente aperti per persona o team.	Tiene sotto controllo multitasking e dispersione di focus.	WIP costantemente oltre i limiti definiti dal team.
Predictability	Rapporto tra lavoro pianificato e lavoro effettivamente completato.	Aiuta a capire se il team promette in modo realistico.	Sprint o finestre in cui molto lavoro slitta sistematicamente.
Interrupt work %	Quota di lavoro non pianificato sul totale completato.	Misura quanto il prodotto viene disturbato da emergenze, supporto o richieste da progetto.	Valore elevato e stabile che rende impossibile costruire roadmap affidabile.

21.6.2 Come leggere throughput, cycle time e WIP

Throughput, cycle time e WIP sono spesso piu onesti della velocity perche descrivono il flusso reale. Il throughput mostra quante unita di lavoro il sistema chiude. Il cycle time mostra quanto impiega a chiudere un item una volta iniziato. Il WIP mostra quante cose sono contemporaneamente in corso. Letti insieme raccontano se il team sta lavorando in modo focalizzato o disperso.

Accanto alle metriche di flusso, BuildTrust deve leggere anche quelle di qualita tecnica e stabilita della piattaforma. Bug rate post-release e defect leakage raccontano quanto il sistema di delivery intercetta i problemi prima che arrivino al cliente. Code coverage e technical debt ratio non vanno letti in modo dogmatico, ma aiutano a capire se la velocita attuale viene comprata sacrificando manutenibilita futura. Deployment frequency e Mean Time to Recovery vanno letti come coppia: un'organizzazione matura non e solo quella che rilascia spesso, ma quella che rilascia con ritmo adatto al contesto e che sa recuperare rapidamente se qualcosa va storto.

Indicatore	Cosa misura	Perche e utile	Segnale di attenzione
Bug rate post-release	Numero di bug rilevati dopo un rilascio.	Misura la qualita percepita del rilascio e la robustezza dei controlli pre-prod.	Bug ripetuti o impatto alto subito dopo ogni release.
Defect leakage	Quota di difetti sfuggiti a review, test o staging e arrivati in produzione.	Aiuta a leggere la tenuta del processo di qualita.	Molti difetti scoperti tardi da cliente o supporto.
Code coverage	Copertura dei test automatici sulle componenti rilevanti.	Segnala se il team sta proteggendo il comportamento del sistema.	Coverage basso o in calo su parti critiche.
Technical debt ratio	Stima del debito tecnico rispetto alla base codice.	Aiuta a capire se la velocita attuale sacrifica manutenibilita futura.	Trend in crescita senza piano di contenimento.
Deployment frequency	Frequenza dei rilasci in produzione.	Legge maturita del flusso di delivery e granularita delle release.	Rilasci troppo rari, troppo grandi o troppo instabili.
Mean Time to Recovery	Tempo medio di recupero dopo incident o regressioni.	Misura la resilienza operativa del sistema e la qualita del runbook.	Ripristini lenti o dipendenti sempre dalle stesse persone.

21.6.3 Come leggere i KPI di progetto insieme ai KPI di sviluppo

Una particolarita di BuildTrust e che il management non puo fermarsi a una vista solo tecnica o solo di delivery. Un progetto puo avere un buon throughput tecnico ma una milestone a rischio per assenza di evidenze o dipendenze esterne non chiuse. Per questo i KPI di progetto e quelli di sviluppo vanno letti in coppia.

La delivery di progetto deve essere letta combinando avanzamento, rischio, readiness delle milestone e dipendenze esterne. Milestone on-time rate e milestone acceptance rate sono spesso gli indicatori piu importanti: il primo dice quanto il progetto e prevedibile, il secondo dice quanto BuildTrust arriva alle milestone con un pacchetto davvero maturo. Issue aging, RAID density e dependency closure rate aiutano a leggere il rischio prima che diventi scostamento di piano. Evidence completeness collega direttamente il lavoro di progetto alla promessa di verificabilita.

Indicatore	Cosa misura	Perche e utile	Segnale di attenzione
Milestone on-time rate	Percentuale di milestone chiuse entro la data attesa.	Rende leggibile la prevedibilita del progetto.	Milestone spostate spesso o chiuse senza readiness reale.
Milestone acceptance rate	Percentuale di milestone accettate alla prima presentazione.	Misura la qualita del lavoro di preparazione, evidenza e narrativa verso il cliente.	Molte milestone contestate o rinviate per incompletezza del pacchetto.
Issue aging	Tempo medio di permanenza delle issue aperte per severita.	Aiuta a capire se il team sta chiudendo i problemi o solo convivendo con essi.	Issue S1/S2 che restano aperte troppo a lungo.
RAID density	Numero e concentrazione di rischi, issue e dipendenze per progetto o fase.	Serve a capire se il progetto e sotto pressione crescente.	Picchi improvvisi o crescita continua senza mitigazioni.
Dependency closure rate	Capacita di chiudere dipendenze esterne nel tempo previsto.	Molto utile in progetti con forte peso di accessi, dati o attori cliente.	Dipendenze ricorrenti senza owner o senza scadenza rispettata.
Evidence completeness	Quota di evidenze raccolte e validate rispetto a quelle richieste per la milestone.	Collega il lavoro tecnico alla prova di avanzamento.	Milestone vicine con evidenze ancora sparse o incomplete.

21.6.4 Cosa deve fare il management quando un indicatore peggiora

La parte piu importante di un sistema di KPI non e la misura, ma il comportamento che genera. Un indicatore che peggiora dovrebbe attivare una conversazione strutturata e non una reazione emotiva. Il management dovrebbe sempre distinguere tra sintomo, causa probabile e azione.

Le soglie non devono essere intese come semafori rigidi uguali per ogni fase, ma come punti di attenzione che attivano domande di management. Se il cycle time supera stabilmente la soglia definita, non basta registrare il dato: bisogna capire dove si crea il collo di bottiglia. Se il defect leakage aumenta, il punto non e colpevolizzare lo sviluppo ma verificare review, test, staging e size delle release. Se la milestone acceptance rate peggiora, va rivista la preparazione del pacchetto di evidenze e non solo il lavoro tecnico.

Indicatore osservato	Interpretazione da verificare	Azione utile
Velocity instabile	Interruzioni, backlog poco pronto o capacita frammentata.	Proteggere sprint/backlog e ridurre intake non pianificato.
Cycle time in aumento	Bottleneck su review, test o decisioni.	Ridurre WIP e lavorare sui colli di bottiglia.
WIP elevato	Troppe cose iniziate rispetto alla capacita reale.	Tagliare contesti attivi e chiarire priorita.
Milestone acceptance debole	Evidenze o narrativa preparate tardi.	Anticipare review di readiness e ownership delle prove.
Bug post-release in aumento	Gate di qualita o sizing delle release inadeguato.	Rafforzare test, review e disciplina di rilascio.

21.6.5 Come il COO deve leggere insieme prodotto e progetto

Per il COO il punto decisivo non e avere molti indicatori, ma saper leggere le combinazioni che raccontano la salute del sistema. Predictability prodotto piu interrupt work alto indicano quasi sempre una pressione eccessiva della delivery o del supporto sulla piattaforma. Milestone slippage piu issue aging in crescita indicano che il progetto non sta chiudendo i problemi abbastanza presto. Bug post-release piu hotfix frequenti indicano che la velocita apparente sta superando la capacita reale di governare il rilascio.

La lettura corretta del COO dovrebbe quindi chiudersi sempre con tre domande: quale tensione e locale e quale e sistemica; quale decisione va presa subito; quale altra funzione deve essere riallineata. Se questo passaggio manca, il management vede i numeri ma non governa ancora il modello.

La parte piu importante della lettura del COO e la connessione tra prodotto e progetto. Se il backlog prodotto peggiora nello stesso periodo in cui aumentano richieste urgenti dai progetti, il sistema sta spostando capacita senza una decisione esplicita. Se i progetti accumulano milestone a rischio mentre il prodotto mantiene indicatori apparentemente buoni, puo esserci una separazione troppo rigida tra capacita di piattaforma e bisogni della delivery. Se entrambi peggiorano insieme, molto probabilmente il problema e a monte: domanda eccessiva, staffing insufficiente, qualita di intake debole o mancanza di regole chiare tra custom e roadmap.

Segnale osservato dal COO	Lettura piu probabile	Decisione da valutare
Predictability prodotto in calo + interrupt work alto	La roadmap e troppo disturbata da urgenze o supporto.	Proteggere capacita prodotto e rivedere policy di intake.
Milestone slippage + issue aging in crescita	I problemi vengono chiusi troppo tardi o non vengono governati presto.	Anticipare escalation e rafforzare delivery review.
Bug post-release alti + hotfix frequenti	Il processo di qualita e rilascio e sotto stress.	Rafforzare gate, test e sizing delle release.
Pipeline presales alta + saturazione ruoli chiave	La domanda prossima supera la struttura disponibile.	Attivare hiring, partner o sequencing commerciale.
WIP alto su prodotto e progetti	L'organizzazione sta disperdendo focus in troppi stream.	Ridurre contesti attivi e chiarire priorita top-down.

21.6.6 La vista integrata che il COO dovrebbe aprire ogni settimana

Per essere davvero utile, la dashboard del COO dovrebbe integrare in una sola vista almeno cinque famiglie di segnali: capacita disponibile e saturazione per ruolo; pipeline presales con domanda ponderata a sei mesi; salute del backlog prodotto; andamento dei progetti attivi; qualita del flusso tecnico e delle release. Nessuno di questi elementi, da solo, racconta la situazione reale. Insieme invece permettono al COO di capire se l'organizzazione sta crescendo in modo governato o se sta accumulando pressione in punti nascosti.

La sequenza di lettura dovrebbe essere stabile: prima si guarda la capacita (dove siamo saturi, dove dipendiamo da una sola persona), poi la domanda in ingresso (opportunità prossime e mix di competenze richieste), poi il prodotto (roadmap, interrupt work, bug, qualita del flusso), infine i progetti (milestone, issue aging, dipendenze, evidence completeness e escalation). Questo ordine e importante perche evita di leggere i sintomi prima di aver guardato il contesto che li genera.

21.6.7 Come leggere insieme le performance del prodotto

Sul lato prodotto il COO non deve sostituirsi al Product Manager, ma deve capire se la piattaforma sta assorbendo la domanda in modo sostenibile. Gli indicatori piu utili sono predictability, interrupt work, bug post-release, cycle time, WIP e qualita del backlog. La domanda corretta non e se il team ha chiuso molti item, ma se il flusso del prodotto e abbastanza stabile da rendere credibili roadmap e rilasci.

Se l'interrupt work cresce e la predictability cala, il COO deve leggere il fenomeno come erosione organizzativa: progetti, supporto o urgenze stanno entrando nel motore prodotto piu di quanto previsto. Se il cycle time cresce insieme al WIP, il problema e spesso dispersione del focus o colli di bottiglia nelle review. Se i bug post-release aumentano mentre la velocity sembra buona, il segnale corretto e che si sta spingendo piu velocita di quanta qualita il sistema riesca a sostenere.

La responsabilita del COO e trasformare queste letture in decisioni di sistema: proteggere capacita di piattaforma, modificare policy di intake, riallineare con commerciale, chiedere al PM una semplificazione della roadmap, introdurre buffer per supporto o rafforzare la qualita del rilascio.

21.6.8 Come leggere insieme le performance dei progetti

Sul lato progetto il COO deve leggere prevedibilità, rischio e qualità della governance. Gli indicatori chiave sono milestone on-time rate, milestone acceptance rate, issue aging, dependency closure rate, evidence completeness e stato delle escalation aperte. Il punto non è sapere se il team sta lavorando, ma se il progetto è davvero in controllo e se la narrativa verso il cliente poggia su basi solide.

Un progetto in salute non è quello senza problemi, ma quello in cui i problemi emergono presto, hanno owner, hanno severità chiara e vengono letti in relazione alla prossima milestone. Se l'issue aging peggiora, il COO deve domandarsi se il team sta convivendo con problemi non risolti. Se la milestone acceptance rate cala, la domanda corretta è se si sta arrivando alle review con pacchetti incompleti. Se le dipendenze cliente non si chiudono, il tema non è tecnico ma di governance del progetto.

La lettura dei progetti deve distinguere tra problemi locali e pattern ripetuti. Un singolo progetto complicato può avere un andamento particolare; tre progetti diversi che mostrano le stesse frizioni indicano quasi sempre un problema sistemico di intake, mobilitazione, staffing o disciplina degli strumenti.

21.6.9 Domande decisionali che il COO dovrebbe farsi

Una lettura manageriale matura si traduce in domande stabili: il backlog prodotto è ancora protetto abbastanza; i progetti stanno entrando troppo presto; stiamo usando i partner per assorbire variabilità o per coprire buchi strutturali; la qualità del rilascio è coerente con la velocità che stiamo chiedendo; le milestone vengono preparate con anticipo o inseguite all'ultimo; le opportunità in presales stanno già consumando più attenzione di quanta la struttura possa sostenere.

Il comportamento corretto dopo la lettura è altrettanto importante: scegliere poche azioni ad alto impatto e seguirle fino alla review successiva. Se la dashboard mostra troppi segnali in peggioramento, il COO non dovrebbe aprire dieci cantieri insieme, ma identificare il nodo più generativo: capacità, intake, qualità o governance progettuale.

Segnale osservato	Lettura più probabile	Decisione da valutare
Predictability prodotto in calo + interrupt work alto	La roadmap è troppo disturbata da urgenze o supporto.	Proteggere capacità prodotto e rivedere policy di intake.
Milestone slippage + issue aging in crescita	I problemi vengono chiusi troppo tardi.	Anticipare escalation e rafforzare delivery review.
Bug post-release alti + hotfix frequenti	Il processo di qualità e rilascio è sotto stress.	Rafforzare gate, test e sizing delle release.
Pipeline presales alta + saturazione ruoli chiave	La domanda prossima supera la struttura disponibile.	Attivare hiring, partner o sequencing commerciale.
WIP alto su prodotto e progetti	L'organizzazione sta disperdendo focus in troppi stream.	Ridurre contesti attivi e chiarire priorità top-down.

22. Matrice RACI

R = Responsible (esegue), A = Accountable (risponde del risultato), C = Consulted (consultato prima), I = Informed (informato dopo).

Process o	CEO	COO	BD	Presale s	DM	PM	Cloud	DataAIE ng	FSD
Strategia azienda le	A	C	I	I	I	C	I	I	I
Pianifica zione op erativa	I	A	I	I	R	C	C	C	I
Pipeline commer ciale	C	C	A	R	I	I	I	I	I
Qualifica zione op portunita	I	C	A	R	C	C	I	I	I
Offerta t ecnico-e conomic a	I	C	A	R	C	C	I	I	I
Gate pre-firma	C ¹	A	R	R	C	I	I	I	I
Setup e readines s di progetto	I	A	I	C	R	C	I	C	I
Setup progetto	I	C	I	C	A	I	R	R	R
Delivery progetto	I	C	I	I	A	C	C	C	R
Gestione mileston e	I	C	I	I	A	I	I	C	C
Raccolta evidenze	I	C	I	I	R	I	C	A	C
Accepta nce mile stone	I	C	I	I	A	I	C	C	C
Evoluzio ne prodotto	C	C	I	I	C	A	C	C	R

Process o	CEO	COO	BD	Presale s	DM	PM	Cloud	DataAIE ng	FSD
Prioritizz azione backlog	I	C	I	I	C	A	C	C	I
Gestione partner	I	A	I	I	C	C	R	C	C
Support o cliente	I	C	I	C	A	C	C	C	R
KPI e reporting	I	A	C	I	R	R	C	C	I
CI/CD e qualita codice	I	I	I	I	I	C	A	C	R

Nota ¹ – Gate pre-firma. La tabella mantiene il COO come **Accountable** del gate, in coerenza con il principio di una ownership operativa chiara. Per i contratti sopra soglia o a rilevanza strategica, il CEO partecipa comunque alla decisione finale secondo i perimetri descritti nella sezione 10.3. La formalizzazione di questa interazione può essere confermata tra CEO e COO in sede di adozione del TOM.

23. Template operativi

23.1 Template Opportunity Brief (HubSpot)

Campo	Contenuto atteso
Nome opportunità	[Ragione sociale cliente] – [Tipo soluzione]
Cliente / Sponsor	Nome del referente con potere decisionale
Problema da risolvere	Max 3 righe: qual è il pain point operativo o contrattuale
Cluster	DP / Platform / TSP / Ibrido
Valore indicativo	EUR – range accettabile
Probabilità realistica	% stimata in modo onesto, non ottimistico
Complessità contrattuale	Bassa / Media / Alta
Fit con piattaforma	Alto / Medio / Basso – con nota
Dipendenze chiave	Accessi, dati, sistemi terzi necessari
Next action	Cosa succede entro 7 giorni e chi lo fa
Owner BD	Nome del responsabile commerciale

23.2 Template Milestone & Acceptance Matrix

Milestone / Rif.	Significato operativo	Evidenza richiesta	Fonte dato	Validatore	Forma di acceptance
------------------	-----------------------	--------------------	------------	------------	---------------------

23.3 Template Project Charter

Campo	Contenuto
Nome progetto	[Sigla cliente] – [Tipo] – [Anno]
Obiettivo	Max 3 righe: cosa si consegna e quale valore crea per il cliente
Scope IN	Elenco puntuale di cosa e incluso
Scope OUT	Elenco puntuale di cosa e escluso – riduce i malintesi
Team interno BuildTrust	DM, Engineering coinvolti, Data & AI Engineer
Team cliente	Sponsor, referente tecnico, referente contrattuale
Partner coinvolti	Per ogni partner: nome, perimetro, owner interno
Milestone principali	Data, descrizione, evidenza attesa, validatore
Dipendenze critiche	Accessi tecnici, dati cliente, approvazioni esterne
Rischi iniziali	Max 5 rischi con severita e owner del rischio
KPI di progetto	Milestone on time %, evidence completeness %, soddisfazione cliente

23.4 Template Weekly Status Note

Campo	Contenuto
Stato generale	RAG: Verde / Giallo / Rosso – con motivazione se non verde
Obiettivi della settimana	Cosa il team si è impegnato a completare questa settimana
Milestone prossime	Data e stato di preparedness per le prossime milestone
Issue / Blocchi attivi	Elenco con severita, owner e data di risoluzione attesa
Decisioni richieste	Cosa il cliente o il COO devono decidere entro quando
Dipendenze cliente	Cosa si aspetta dal cliente per non bloccare l'avanzamento
Azioni e owner	Chi fa cosa entro quando – max 5 azioni prioritarie

23.5 Template Closure Memo

Campo	Contenuto
Obiettivi iniziali vs risultati	Confronto tra quanto pianificato nel Project Charter e quanto consegnato
Milestone raggiunte	Elenco con data pianificata vs data reale
Evidenze prodotte	N. evidence pack completi, milestone validate al primo passaggio
Problemi incontrati	Top 3 blocchi con causa e come sono stati risolti
Learnings	Cosa si farebbe diversamente – input per il team
Debiti tecnici aperti	Issue in sospeso con piano di chiusura
Input al backlog prodotto	Feature Request aperte durante il progetto, con link ad Azure DevOps

PARTE VIII – CHANGE MANAGEMENT E PIANO DI IMPLEMENTAZIONE

24. Rischi organizzativi e contromisure

Rischio	Probabilità	Impatto	Contromisura principale
Founder dependency persistente – eccessiva dipendenza su poche figure per prodotto, delivery e partner	Alta	Alto	Separazione formale Product Manager / Delivery Manager con RACI condivisa. Riduzione dipendenza da singoli tramite documentazione processi e partner register.
Product cannibalized by projects – roadmap guidata dall'urgenza dei clienti	Alta	Alto	Separazione PM / DM. Gate triage obbligatorio per ogni Feature Request. Reportistica interrupt work in Azure DevOps.
Partner sprawl – proliferazione partner non governati, dispersione know-how	Media	Alto	Owner interno per ogni partner. Work Package formali. Knowledge capture obbligatorio a fine fase.
Overcommitment commerciale – contratti firmati senza capacità disponibile	Media	Molto Alto	Gate COO obbligatorio pre-firma. Capacity board aggiornato settimanalmente. Regola di parallelismo max 3 come policy.

Rischio	Probabilità	Impatto	Contromisura principale
Evidence pack incompleti – milestone fragili	Media	Alto	Evidence Model obbligatorio per ogni progetto. Quality check Data Engineer prima di ogni milestone. KPI evidence completeness.
Ambiguità nei ruoli – conflitti PM/DM/COO	Media	Medio	RACI condivisa. Onboarding formale su processi. Decision log per tracciare chi ha deciso cosa.
Formalismo senza adozione – processi definiti ma non usati	Media	Alto	Adozione graduale. COO come modello di comportamento. Revisione snellita se un processo genera attrito senza valore.
Readiness di progetto troppo dipendente da poche persone	Alta	Alto	La documentazione sistematica è necessaria ma non sufficiente: riduce il rischio di perdita di know-how ma non garantisce la continuità operativa. Per mitigare meglio questo punto, è raccomandabile far crescere la capacità del Delivery Manager di leggere milestone, acceptance e prerequisiti già nel handover, affiancando Data & AI Engineer e COO su almeno un progetto reale e standardizzando il Milestone & Acceptance Matrix.

Rischio	Probabilità	Impatto	Contromisura principale
Mancanza di presidio su privacy e compliance dei dati (GDPR)	Media	Alto	BuildTrust tratta dati di progetto complessi (dati di campo IoT, dati contrattuali, evidenze certificate su blockchain), spesso per conto di clienti con obblighi di compliance propri. L'assenza di un presidio esplicito su GDPR e data residency espone l'azienda a rischi contrattuali e reputazionali. Per questo sarebbe opportuno che COO e supporto legale del gruppo Simem (Studio S.L.T.) verificchino nelle prime fasi di adozione del TOM il perimetro GDPR dei progetti, la residenza dei dati nelle Azure region utilizzate, il ruolo di data controller e data processor e l'eventuale utilità di una clausola standard di data processing agreement da includere nella contrattualistica.

25. Change management e piano di adozione

25.1 Filosofia di adozione

Il TOM funziona solo se viene adottato, non solo approvato. L'adozione non avviene per decreto: si costruisce attraverso successi visibili nel breve termine, strumenti che semplificano il lavoro (non lo complicano) e un management che incarna il modello prima di esigerlo dagli altri.

Il rischio più alto non è la resistenza aperta al cambiamento, ma l'inerzia: continuare a fare come si è sempre fatto perché funziona abbastanza bene nell'immediato. Il COO è la figura chiave per contrastare questa inerzia, rendendo il nuovo modo di lavorare il percorso di minor resistenza.

25.2 Fattori critici di successo

- Il COO conduce ogni rituale con disciplina, anche quando sembra ridondante nelle prime settimane.

- I template vengono usati dal primo progetto reale, non solo testati in sandbox.
- Azure DevOps viene aperto ogni giorno dal team, non solo in occasione delle review.
- I problemi vengono dichiarati nel RAID log prima di diventare crisi, non dopo.
- Il CEO protegge il COO nelle decisioni scomode (intake refusals, escalation, no-go su offerte non fattibili).

25.3 Segnali di adozione riuscita a 90 giorni

- Ogni progetto attivo ha un board Azure DevOps aggiornato e un RAID log con almeno un ciclo di review completato.
- Almeno un progetto reale gestito end-to-end con handover, board, milestone, evidenze e acceptance coerenti.
- La pipeline HubSpot è aggiornata ogni settimana senza sollecitazione del COO.
- Le decisioni operative rilevanti sono nel decision log entro 24h dalla presa.
- Il team usa 'apro una issue in Azure DevOps' come risposta automatica ai problemi, non 'mando una mail'.

25.4 Comportamenti attesi del nuovo modello operativo

Il TOM produce valore solo se si traduce in comportamenti osservabili. La nuova modalità di lavoro BuildTrust non richiede a tutti di fare tutto, ma richiede a tutti di rispettare un sistema comune. Per i manager significa decidere e leggere il lavoro usando gli stessi artefatti che usano i team. Per i tecnici significa rendere il proprio lavoro tracciabile, spiegabile e collegato al valore di progetto o di prodotto.

I manager BuildTrust dovrebbero abituarsi a fare review operative partendo dagli strumenti e non da resoconti generici. Questo richiede una disciplina manageriale precisa: chiedere aggiornamenti sugli artefatti e non accettare che decisioni rilevanti restino solo verbali; usare i rituali per sciogliere nodi e non per ricostruire informazioni assenti; distinguere velocemente se un problema è di priorità, di capacità, di qualità del processo o di esecuzione tecnica.

Per i tecnici la novità non è l'introduzione di regole finì a se stesse, ma la richiesta di lavorare in un modo che renda il valore del proprio contributo più visibile e più integrabile. Ogni attività importante deve avere un work item. Ogni modifica rilevante deve passare da branch e PR. Ogni blocco che impatta milestone, qualità o tempi deve emergere presto.

25.5 Onboarding e knowledge capture

Ogni nuova risorsa o partner che entra nel sistema BuildTrust dovrebbe imparare velocemente non solo il contenuto del TOM, ma il comportamento atteso nell'uso degli strumenti e dei processi. L'onboarding serve proprio a questo: ridurre il tempo necessario per diventare produttivi senza creare varianti locali del modo di lavorare.

La knowledge capture è il completamento naturale dell'onboarding. Ogni progetto, partner, incidente, decisione importante e milestone non dovrebbe solo essere chiuso, ma lasciare dietro di sé un artefatto che renda il sistema un po' più apprenditivo.

25.6 Il valore organizzativo della precisione

Essere precisi, nel contesto BuildTrust, non significa riempire gli strumenti di micro-task o scrivere documenti lunghissimi. Significa rendere ogni passaggio abbastanza chiaro da non doverlo reinterpretare ogni volta. La precisione è un investimento in velocità futura: meno errori di comprensione, meno ricostruzioni manuali, meno conflitti di aspettativa, più possibilità di delegare e scalare.

Per questo il TOM dovrebbe essere applicato con rigore soprattutto nei punti in cui l'organizzazione oggi rischia di perdere valore: handover tra funzioni, distinzione tra prodotto e custom, preparazione delle milestone, uso disciplinato di Azure DevOps, lettura dei KPI e gestione delle escalation.

25.7 Piano suggerito di adozione del TOM

L'adozione dovrebbe essere trattata come un programma operativo a fasi e non come un rilascio unico. Per BuildTrust è consigliabile una sequenza in quattro fasi, con una logica molto pragmatica: prima rendere visibili i flussi minimi, poi disciplinare il governo, poi usare davvero i dati per decidere, infine consolidare la struttura e misurare la tenuta del modello. Questo approccio riduce il rischio di rigetto organizzativo e permette di imparare mentre il sistema entra in funzione.

Fase	Finestra temporale	Obiettivo principale	Risultato atteso
Fase 1 - Fondazioni operative	0-30 giorni	Attivare strumenti, rituali minimi, board, RAID, handover e ownership.	Ogni progetto attivo e ogni stream prodotto hanno un luogo chiaro di governo.
Fase 2 - Governo dei flussi	30-60 giorni	Stabilizzare handover, triage prodotto, delivery review, partner management e metriche base.	Le decisioni chiave iniziano a poggiare su artefatti aggiornati e non su memoria dispersa.
Fase 3 - Capacity e demand planning	60-120 giorni	Collegare pipeline presales, staffing tipico, KPI e priorità di struttura.	COO e commerciale iniziano a decidere hiring, sequencing e protezione roadmap con metodo.
Fase 4 - Consolidamento software factory	120-180 giorni	Rendere il modello il modo normale di lavorare, con review periodiche e correzioni mirate.	BuildTrust lavora con maggiore prevedibilità, meno reattività e più capacità di apprendimento.

25.8 Cosa fare nei primi 180 giorni

Nei primi trenta giorni l'obiettivo non deve essere perfezionare tutto, ma introdurre le fondamenta non negoziabili. Il management dovrebbe scegliere un perimetro pilota fatto da pochi progetti attivi e dal backlog di prodotto principale. Su questo perimetro vanno resi obbligatori alcuni comportamenti: board aggiornato, work item per il lavoro significativo, RAID attivo, kickoff interno strutturato, handover commerciale formalizzato, weekly operating review e delivery review su artefatti reali.

Tra 30 e 60 giorni il secondo passo è migliorare la qualità del governo. Qui BuildTrust dovrebbe rafforzare classificazione delle richieste, distinzione tra prodotto e custom, triage prodotto, gestione partner, template di milestone ed evidence, uso delle PR e delle branch policy, oltre a introdurre i primi KPI letti in modo manageriale. E anche il momento giusto per correggere eventuali configurazioni degli strumenti che si stanno rivelando poco pratiche.

Tra 60 e 120 giorni il TOM smette di essere solo disciplina operativa e diventa leva di management. Il COO e il commerciale dovrebbero introdurre in modo stabile la review di demand planning, leggere la pipeline con probabilità di ingresso, confrontarla con lo staffing tipico dei progetti e definire trigger di hiring o di ricorso ai partner. In parallelo il PM dovrebbe leggere backlog health, interrupt work e qualità di rilascio per capire se la piattaforma è davvero protetta.

Tra 120 e 180 giorni il focus dovrebbe spostarsi dal setup al consolidamento. Le pratiche che hanno funzionato vanno rese standard; quelle che hanno generato attrito inutile vanno semplificate senza perdere il principio che proteggevano. Il management dovrebbe fare una review completa del modello: quali rituali producono valore, quali KPI guidano davvero decisioni, dove il confine tra prodotto e progetto è ancora fragile, quali ruoli sono sotto stress e quali hiring o riallocazioni sono ora chiaramente giustificati.

25.9 Governance dell'adozione e rischi tipici

Un piano di adozione senza una governance dedicata tende a perdere forza dopo le prime settimane. Per questo BuildTrust dovrebbe trattare l'adozione del TOM come un programma manageriale con ownership chiara. Il CEO sostiene la direzione e protegge la coerenza strategica. Il COO guida l'adozione operativa, misura i segnali di tenuta e interviene dove emergono attriti o deviazioni. Il Product Manager presidia ciò che impatta backlog, standard di piattaforma e rapporto tra prodotto e custom. I Delivery Manager presidiano la disciplina sui progetti attivi. I referenti tecnici trasformano le regole di processo in abitudini concrete su board, branch, PR, release e documentazione.

Questa governance non richiede un programma parallelo e pesante, ma richiede una routine esplicita. Una review di adozione bisettimanale nei primi 60 giorni e poi mensile fino a 180 giorni è in genere sufficiente. In quella sede non si dovrebbe discutere teoria, ma leggere segnali molto concreti: quanti progetti stanno usando davvero board e RAID, quante richieste prodotto entrano già con classificazione corretta, quante decisioni sono tracciate, quante milestone arrivano preparate, quante eccezioni vengono ancora gestite fuori processo.

Rischio di adozione	Come si manifesta	Contromisura consigliata
Fatica iniziale	Percezione che il nuovo modo rallenti tutto.	Ridurre il perimetro iniziale e presidiare solo i comportamenti non negoziabili.
Eccezioni frequenti	Nuove opportunit� o urgenze saltano board, gate e handover.	Far autorizzare esplicitamente ogni deroga dal COO o dal CEO.
Falsa adozione	Gli strumenti sono compilati, ma le decisioni vere avvengono altrove.	Usare board, RAID e KPI nelle review operative reali.
Sovra-burocratizzazione	Troppi passaggi, troppi campi, troppi documenti poco usati.	Semplificare senza perdere tracciabilit� e ownership.
Ricaduta nel modello precedente	Dopo qualche settimana si torna a chat, memoria e urgenze locali.	Fare review di adozione e correggere subito i comportamenti regressivi.

25.10 Segnali concreti di adozione riuscita e prime decisioni

Per valutare l'adozione non basta chiedere alle persone se il modello piace. Servono segnali osservabili. I migliori sono quelli che mostrano meno dipendenza da ricostruzioni manuali e maggiore capacit  di lettura anticipata. Per esempio: i project board raccontano davvero lo stato, i rituali chiudono con decisioni e owner, le feature di prodotto nate dai progetti vengono triagate con metodo, le opportunit  mature entrano nel demand planning, le milestone arrivano con evidenze pi  pronte e le escalation vengono fatte prima del punto di crisi.

Esistono anche segnali indiretti molto importanti. Il management impiega meno tempo a capire cosa stia succedendo. I clienti ricevono comunicazioni pi  stabili e meno difensive. I tecnici devono fare meno lavoro di spiegazione ex post perche il flusso   gi  tracciato. Le decisioni di hiring o di partnering diventano pi  fondate. In sintesi, il TOM sta funzionando quando abbassa il costo di coordinamento e aumenta la qualit  delle decisioni.

Per evitare che il TOM venga percepito come un documento importante ma non urgente, BuildTrust dovrebbe chiudere la sua approvazione con un set molto chiaro di decisioni iniziali. La prima   nominare l'owner operativo dell'adozione, che in questo modello coincide naturalmente con il COO. La seconda   scegliere il perimetro pilota su cui applicare subito il nuovo metodo: quali progetti attivi, quale backlog di prodotto, quali partner gi  in corso. La terza   confermare i comportamenti non negoziabili dei primi 30 giorni: board attivi, handover formalizzato, RAID, milestone con evidenze, review operative sugli strumenti e uso disciplinato di Azure DevOps. La quarta   definire il calendario delle review di adozione e demand planning, in modo che il modello entri subito nella routine manageriale.

PARTE IX – APPENDICE MANAGERIALE

Questa parte raccoglie chiavi di lettura manageriali, esempi di interpretazione e criteri di applicazione del TOM per il contesto BuildTrust. I capitoli principali del documento restano la fonte ufficiale per processi, ruoli, strumenti e KPI; questa appendice li rilegge dal punto di vista di CEO, COO, Product Manager, Delivery Manager e commerciale, per facilitare decisioni coerenti di priorit , capacit  e

struttura.

Chiave di lettura manageriale del product management

Le regole operative di product management restano definite nel capitolo 15. Questa sezione non aggiunge nuove prescrizioni: aiuta management e leadership a leggere il ruolo del Product Manager come presidio di coerenza della piattaforma nel tempo.

In questa chiave di lettura il Product Manager dovrebbe proteggere la piattaforma da due rischi opposti: la deriva custom, che fa entrare ogni bisogno cliente nella roadmap, e l'astrazione eccessiva, che ignora i pattern reali emersi nei progetti.

Ogni richiesta che nasce da un progetto deve essere letta come segnale, non come comando. La vera domanda non è solo se serve al cliente attuale, ma se rafforza una capability riusabile, sostenibile e coerente con l'architettura di BuildTrust.

Il backlog di prodotto deve essere comprensibile nel tempo: ogni feature deve spiegare problema, valore atteso, ipotesi di riuso, impatto tecnico e motivo della priorità.

Lettura manageriale dei ruoli chiave del modello

Ruoli, responsabilità e perimetri decisionali sono già formalizzati nei capitoli 8, 9 e 10. Questa sezione li rilegge in ottica manageriale, per chiarire come le diverse figure dovrebbero osservare lo stesso sistema senza produrre letture incoerenti o concorrenti.

I ruoli chiave del modello dovrebbero poter leggere lo stesso sistema con prospettive diverse ma compatibili. Il CEO guarda coerenza strategica e credibilità. Il COO guarda capacità, rischio operativo e stabilità del sistema. Il PM guarda coerenza della piattaforma. Il DM guarda execution reale, milestone e cliente.

Quando questi sguardi si allineano, BuildTrust si comporta come una software factory. Quando si scollegano, l'organizzazione torna a reagire per urgenze locali.

Lettura manageriale di capacity e priorità

I criteri operativi ufficiali di capacity, intake e demand planning sono definiti nel capitolo 19. Questa sezione non introduce nuove regole, ma propone la chiave di lettura con cui il management dovrebbe interpretarle quando deve scegliere tra crescita, protezione della piattaforma e sostenibilità della delivery.

Il capacity management in BuildTrust non serve solo a pianificare chi lavora su cosa, ma a proteggere il sistema dalle decisioni incoerenti. In una realtà piccola e ad alta specializzazione il sovraccarico non si manifesta sempre come saturazione evidente; spesso si manifesta come rallentamento diffuso, review che slittano, item che invecchiano in board, maggior dipendenza da singole persone e minore qualità di coordinamento.

Per questo la capacita va letta sia in termini quantitativi sia qualitativi. Non basta sapere che una risorsa e allocata al 70% o al 90%; bisogna capire su quanti contesti diversi sta lavorando, quante dipendenze trasversali sta assorbendo, quanto del suo tempo e realmente focalizzato e quanto e frammentato da interruzioni, supporto o decisioni sospese.

Le scelte di priorit  piu difficili dovrebbero essere ricondotte a poche domande stabili: quale lavoro protegge meglio il valore della piattaforma, quale lavoro sblocca milestone contrattuali, quale lavoro riduce rischio sistemico, quale lavoro puo aspettare senza aumentare debito operativo e quale lavoro, pur importante, andrebbe rifiutato o rimandato per non destabilizzare l'intero assetto.

Quando il team e in tensione	Decisione sana
Molti progetti attivi e backlog prodotto in crescita	Congelare nuove iniziative di piattaforma non critiche e proteggere bug, milestone e affidabilit�.
Una richiesta cliente appare strategica ma molto specifica	Passarla in triage strutturato e non farla entrare automaticamente nel backlog di prodotto.
Una risorsa chiave e coinvolta in troppi stream	Ridurre contesti attivi, chiarire priorit� e usare partner solo con owner forte.
Il commerciale vuole accelerare un avvio	Usare il gate di capacita e, se necessario, sequenziare invece di promettere parallelismo non sostenibile.

Lettura manageriale integrata di metriche e performance

Le definizioni operative dei KPI, delle dashboard e dei criteri di lettura sono contenute nel capitolo 21 (§21.6.1–21.6.9). Questa nota rilegge il principio manageriale fondamentale: le metriche non devono servire a controllare le persone, ma a rendere leggibile il sistema.

Le metriche utili non sono quelle piu facili da mostrare, ma quelle che aiutano a decidere. Conviene separare i KPI di prodotto (flusso, qualita, stabilit  della piattaforma) dai KPI di progetto (milestone, dipendenze, rischi, affidabilit  della delivery), pur mantenendo una vista unificata per COO e management.

Un errore frequente e usare una sola metrica come risposta a tutto. La velocity, da sola, non dice se il team sta lavorando bene. Il numero di task chiusi non dice se il progetto e sano se le milestone continuano a slittare. BuildTrust dovrebbe usare set di indicatori combinati. La lettura dettagliata di ogni indicatore — tabelle, soglie e comportamento atteso del management — e sviluppata integralmente in §21.6.1–21.6.4 (KPI di prodotto, qualita tecnica, delivery di progetto, comportamento management) e in §21.6.5–21.6.9 (lettura COO integrata, vista settimanale, domande decisionali).

Appendice di lettura della dashboard integrata

La struttura ufficiale della dashboard e dei KPI è definita nelle sezioni §21.5 e §21.6. I dettagli operativi — tabelle degli indicatori di prodotto, qualità tecnica, delivery di progetto, soglie, comportamento del management — sono sviluppati in §21.6.1–21.6.4. La lettura integrata settimanale del COO, la sequenza di segnali da monitorare e le domande decisionali sono in §21.6.5–21.6.9.

Chiave di lettura dei flussi interfunzionali

Le fasi operative dei passaggi tra commerciale, presales, prodotto, delivery ed engineering — informazioni minime da raccogliere, gate pre-firma, artefatti di avvio progetto e matrice degli scambi tra funzioni — sono descritte integralmente in §12.4.1–12.4.5.

Appendice manageriale di demand planning a sei mesi

Il processo operativo di demand planning — perché partire dalle presales, come stimare la probabilità di ingresso, staffing tipico per classi di progetto, come leggere i risultati, policy di hiring e ruoli di COO e commerciale nel processo — è sviluppato integralmente in §19.4.1–19.4.5.

Appendice di lettura del COO su prodotto e progetto

La lettura integrata COO — vista settimanale, come leggere le performance del prodotto e dei progetti, come collegare i due mondi e le domande decisionali — è sviluppata integralmente in §21.6.5–21.6.9.

Nota finale su BuildTrust e piano di adozione del TOM

Le linee di change management e di adozione restano formalmente definite nel capitolo 25. Questa chiusura finale le rilegge in una forma più sintetica e direzionale, per aiutare il management a vedere insieme senso della trasformazione, tempi e sequenza di adozione.

BuildTrust si trova in un passaggio molto delicato ma anche molto promettente. Non è più una realtà che può vivere solo di energia imprenditoriale, adattamento rapido e coordinamento informale; allo stesso tempo non deve diventare un'organizzazione pesante, lenta o burocratica. Il valore del TOM sta proprio qui: aiutare BuildTrust a diventare più prevedibile, più leggibile e più scalabile senza perdere velocità, qualità relazionale e capacità di affrontare contesti complessi.

Il punto centrale non è introdurre più regole in astratto, ma costruire un sistema di lavoro comune che renda naturale fare bene le cose importanti: qualificare meglio le opportunità, separare prodotto da custom, avviare i progetti con maggiore disciplina, usare gli strumenti come fonte reale di verità, leggere per tempo i segnali di tensione, fare scelte di capacità prima che i problemi esplodano. Se questo avviene, BuildTrust non guadagna solo efficienza; guadagna credibilità verso clienti, partner e verso sé stessa.

La trasformazione però non avviene perché il documento è ben scritto. Avviene se il management sceglie pochi comportamenti non negoziabili e li rende reali nel quotidiano. In questo senso il TOM va letto come una proposta di disciplina progressiva: prima si stabilizzano flussi, ruoli e strumenti essenziali, poi si rafforzano metriche, governance e capacità previsiva, infine si consolida il modello come modo normale di lavorare. Il successo non sarà dato dall'adozione formale dei template, ma dal fatto che l'organizzazione smetterà di dipendere dalla ricostruzione manuale dello stato e inizierà a decidere con più anticipo e meno attrito.

Piano suggerito di adozione del TOM

L'adozione dovrebbe essere trattata come un programma operativo a fasi e non come un rilascio unico. Per BuildTrust è consigliabile una sequenza in quattro fasi, con una logica molto pragmatica: prima rendere visibili i flussi minimi, poi disciplinare il governo, poi usare davvero i dati per decidere, infine consolidare la struttura e misurare la tenuta del modello. Questo approccio riduce il rischio di rigetto organizzativo e permette di imparare mentre il sistema entra in funzione.

Fase	Finestra temporale	Obiettivo principale	Risultato atteso
Fase 1 - Fondazioni operative	0-30 giorni	Attivare strumenti, rituali minimi, board, RAID, handover e ownership.	Ogni progetto attivo e ogni stream prodotto hanno un luogo chiaro di governo.
Fase 2 - Governo dei flussi	30-60 giorni	Stabilizzare handover, triage prodotto, delivery review, partner management e metriche base.	Le decisioni chiave iniziano a poggiare su artefatti aggiornati e non su memoria dispersa.
Fase 3 - Capacity e demand planning	60-120 giorni	Collegare pipeline presales, staffing tipico, KPI e priorità di struttura.	COO e commerciale iniziano a decidere hiring, sequencing e protezione roadmap con metodo.
Fase 4 - Consolidamento software factory	120-180 giorni	Rendere il modello il modo normale di lavorare, con review periodiche e correzioni mirate.	BuildTrust lavora con maggiore prevedibilità, meno reattività e più capacità di apprendimento.

Cosa fare nei primi 30 giorni

Nei primi trenta giorni l'obiettivo non deve essere perfezionare tutto, ma introdurre le fondamenta non negoziabili. Il management dovrebbe scegliere un perimetro pilota fatto da pochi progetti attivi e dal backlog di prodotto principale. Su questo perimetro vanno resi obbligatori alcuni comportamenti: board aggiornato, work item per il lavoro significativo, RAID attivo, kickoff interno strutturato, handover commerciale formalizzato, weekly operating review e delivery review su artefatti reali.

In questa fase il COO ha un ruolo molto operativo: rimuovere ambiguità, chiarire ownership, impedire eccezioni premature e proteggere il tempo del team da iniziative laterali. L'obiettivo non è la

sofisticazione degli strumenti, ma la creazione di una disciplina minima comune.

Cosa fare tra 30 e 60 giorni

Una volta che i flussi minimi sono visibili, il secondo passo è migliorare la qualità del governo. Qui BuildTrust dovrebbe rafforzare classificazione delle richieste, distinzione tra prodotto e custom, triage prodotto, gestione partner, template di milestone ed evidence, uso delle PR e delle branch policy, oltre a introdurre i primi KPI letti in modo manageriale. È anche il momento giusto per correggere eventuali configurazioni degli strumenti che si stanno rivelando poco pratiche.

Il segnale che questa fase sta funzionando è semplice: diminuiscono le conversazioni in cui qualcuno deve ricostruire da zero lo stato di un progetto o di una richiesta di prodotto. Quando le persone iniziano a dire 'guardiamo il board', 'guardiamo il RAID', 'guardiamo il backlog' invece di affidarsi a memoria e chat, l'adozione sta diventando reale.

Cosa fare tra 60 e 120 giorni

Questa è la fase in cui il TOM smette di essere solo disciplina operativa e diventa leva di management. Il COO e il commerciale dovrebbero introdurre in modo stabile la review di demand planning, leggere la pipeline con probabilità di ingresso, confrontarla con lo staffing tipico dei progetti e definire trigger di hiring o di ricorso ai partner. In parallelo il PM dovrebbe leggere backlog health, interrupt work e qualità di rilascio per capire se la piattaforma è davvero protetta.

In questa fase vale la pena anche osservare il comportamento del sistema nel suo insieme: quanti progetti partono più puliti, quante milestone arrivano con evidenze pronte, quanto sono diminuite le richieste non tracciate, quanto è migliorata la qualità della previsione di capacità. È qui che si misura se BuildTrust sta veramente diventando una struttura governata.

Cosa fare tra 120 e 180 giorni

Negli ultimi due mesi del primo ciclo di adozione il focus dovrebbe spostarsi dal setup al consolidamento. Le pratiche che hanno funzionato vanno rese standard; quelle che hanno generato attrito inutile vanno semplificate senza perdere il principio che proteggevano. Il management dovrebbe fare una review completa del modello: quali rituali producono valore, quali KPI guidano davvero decisioni, dove il confine tra prodotto e progetto è ancora fragile, quali ruoli sono sotto stress e quali hiring o riallocazioni sono ora chiaramente giustificati.

A questo punto BuildTrust dovrebbe essere in grado di dire non solo di avere adottato un TOM, ma di avere cominciato a usarlo come sistema di governo. È questo il vero traguardo: non un documento in più, ma un'organizzazione che sa leggere meglio sé stessa, decidere con maggiore anticipo e crescere senza perdere controllo.

Governance dell'adozione del TOM

Un piano di adozione senza una governance dedicata tende a perdere forza dopo le prime settimane. Per questo BuildTrust dovrebbe trattare l'adozione del TOM come un programma manageriale con ownership chiara. Il CEO sostiene la direzione e protegge la coerenza strategica. Il COO guida l'adozione operativa, misura i segnali di tenuta e interviene dove emergono attriti o deviazioni. Il Product Manager presidia ciò che impatta backlog, standard di piattaforma e rapporto tra prodotto e custom. I Delivery Manager presidiano la disciplina sui progetti attivi. I referenti tecnici trasformano le regole di processo in abitudini concrete su board, branch, PR, release e documentazione.

Questa governance non richiede un programma parallelo e pesante, ma richiede una routine esplicita. Una review di adozione bisettimanale nei primi 60 giorni e poi mensile fino a 180 giorni è in genere sufficiente. In quella sede non si dovrebbe discutere teoria, ma leggere segnali molto concreti: quanti progetti stanno usando davvero board e RAID, quante richieste prodotto entrano già con classificazione corretta, quante decisioni sono tracciate, quante milestone arrivano preparate, quante eccezioni vengono ancora gestite fuori processo.

Ruolo	Responsabilità nell'adozione	Segnale che il ruolo sta funzionando
CEO	Sostenere priorità, proteggere le scelte difficili e rafforzare il messaggio organizzativo.	Le eccezioni non svuotano il modello appena introdotto.
COO	Guidare adozione, leggere i dati, sciogliere attriti e prendere decisioni di capacità e governance.	Le review generano correzioni reali e non solo reporting.
Product Manager	Stabilizzare backlog prodotto, triage, qualità del flusso e difesa della piattaforma.	Diminuisce la confusione tra feature di prodotto e richieste cliente.
Delivery Manager	Rendere vivi board, RAID, milestone, evidenze e narrativa verso cliente.	Lo stato dei progetti è leggibile senza ricostruzioni manuali.
Engineering lead / tecnici	Applicare regole di branch, PR, ticketing e qualità del rilascio.	Il lavoro tecnico diventa tracciabile e più prevedibile.

Rischi tipici di adozione e contromisure

Ogni trasformazione operativa incontra resistenze prevedibili. La prima è la fatica iniziale: gli strumenti sembrano richiedere più tempo perché il team sta passando da un coordinamento molto basato sulle persone a un coordinamento più basato sul sistema. La seconda è l'eccezione continua: opportunità urgenti, clienti importanti, richieste interne speciali che chiedono di saltare il metodo. La terza è la falsa adozione: i template esistono e i board ci sono, ma il lavoro vero continua a vivere altrove. La quarta è la sovra-ingegnerizzazione: nel tentativo di fare bene, il team introduce più passaggi e più dettaglio del necessario.

BuildTrust dovrebbe affrontare questi rischi in modo molto pratico. Alla fatica iniziale si risponde riducendo i must-have ai comportamenti essenziali. Alle eccezioni si risponde con sponsorship manageriale: se una deroga è necessaria, deve essere esplicita e temporanea, non un ritorno implicito al vecchio modello. Alla falsa adozione si risponde leggendo gli strumenti in review vere, così

le persone capiscono che quello è davvero il luogo di verità. Alla sovra-ingegnerizzazione si risponde semplificando ciò che non sta producendo valore, senza toccare le regole che proteggono chiarezza, tracciabilità e capacità di decisione.

Rischio di adozione	Come si manifesta	Contromisura consigliata
Fatica iniziale	Percezione che il nuovo modo rallenti tutto.	Ridurre il perimetro iniziale e presidiare solo i comportamenti non negoziabili.
Eccezioni frequenti	Nuove opportunità o urgenze saltano board, gate e handover.	Far autorizzare esplicitamente ogni deroga dal COO o dal CEO.
Falsa adozione	Gli strumenti sono compilati, ma le decisioni vere avvengono altrove.	Usare board, RAID e KPI nelle review operative reali.
Sovra-burocratizzazione	Troppi passaggi, troppi campi, troppi documenti poco usati.	Semplificare senza perdere tracciabilità e ownership.
Ricaduta nel modello precedente	Dopo qualche settimana si torna a chat, memoria e urgenze locali.	Fare review di adozione e correggere subito i comportamenti regressivi.

Segnali concreti che il TOM sta funzionando

Per valutare l'adozione non basta chiedere alle persone se il modello piace. Servono segnali osservabili. I migliori sono quelli che mostrano meno dipendenza da ricostruzioni manuali e maggiore capacità di lettura anticipata. Per esempio: i project board raccontano davvero lo stato, i rituali chiudono con decisioni e owner, le feature di prodotto nate dai progetti vengono triagate con metodo, le opportunità mature entrano nel demand planning, le milestone arrivano con evidenze più pronte e le escalation vengono fatte prima del punto di crisi.

Esistono anche segnali indiretti molto importanti. Il management impiega meno tempo a capire cosa stia succedendo. I clienti ricevono comunicazioni più stabili e meno difensive. I tecnici devono fare meno 'lavoro di spiegazione' ex post perché il flusso è già tracciato. Le decisioni di hiring o di partnering diventano più fondate. In sintesi, il TOM sta funzionando quando abbassa il costo di coordinamento e aumenta la qualità delle decisioni.

Come usare i primi sei mesi per far crescere BuildTrust

I primi sei mesi non dovrebbero essere usati solo per 'mettere ordine', ma per far emergere il vero profilo operativo di BuildTrust. Quali tipi di progetto assorbono più management di quanto sembrasse. Dove la piattaforma viene disturbata più spesso. Quali competenze diventano colli di bottiglia. Quali partner meritano di essere resi strutturali e quali no. Quali passaggi del funnel commerciale si traducono davvero in domanda operativa. Questa lettura è il vero valore del primo ciclo di adozione: far vedere all'azienda, con più precisione, che tipo di software factory sta diventando.

Se BuildTrust userà il TOM in questo modo, il documento non resterà una fotografia organizzativa, ma diventerà un meccanismo di apprendimento e di crescita. È qui che si chiude il senso del lavoro proposto: costruire un modello che non serva solo a descrivere come l'azienda dovrebbe funzionare, ma ad aiutarla concretamente a funzionare meglio.

Le prime decisioni da prendere subito dopo l'approvazione del TOM

Per evitare che il TOM venga percepito come un documento importante ma non urgente, BuildTrust dovrebbe chiudere la sua approvazione con un set molto chiaro di decisioni iniziali. La prima è nominare l'owner operativo dell'adozione, che in questo modello coincide naturalmente con il COO. La seconda è scegliere il perimetro pilota su cui applicare subito il nuovo metodo: quali progetti attivi, quale backlog di prodotto, quali partner già in corso. La terza è confermare i comportamenti non negoziabili dei primi 30 giorni: board attivi, handover formalizzato, RAID, milestone con evidenze, review operative sugli strumenti e uso disciplinato di Azure DevOps. La quarta è definire il calendario delle review di adozione e demand planning, in modo che il modello entri subito nella routine manageriale.

Una quinta decisione, spesso sottovalutata, riguarda il messaggio organizzativo. Il management dovrebbe spiegare con chiarezza perché il TOM esiste, quali problemi vuole risolvere e che cosa ci si aspetta concretamente dalle diverse funzioni. Quando questo messaggio manca, il team tende a leggere il cambiamento come un adempimento amministrativo. Quando invece il messaggio è chiaro, il TOM viene capito come strumento per lavorare meglio, con meno caos e con maggiore capacità di crescere.

Decisione iniziale	Perché è decisiva	Owner
Nominare owner adozione	Serve una regia chiara e non diffusa.	COO
Scegliere perimetro pilota	L'adozione parte su casi reali e controllabili.	COO + PM + DM
Definire i must-have dei primi 30 giorni	Evita dispersione e falsa adozione.	COO
Calendario review operative e demand planning	Trasforma il TOM in routine manageriale.	COO + commerciale + PM
Messaggio di sponsorship al team	Dà senso e legittimità al cambiamento.	CEO + COO

PARTE X – APPENDICE OPERATIVA

Questa parte non introduce regole nuove rispetto al corpo principale del TOM. Funziona come appendice operativa di supporto: raccoglie checklist, promemoria, chiavi di applicazione, quick reference e letture sintetiche utili per usare con continuità i processi già definiti nei capitoli ufficiali.

Le sezioni che seguono servono quindi a facilitare il lavoro quotidiano di manager, delivery e team tecnici senza spostare qui il cuore del modello. Il riferimento normativo resta nei capitoli precedenti; qui resta la versione più leggibile e immediatamente utilizzabile nei momenti di esecuzione.

Come usare questa appendice nel quotidiano

Questa sezione non sostituisce i capitoli ufficiali su governance, processi, toolchain, KPI e adozione. Serve come mappa di utilizzo rapido: aiuta a ritrovare più in fretta i comportamenti e i controlli minimi da applicare nel lavoro reale del team.

Nel contesto BuildTrust il salto verso software factory non avviene solo per strutture formali, ma per comportamenti ripetibili: board aggiornati, issue visibili, milestone preparate, handover chiari, backlog disciplinato, decisioni tracciate e strumenti usati in modo coerente.

Disciplina di conduzione dei rituali

Momento	Regola operativa	Strumento di riferimento
Prima del rituale	Board, RAID log, capacity plan e note devono essere già aggiornati. La riunione non serve a ricostruire dati mancanti.	Azure DevOps, SharePoint, HubSpot
Durante il rituale	Si navigano gli strumenti live. Le slide sono eccezioni; il board e la fonte primaria di verità.	Teams con condivisione schermo + Azure DevOps/HubSpot
Chiusura del rituale	Ogni decisione rilevante ha owner, scadenza e luogo di tracciamento. Se manca uno dei tre elementi, la decisione non è chiusa.	Decision Log o work item Azure DevOps

Handover commerciale verso delivery

L'handover commerciale non è completo quando la proposta è firmata, ma quando il Delivery Manager può iniziare il setup senza dover reinterpretare il venduto. Per questo il passaggio tra BD, Presales e Delivery deve essere trattato come un mini-gate operativo.

Area	Contenuto minimo obbligatorio
Scope	Scope IN, scope OUT, ipotesi di soluzione proposta, elementi custom vs prodotto, dipendenze da partner.
Stakeholder	Sponsor, referente operativo, referente tecnico, referente contrattuale, soggetti con potere di approvazione milestone.
Dati e accessi	Fonti dati previste, disponibilità reale, formati attesi, accessi tecnici promessi, eventuali prerequisiti del cliente.
Governance contrattuale	Milestone, criteri di accettazione, documenti richiesti dal cliente, SLA o vincoli temporali già negoziati.
Red flags	Punti non chiusi, ambiguità emerse, richieste sensibili fuori scope, rischi politici o di allineamento lato cliente.

Promemoria operativo di progetto

Il processo ufficiale di delivery resta descritto nei capitoli 14, 16 e 17. Questa sezione ne riassume i punti di attenzione più utili per il lavoro quotidiano, senza aggiungere nuove prescrizioni rispetto al corpo principale.

Il Delivery Manager in BuildTrust non governa solo task, ma coerenza tra contratto, execution tecnica, stakeholder, evidenze e narrativa verso il cliente. Questo significa che la lettura dello stato non può dipendere dalla memoria individuale o da chat disperse.

Il progetto è sano quando il board riflette davvero il lavoro in corso, le dipendenze esterne sono visibili, le milestone hanno criteri di accettazione leggibili e i blocchi vengono dichiarati prima di trasformarsi in emergenza.

La disciplina di delivery si gioca soprattutto nelle prime due settimane, nella preparazione delle milestone e nella costanza con cui il team aggiorna strumenti e artefatti.

Promemoria operativo di prodotto, toolchain e disciplina tecnica

Le regole ufficiali su prodotto, toolchain, Azure DevOps, qualità tecnica e KPI restano nei capitoli 15, 20 e 21. Questa sezione le raccoglie in forma più sintetica e applicativa, come promemoria di uso coerente del modello.

L'efficacia della toolchain BuildTrust non dipende dal numero di strumenti ma dalla chiarezza dei confini tra loro. Ogni informazione deve avere una casa primaria: HubSpot per la pipeline, Azure DevOps per il lavoro, SharePoint per i documenti stabili, Teams per la comunicazione e Outlook per le formalizzazioni verso esterno.

Il principio del single source of truth è essenziale per evitare doppiopioni, versioni divergenti e status ricostruiti a voce. Se il team usa gli strumenti in modo coerente, il COO può governare capacity, intake, milestone e priorità senza inseguire le persone.

Regole operative trasversali

Questa sezione consolida working agreement, architettura documentale, modello milestone-evidence, casi end-to-end, governance dati, onboarding e anti-pattern operativi. Lo scopo non è riempire il documento, ma rendere trasferibile il modo corretto di lavorare dentro BuildTrust.

Regole di uso del board di progetto

Work item	Quando si usa	Owner tipico
Milestone	Checkpoint contrattuale o di avanzamento significativo verso il cliente.	Delivery Manager
Task	Lavoro tecnico o operativo eseguibile in pochi giorni, sempre collegato a una milestone o a una issue.	Engineering / partner
Issue (RAID)	Rischio, blocco, assunzione o decisione che può alterare tempi, scope, qualità o dipendenze.	DM o COO
Evidence Item	Documento, dato, output di sistema o prova tecnica richiesta per verificare una milestone.	Data & AI Engineer / DM
Bug	Difetto applicativo emerso in delivery o collaudo e che richiede fix tecnico.	Engineering

Quattro domande per distinguere prodotto da custom

Le quattro domande di classificazione (prodotto vs custom) sono sviluppate con la loro tabella nel capitolo sul processo di sviluppo prodotto: **§15.2.1**.

Working agreement Engineering

Ambito	Accordo operativo BuildTrust
Board	Ogni attività ha un work item. Nessun lavoro significativo parte da messaggi o call senza tracciatura.
Branching	Un branch per work item o change set coerente. Naming leggibile e riferimento al task.
Pull Request	Ogni PR ha reviewer, breve descrizione del perché del cambiamento, test eseguiti e impatto atteso.
Code review	La review valuta correttezza, leggibilità, impatto su ambienti, documentazione minima e backward compatibility.
Testing	Ogni change importante ha almeno smoke test o test automatico coerente con il rischio introdotto.
Documentazione	Se cambia il funzionamento di una capability, cambia anche la documentazione minima o il runbook.

Architettura documentale e regole SharePoint

Cartella	Contenuto	Owner di aggiornamento
01_Commerciale	Opportunity brief, discovery note, offerta, handover pack.	BD / Presales
02_Governance	Stakeholder map, verbali kickoff, decisioni chiave, piano di comunicazione.	DM
03_Delivery	Project Charter, work plan, status note, closure memo, milestone plan.	DM
04_Tecnico	Documentazione tecnica, architettura, runbook, configurazioni rilevanti, link repo.	Engineering owner
05_Evidenze	Evidence register, evidenze per milestone, allegati di validazione.	Data & AI Engineer + DM
06_Partner	Work package, checkpoint, deliverable partner, knowledge capture.	Owner partner
07_Support	Ticket summary, post go-live memo, note di continuit� operativa.	DM / support owner

Modello milestone, evidence e acceptance

Passaggio	Domanda guida	Output richiesto
Definizione milestone	Cosa deve essere vero per poter dire che la milestone e raggiunta?	Testo milestone e acceptance criteria.
Mappatura evidenze	Quali dati, documenti o output provano il raggiungimento?	Evidence register collegato alla milestone.
Raccolta	Chi produce e chi verifica ogni evidenza?	Owner per ogni evidence item.
Review interna	Il team BuildTrust presenterebbe questa milestone con tranquillit� al cliente?	Checklist di readiness firmata da DM.
Acceptance cliente	Chi approva, con quale forma e con quali tempi?	Verbale, mail formale o documento di accettazione.

Checklist di readiness: kickoff, milestone, go-live

Momento	Verifiche minime obbligatorie
Kickoff interno	Project Charter letto dal team; ruoli chiari; board attivo; RAID aperto; milestone nominate; stakeholder cliente identificati; ipotesi critiche esplicitate.
Kickoff cliente	Referenti confermati; canale di comunicazione definito; frequenza status concordata; piano di accessi approvato; logica di acceptance spiegata.
Pre-milestone	Task critici chiusi o chiaramente residui; evidence item raccolti; criteri di accettazione verificati; narrativa di presentazione preparata; issue aperte con impatto noto.
Go-live tecnico	Ambiente monitorato; rollback o piano di rientro definito; ownership supporto chiara; release note e runbook aggiornati; stakeholder informati.
Go-live operativo	Utenti o referenti cliente allineati; canale supporto attivo; backlog di ottimizzazioni separato dai bug; DM informato degli impatti del rilascio.

Framework decisionale: build, custom, partner, no-go

Opzione	Quando ha senso	Condizione di successo	Rischio da monitorare
Build interno	Capability core, riuso atteso, impatto architetturale rilevante.	PM e COO proteggono capacità e priorit.	Sottostima del tempo o cannibalizzazione della delivery.
Custom di progetto	Esigenza specifica cliente con basso potenziale di riuso.	Perimetro, costo e manutenzione futuri chiariti.	Entrare di nascosto nella piattaforma come eccezione permanente.
Partner governato	Serve velocità o specializzazione non ancora interna, ma con ownership BuildTrust.	Owner interno forte, work package chiaro, knowledge capture obbligatorio.	Dipendenza esterna e perdita di know-how.
No-go	Bisogno non sostenibile, fuori strategia o incoerente con capacità disponibile.	COO e CEO sostengono la decisione commercialmente.	Accettare il lavoro per debolezza e pagarlo poi in execution.

Come leggere la salute reale di un progetto

Un progetto può apparire molto attivo, con molte call, molti task aperti e molte persone coinvolte, ma essere in realtà in peggioramento. La vera salute si vede nella leggibilità del board, nella prevedibilità delle milestone, nella chiarezza delle dipendenze esterne e nella qualità della narrativa verso il cliente.

Segnale	Cosa indica	Azione consigliata
Troppi item fermi in In Progress	Multitasking eccessivo o blocchi non dichiarati.	Ridurre WIP, chiarire priorit�, aprire issue dove serve.
Milestone discusse solo a ridosso della scadenza	Assenza di previsione reale e di readiness progressiva.	Introdurre review pre-milestone e ownership esplicita delle evidenze.
Dipendenze cliente senza owner	Il progetto dipende da fattori esterni non governati.	Rendere la dipendenza visibile in RAID e riallinearla con il cliente.
Partner silenziosi o opachi	Rischio di slittamento nascosto e perdita di controllo.	Fare checkpoint formale e verificare output intermedi.
Feature request da progetto non instradate	Confusione crescente tra delivery e prodotto.	Aprire subito work item nel backlog Platform o classificare come custom.
Weekly Status Note poco chiaro	Il DM non ha una narrativa stabile dello stato.	Ricostruire lo stato a partire da board, issue e milestone.

Governance di dati, integrazioni e adozione del modello

Una quota rilevante della complessit  di BuildTrust non nasce tanto dal codice applicativo, quanto dalla qualit  delle integrazioni e dei flussi dati. Ogni progetto tecnico dovrebbe avere una vista dedicata alla governance dati e integrazioni, per evitare che problemi di accesso, mapping o qualit  emergano solo come ritardo.

Elemento di governance	Domanda da chiarire	Owner primario
Fonte dati	Chi controlla davvero la sorgente e con quale affidabilit�?	DM + referente cliente
Accesso	L'accesso � attivo, stabile e coerente con il formato atteso?	Cloud Engineer / Data & AI Engineer
Mapping	Come traduciamo il dato sorgente in concetti utili alla piattaforma e alle milestone?	Data & AI Engineer
Qualit�	Qual � il livello minimo accettabile di completezza, frequenza e coerenza?	DM + Data & AI Engineer
Escalation	Cosa facciamo se il dato non arriva o non � affidabile?	DM + COO
Riuso	Questo pattern di integrazione puo diventare capability standard di piattaforma?	PM + Data & AI Engineer

Cadenza dei primi 90 giorni di adozione del TOM

Finestra	Focus operativo	Segnale che stiamo andando bene
0-30 giorni	Attivare toolchain, rituali, board e template minimi su casi reali.	Ogni progetto attivo ha board e RAID leggibili; i rituali producono action item tracciati.
30-60 giorni	Migliorare qualita di backlog, decisioni e capacity management.	Le issue critiche emergono prima; PM e DM distinguono meglio prodotto e custom.
60-90 giorni	Stabilizzare il modello e usarlo per decisioni di priorita e struttura.	La direzione usa dati e non impressioni per decidere intake, roadmap e rinforzi di capacita.

Anti-pattern operativi da evitare

- Vendere un progetto prima che esista una lettura minima di dati, accessi e capacita.
- Aprire canali paralleli di coordinamento fuori dagli strumenti ufficiali per velocita apparente.
- Mescolare backlog prodotto e backlog cliente fino a perdere il confine tra piattaforma e custom.
- Lasciare issue o rischi noti senza severita e senza owner perche li sistemiamo dopo.
- Considerare partner e fornitori come scatole nere invece che come capacita da governare.
- Ritenere chiuso un item tecnico senza documentazione minima o senza impatto comprensibile per DM e PM.
- Congelare il capacity board quando i progetti aumentano, proprio nel momento in cui servirebbe di piu.
- Fare escalation solo quando il cliente ha gia percepito il problema.

Presales tecnico e qualificazione delle opportunita complesse

Questa sintesi richiama in forma rapida quanto gia definito nel capitolo 12 sul processo commerciale e presales. Serve come promemoria operativo nei casi complessi, non come ridefinizione del gate pre-firma.

Nelle opportunita BuildTrust piu complesse il presales non puo limitarsi a descrivere la soluzione desiderata o a raccogliere requisiti in senso generico. Deve invece costruire una lettura eseguibile del problema: quali attori sono coinvolti, quali fonti dati esistono davvero, quali milestone contrattuali sono rilevanti, quali dipendenze tecniche devono essere risolte prima dell'avvio e quali elementi della piattaforma coprono gia il bisogno rispetto a quelli che richiedono configurazione o sviluppo.

Una discovery di qualita deve ridurre l'ambiguita, non solo produrre documentazione. Questo significa che il presales dovrebbe uscire dalle sessioni con tre risultati minimi: una mappa del problema del cliente formulata in termini operativi, una prima lettura di fattibilita rispetto alla piattaforma BuildTrust e una lista di rischi da sottoporre al COO e al Delivery Manager nel gate pre-firma.

La qualita del presales ha un impatto diretto sulla salute della delivery. Una proposta con scope vago, dati assunti ma non verificati o ruoli lato cliente non chiariti genera quasi sempre un progetto che parte con ritardo invisibile. Per questo BuildTrust dovrebbe trattare la discovery come un investimento operativo e non come una fase commerciale leggera.

Domanda di discovery	Perche e critica per BuildTrust
Quale decisione operativa del cliente vogliamo rendere piu oggettiva?	Aiuta a collegare la soluzione al valore reale e a definire meglio milestone ed evidenze.
Quali dati esistono oggi e con quale affidabilita?	Riduce il rischio di vendere integrazioni che poi non partono.
Chi approvera davvero l'avanzamento e con quale forma?	Anticipa il modello di acceptance che il progetto dovra sostenere.
Quali eccezioni o personalizzazioni sono gia state chieste?	Aiuta a distinguere presto tra prodotto, configurazione e custom.
Quale partner o competenza esterna potrebbe servire?	Permette di leggere il vero assorbimento di capacita e il rischio di dipendenza.

Gestione dei partner durante l'esecuzione

Il modello ufficiale di partner management resta descritto nel capitolo 16. Qui viene riportato solo come quick reference per verificare, in esecuzione, se il perimetro partner e governato nel modo atteso.

Nel modello BuildTrust il partner non deve mai diventare una seconda regia. Anche quando esegue una parte tecnica importante, il partner resta una capacita coordinata da BuildTrust e non un soggetto che ridefinisce scope, priorita o criteri di completamento. Questo principio e essenziale per difendere know-how, ritmo operativo e narrativa verso il cliente.

La gestione corretta dei partner richiede tre discipline minime. Primo: ogni partner ha un owner interno BuildTrust che si assume la responsabilita della relazione e del risultato. Secondo: il lavoro del partner e sempre tradotto in work package con output attesi, criteri di review e checkpoint intermedi. Terzo: alla fine di ogni fase il know-how torna in casa, tramite documentazione, commenti sui work item, evidenze o review di consegna.

Il rischio piu comune non e il partner che fallisce apertamente, ma il partner che procede in autonomia su un perimetro poco chiarito e restituisce tardi un deliverable difficile da integrare. In questi casi la perdita di controllo non e immediatamente visibile ma si traduce in debito di coordinamento, ritardo, rilavorazione interna e minore credibilita verso il cliente.

Checkpoint partner	Cosa verificare	Owner BuildTrust
Kickoff del work package	Scope, output atteso, vincoli, accessi, formato della consegna.	Owner interno
Checkpoint intermedio 1	Avanzamento reale, blocchi, qualita degli output prodotti finora.	Owner interno + referente tecnico
Checkpoint intermedio 2	Coerenza con milestone di progetto e impatto su altre dipendenze.	DM + owner interno
Review finale	Qualita del deliverable, documentazione, learnings, passaggio di conoscenza.	Owner interno + reviewer tecnico

Comunicazione cliente e narrativa di progetto

Le regole di comunicazione con il cliente, la narrativa di progetto e i comportamenti attesi del Delivery Manager sono sviluppati nel corpo del documento: **§14.5**.

Release, go-live e continuit  operativa

Le regole ufficiali su supporto, go-live e toolchain di rilascio sono nei capitoli 17 e 20. Questa sezione le riassume come checklist di lettura rapida nei momenti in cui serve verificare readiness tecnica e readiness organizzativa.

Nel contesto BuildTrust il rilascio non dovrebbe mai essere considerato solo un evento tecnico. Ogni release impatta processi, milestone, supporto, aspettative dei Delivery Manager e in alcuni casi la logica con cui il cliente interpreta stati e avanzamenti. Questo rende la release readiness una verifica di sistema e non solo di pipeline.

Il passaggio da staging a produzione dovrebbe essere sostenuto da un set minimo di verifiche leggibili: quality gate superato, smoke test eseguiti, effetti su dati e configurazioni noti, eventuali dipendenze di progetto comunicate, runbook aggiornato, finestra di osservazione post-rilascio definita e ownership chiara in caso di incident. Se uno di questi elementi manca, il rischio non   solo tecnico ma organizzativo.

Il go-live operativo richiede inoltre una forma di allineamento con il delivery. Se una capability nuova o modificata entra in produzione ma i DM non ne capiscono impatto, limiti o prerequisiti, il team si trover  a fare supporto reattivo su qualcosa che era nato per generare scalabilit . Anche questo   un punto chiave della software factory: nessun rilascio produce valore se non   assorbito correttamente dall'organizzazione.

Aspetto	Domanda di readiness
Qualit� tecnica	I test rilevanti sono passati e il quality gate � verde?
Dati e configurazioni	Sappiamo se la release tocca mapping, pipeline, secrets o ambienti?
Impatto sui progetti	I DM coinvolti sono stati informati prima del deploy?
Runbook	Esiste una guida minima per supporto, recovery o verifiche post-release?
Osservazione post go-live	Chi guarda alert e bug nelle prime 24-48 ore?

Onboarding e knowledge capture

Il principio di onboarding e apprendimento organizzativo è già richiamato nel capitolo 25. Qui resta una versione più pratica, pensata come promemoria per fare in modo che nuove persone, partner e progetti assorbano davvero il modello BuildTrust.

Ogni nuova risorsa o partner che entra nel sistema BuildTrust dovrebbe imparare velocemente non solo il contenuto del TOM, ma il comportamento atteso nell'uso degli strumenti e dei processi. L'onboarding serve proprio a questo: ridurre il tempo necessario per diventare produttivi senza creare varianti locali del modo di lavorare.

L'errore più comune è trattare l'onboarding come trasferimento di documenti o accessi. In realtà la comprensione arriva quando la persona vede un board reale, una milestone reale, una issue reale, una release reale. Per questo BuildTrust dovrebbe sempre mostrare esempi vivi: un progetto in corso, un work package partner, una feature request da progetto a prodotto, un evidence pack e un Closure Memo ben fatti.

La knowledge capture è il completamento naturale dell'onboarding. Ogni progetto, partner, incidente, decisione importante e milestone non dovrebbe solo essere chiuso, ma lasciare dietro di sé un artefatto che renda il sistema un po' più apprenditivo. È proprio questa accumulazione di apprendimento strutturato che differenzia una startup reattiva da una software factory affidabile.

- Ogni progetto dovrebbe chiudersi con almeno un learning che cambia un template, una checklist o una regola di lavoro.
- Ogni partner dovrebbe lasciare documentazione riutilizzabile, non solo deliverable finali.
- Ogni nuova risorsa dovrebbe poter capire dove guardare per leggere stato, rischio, documenti e priorità.

Promemoria sui comportamenti attesi del nuovo modello operativo

Il change management e i comportamenti attesi sono già trattati nel capitolo 25. Questa sezione non li ridefinisce: li rende più facili da ricordare e osservare nel lavoro quotidiano, soprattutto durante i primi mesi di adozione.

Il TOM produce valore solo se si traduce in comportamenti osservabili. La nuova modalità di lavoro BuildTrust di domani non richiede a tutti di fare tutto, ma richiede a tutti di rispettare un sistema comune. Per i manager significa decidere e leggere il lavoro usando gli stessi artefatti che usano i team. Per i tecnici significa rendere il proprio lavoro tracciabile, spiegabile e collegato al valore di progetto o di prodotto. Per il commerciale significa promettere con maggiore disciplina. Per il delivery significa governare il cliente a partire da dati e preparazione, non da sola reattività.

Il cambiamento più importante e culturale: BuildTrust non dovrebbe più dipendere dalla memoria dei singoli per capire a che punto sia un progetto, perché una feature sia stata sviluppata, quali rischi siano aperti o quali evidenze manchino a una milestone. Tutto questo deve poter essere letto negli strumenti e negli artefatti ufficiali. Questo non elimina il valore delle persone chiave; al contrario lo moltiplica, perché il loro tempo può essere usato per decidere e migliorare, non per ricostruire

continuamente il passato.

Cosa deve cambiare per i manager

I manager BuildTrust dovrebbero abituarsi a fare review operative partendo dagli strumenti e non da resoconti generici. Il CEO guarda pipeline, priorit  e saturazione. Il COO governa gate, capacity, rischi e coerenza operativa. Il PM legge backlog, riuso, qualit  e impatto sulla piattaforma. Il DM legge milestone, readiness, dipendenze e narrative verso il cliente. Tutti devono essere in grado di fare domande partendo dallo stesso dato, invece che da versioni diverse dello stato.

Questo richiede una disciplina manageriale precisa: chiedere aggiornamenti sugli artefatti e non accettare che decisioni rilevanti restino solo verbali; usare i rituali per sciogliere nodi e non per ricostruire informazioni assenti; distinguere velocemente se un problema   di priorit , di capacit , di qualit  del processo o di esecuzione tecnica; chiudere ogni review con owner, decisione e data di verifica.

Cosa deve cambiare per i tecnici

Per i tecnici la novita non   l'introduzione di regole fini a se stesse, ma la richiesta di lavorare in un modo che renda il valore del proprio contributo pi  visibile e pi  integrabile. Ogni attivit  importante deve avere un work item. Ogni modifica rilevante deve passare da branch e PR. Ogni blocco che impatta milestone, qualit  o tempi deve emergere presto. Ogni rilascio deve lasciare dietro di s  un minimo di documentazione operativa. Questo non serve a controllare lo sviluppatore, ma a far funzionare l'organizzazione quando i contesti e i progetti aumentano.

Un team tecnico maturo in BuildTrust non aspetta che il DM scopra il problema, ma rende visibili gli indicatori di rischio: item troppo grandi, review in coda, dipendenze esterne non risolte, qualit  che sta peggiorando, cambiamenti che richiedono aggiornamento di runbook o supporto. La responsabilit  tecnica non si esaurisce nel codice che compila, ma include il modo in cui quel codice entra nel sistema operativo dell'azienda.

La settimana tipo del nuovo modello operativo

Per rendere il modello concreto puo essere utile immaginare una settimana tipo. All'inizio della settimana il COO guarda capacity, milestone a rischio e opportunit  prossime a gate. Il PM rilegge backlog di prodotto, richieste candidate dai progetti e bug aperti. I DM aggiornano board, RAID e stato delle evidenze prima delle review. Gli sviluppatori riallineano work item, branch e PR aperte. Le review settimanali non ricostruiscono il lavoro: leggono uno stato gi  preparato e decidono dove intervenire.

Durante la settimana i tecnici lavorano su item tracciati, mantengono visibile il flusso, aprono PR gestibili e segnalano blocchi. Il DM mantiene allineati milestone, dipendenze e stakeholder. Il PM intercetta le richieste che hanno significato di piattaforma. Il commerciale aggiorna il contesto delle opportunit  che potrebbero influenzare capacit  o roadmap. A fine settimana la lettura manageriale si concentra su trend e decisioni, non su ricostruzione dello stato.

Momento della settimana	Focus operativo	Output atteso
Inizio settimana	Allineamento board, priorit� e rischi.	Stato leggibile prima delle review.
Meta settimana	Gestione blocchi, decisioni di triage, review PR e avanzamento milestone.	Flusso protetto e issue trattate presto.
Fine settimana	Status verso cliente e management, aggiornamento KPI e decisioni di capacity.	Narrativa chiara e azioni chiuse con owner.

Errori organizzativi che il nuovo modello deve eliminare

Per capire se il TOM sta davvero prendendo piede pu  essere utile osservare se alcuni errori storici iniziano a diminuire. Il primo errore   partire troppo presto: progetto firmato ma ancora opaco. Il secondo   promettere come prodotto cio che   solo custom. Il terzo   usare strumenti diversi come luoghi paralleli e in conflitto di verita. Il quarto   arrivare alle milestone con molto lavoro fatto ma poche evidenze preparate. Il quinto   considerare normale che solo poche persone sappiano davvero cosa sta succedendo.

Se dopo l'adozione del modello questi errori continuano con la stessa frequenza, il problema non sar  nel testo del TOM ma nella disciplina di adozione. In quel caso il management dovr  intervenire non aggiungendo nuove regole, ma facendo rispettare meglio quelle essenziali gi  definite.

Lettura rapida del ciclo di un progetto tecnico in BuildTrust

La sequenza ufficiale del processo di delivery resta descritta nel capitolo 14. Questa sezione la rilegge in modo pi  compatto, come supporto di lettura per manager, Delivery Manager e tecnici quando serve capire rapidamente in quale fase il progetto si trovi e quale tipo di attenzione richieda.

Per essere davvero utile ai manager e ai tecnici, il TOM deve spiegare non solo chi fa cosa, ma come il progetto si muove nel tempo. In BuildTrust il progetto tecnico dovrebbe essere governato come una sequenza di fasi leggibili, ciascuna con un obiettivo preciso, un set minimo di artefatti, un criterio di uscita e un tipo di attenzione manageriale. Questo non significa irrigidire il lavoro in waterfall, ma impedire che fasi concettualmente diverse vengano mescolate fino a perdere controllo.

Una delle cause principali di inefficienza nelle organizzazioni in crescita   trattare tutto come se fosse sempre sviluppo. In realt  prima dello sviluppo esistono attiv  di qualificazione, discovery, setup e definizione del perimetro; durante lo sviluppo esistono attiv  di coordinamento, testing e raccolta di evidenze; dopo lo sviluppo esistono preparazione al rilascio, supporto al go-live e chiusura formale. Se queste fasi non sono nominate e governate, il team si trova a inseguire urgenze miste e il management fatica a capire dove il progetto si stia davvero bloccando.

Fase di mobilitazione e setup

La mobilitazione comincia appena il progetto entra realmente nel perimetro operativo. In questa fase il Delivery Manager deve trasformare il materiale commerciale e presales in una macchina di lavoro pronta a partire. Gli obiettivi non sono ancora 'sviluppare', ma allineare ruoli, creare il board, chiarire milestone, aprire il RAID iniziale, verificare accessi, validare dipendenze e costruire una prima narrativa condivisa all'interno del team.

Se questa fase è fatta bene, il progetto parte con un linguaggio comune. Se è fatta male, i problemi emergono subito sotto forma di task mal definiti, domande già risolte che riemergono, attività urgenti che non trovano owner e continue richieste di chiarimento verso commerciale o presales. La mobilitazione ha quindi una funzione protettiva: abbassa il debito di ambiguità con cui il progetto entra nella delivery.

Fase	Cosa deve succedere	Segnale di completamento
Mobilitazione	Ruoli, scope, milestone e board iniziale sono allineati internamente.	Kickoff interno svolto con action item chiusi.
Setup strumenti	Repository, board, cartelle documentali e template sono attivi.	Il team sa dove guardare e dove scrivere.
RAID iniziale	Rischi, assunzioni, issue e dipendenze già note sono esplicitate.	Esiste una vista iniziale dei punti di attenzione.
Preparazione kickoff cliente	Narrativa di avvio, stakeholder e prerequisiti sono pronti.	Il kickoff cliente non viene usato per improvvisare.

Fase di discovery operativa e chiarimento requisiti

Anche quando la vendita è stata fatta bene, quasi tutti i progetti tecnici hanno una quota di discovery residua. BuildTrust dovrebbe trattarla come fase esplicita e non come attività nascosta dentro i primi task di sviluppo. La discovery operativa serve a trasformare ipotesi in dati utilizzabili: quali fonti esistono davvero, quali regole di business sono stabili, quali utenti o stakeholder prenderanno parte alla validazione, quali componenti di piattaforma saranno usate e quali no.

La cosa importante è che la discovery non resti un esercizio indefinito. Ogni tema aperto dovrebbe avere owner, domanda da chiudere, data attesa e impatto sull'esecuzione. Se manca questo rigore, la discovery si trasforma in un sottofondo continuo che erode capacità e impedisce di distinguere tra lavoro di chiarimento e lavoro di costruzione.

Per i manager questa fase richiede una vigilanza particolare: non va accelerata in modo cieco, ma nemmeno lasciata aperta senza disciplina. Il criterio corretto non è avere tutte le risposte, ma avere abbastanza chiarezza per iniziare il lavoro successivo in modo governato.

Fase di progettazione della soluzione

Una volta chiariti i punti essenziali, il progetto entra nella progettazione della soluzione. Qui il team traduce il problema in un disegno tecnico e operativo: architettura coinvolta, integrazioni, configurazioni, mapping dati, regole di deployment, criteri di test e forma delle evidenze. La progettazione non deve diventare documentazione eccessiva, ma nemmeno restare tutta nella testa degli specialisti. Per BuildTrust è sufficiente un livello di progettazione che renda chiaro cosa verrà costruito, con quali dipendenze e con quali principali scelte.

In questa fase il Product Manager, se coinvolto, ha un ruolo di protezione della piattaforma. Deve verificare che la soluzione non introduca scorciatoie tecniche che sembrano utili sul progetto ma che aumentano complessità futura per il prodotto. Il Delivery Manager invece deve assicurarsi che la soluzione sia traducibile in task, milestone e aspettative leggibili per il cliente.

Per i tecnici la progettazione è anche il momento in cui chiarire i criteri di done. Se il progetto richiede non solo codice ma anche configurazioni, test di integrazione, evidenze documentali o aggiornamento di runbook, tutto questo va reso esplicito prima di entrare pienamente in sviluppo.

Fase di costruzione e integrazione

La fase di costruzione e integrazione è quella in cui il team tecnico produce effettivamente software, configurazioni, mapping e collegamenti con i sistemi esterni. Il rischio qui è considerarla una corsa individuale alla chiusura dei task. In BuildTrust invece dovrebbe essere letta come fase di flusso: il lavoro deve restare piccolo, visibile, prioritizzato e costantemente collegato all'esito di progetto o di prodotto a cui contribuisce.

Il Delivery Manager non deve entrare nel merito di ogni scelta tecnica, ma deve poter leggere se il flusso è sano: quanti item sono aperti, quali sono bloccati, quanto invecchiano le issue, quali task impattano la milestone e quali dipendenze esterne stanno rallentando il team. Il PM osserva invece se lo sviluppo sta assorbendo correttamente il lavoro prodotto o se il backlog viene continuamente destabilizzato da richieste reattive.

Per il team tecnico la qualità del lavoro in questa fase dipende soprattutto da tre discipline: task ben definiti, branch brevi con PR leggibili e feedback rapido su bug o problemi di integrazione. Se una di queste tre discipline salta, aumenta subito il debito di coordinamento.

Fase di verifica, UAT e preparazione evidenze

Molti progetti sembrano vicini alla chiusura tecnica ma arrivano impreparati alla verifica. Questo succede quando test, UAT ed evidenze vengono lasciati alla fine come passaggio amministrativo. In BuildTrust dovrebbero invece essere preparati progressivamente. Ogni milestone dovrebbe avere sin dall'inizio una lista di prove attese e un piano minimo di validazione. In questo modo, mentre il lavoro procede, il team raccoglie già ciò che servirà per dimostrare il risultato.

Il Delivery Manager ha qui una responsabilità chiave: evitare che la milestone venga raccontata solo in termini di task chiusi. Il cliente e il management non comprano task; leggono avanzamento, risultato e affidabilità. Per questo il pacchetto di verifica deve spiegare cosa è stato realizzato, quali test o controlli sono stati eseguiti, quali limiti o prerequisiti restano e quale forma di accettazione si chiede.

Per i tecnici questa fase significa anche preparare l'organizzazione al rilascio. Se una soluzione funziona ma richiede monitoraggio, interventi manuali o configurazioni sensibili, questi aspetti devono essere resi visibili prima della produzione.

Fase di go-live e stabilizzazione

Il go-live è un momento breve ma ad alta densità di rischio. In BuildTrust non dovrebbe essere trattato come passaggio solo del team tecnico. Il rilascio coinvolge delivery, supporto, eventuali partner e talvolta il cliente stesso. Questo significa che oltre alla readiness tecnica occorre verificare readiness informativa: chi sa che cosa sta per essere rilasciato, chi osserva il comportamento nelle ore successive, come si gestisce un rollback o una escalation.

La stabilizzazione successiva al go-live serve a evitare che il progetto dichiari successo troppo presto. Le prime 24-72 ore dovrebbero essere lette come finestra di osservazione: bug emersi, comportamenti inattesi, necessità di supporto, adozione lato cliente, tenuta dei dati o delle integrazioni. Solo dopo questa lettura il team può veramente dire di aver portato il progetto oltre il semplice deployment.

Chiusura, apprendimento e passaggio a supporto o continuità

La chiusura di progetto non coincide con l'ultimo task chiuso. In BuildTrust dovrebbe comprendere una verifica formale di cosa è stato consegnato, un riepilogo di milestone ed evidenze, la raccolta dei principali learning, il trasferimento di informazioni a supporto o al team interno che erediterà la soluzione e una lettura onesta di cosa ha funzionato e cosa no nel processo.

Questa fase è essenziale perché trasforma l'esperienza del singolo progetto in patrimonio organizzativo. Se ogni progetto si chiude senza lasciare un miglioramento a template, checklist, toolchain o regole di lavoro, l'organizzazione continuerà a pagare gli stessi errori. Una software factory matura cresce anche così: non solo consegnando, ma imparando in modo esplicito.

Fase del progetto	Domanda manageriale corretta	Segnale da leggere
Mobilizzazione	Siamo davvero pronti a partire o stiamo ancora inseguendo il venduto?	Board, ruoli e prerequisiti iniziali sono chiari.
Discovery	Le ambiguità stanno diminuendo o si stanno spostando avanti?	Le assunzioni si chiudono con owner e date.
Design	La soluzione è sostenibile per piattaforma e delivery?	Scelte tecniche leggibili e criteri di done chiari.
Build	Il flusso è sano o stiamo solo accumulando lavoro in corso?	WIP, cycle time, blocchi e review in coda.
Verifica	Siamo pronti a dimostrare il valore o solo a dichiararlo?	Evidenze complete e acceptance preparata.
Go-live	L'organizzazione sa assorbire il rilascio?	Runbook, osservazione post-release e owner del supporto.
Closure	Abbiamo chiuso solo l'attività o anche l'apprendimento?	Closure memo e learnings riutilizzabili.

Quick reference di rituali, KPI ed escalation

Le definizioni ufficiali di rituali, escalation, KPI e artefatti restano nei capitoli 10, 18 e 21. Questa sezione le riassume come riferimento rapido da usare nelle review e nei momenti in cui serve verificare se il sistema sta funzionando con la disciplina attesa.

I rituali hanno valore solo se sono costruiti attorno a una domanda di governo. Quando una riunione esiste 'per aggiornarci', tende a riempirsi di informazioni eterogenee e a produrre pochi esiti. In BuildTrust ogni rituale dovrebbe invece avere un oggetto chiaro: decidere intake, leggere capacità, governare backlog, proteggere una milestone, sbloccare un progetto, fare triage delle richieste prodotto, leggere qualità e flusso tecnico.

Questo significa che ogni rituale deve partire da artefatti già preparati. Il board è aggiornato prima della riunione, non durante. Le issue aperte hanno già severità e owner. Le richieste candidate al backlog prodotto arrivano con contesto minimo. Le milestone da discutere hanno già un primo stato di evidenze e readiness. Solo così il tempo delle persone più senior viene speso per leggere e decidere, non per inseguire dati mancanti.

Weekly operating review

La weekly operating review dovrebbe essere il rito principale del COO con PM e DM. Il suo obiettivo non è passare in rassegna tutto, ma evidenziare le deviazioni rilevanti del sistema. Le domande corrette sono: quali milestone sono a rischio, quali issue stanno invecchiando, quali interruzioni stanno erodendo la roadmap, quali opportunità stanno chiedendo capacità non ancora disponibile e dove serve decisione manageriale immediata.

Per funzionare bene questa review dovrebbe usare una dashboard e pochi link profondi ai board. Ogni punto trattato dovrebbe chiudersi con una scelta: riallocare capacità, chiedere escalation al cliente, spostare una priorità, congelare una richiesta, far entrare un partner, anticipare una review di qualità o modificare il piano di una milestone.

Product triage e backlog review

Il product triage dovrebbe servire a governare la pressione in ingresso sul backlog di piattaforma. In BuildTrust questa review ha un valore particolare perché molte richieste nasceranno da progetti reali e tenderanno a presentarsi come urgenti. Il PM deve usare il triage per rimettere ordine: che cosa entra davvero in roadmap, che cosa resta richiesta da progetto, che cosa richiede una discovery ulteriore e che cosa va fermato.

La review di backlog non dovrebbe soffermarsi solo sull'ordine degli item, ma anche sulla loro qualità. Un backlog maturo contiene item comprensibili, con outcome, priorità, criterio di done e collegamento ai motivi per cui il lavoro esiste. Se il backlog cresce ma diventa opaco, l'organizzazione perde già capacità prima ancora di iniziare a sviluppare.

Delivery review di progetto

La delivery review è la sede in cui il DM verifica salute del progetto e prepara la narrativa verso cliente e management. Qui non basta leggere il task board; occorre collegare task, issue, milestone, dipendenze esterne ed evidenze. Un progetto può avere molto lavoro chiuso ma essere comunque indietro sulla sua capacità di dimostrare avanzamento. La review serve proprio a intercettare questa discrepanza.

Il delivery review ben fatto produce almeno tre risultati: chiarezza sulla prossima milestone, lista dei blocchi o rischi che richiedono intervento e preparazione della comunicazione verso il cliente. Se questi esiti non emergono, la review sta probabilmente cadendo nel semplice aggiornamento operativo.

Tech review e qualità del flusso di sviluppo

La tech review non deve essere confusa con la code review. È il momento in cui engineering lead, PM e, quando utile, DM leggono insieme la salute del flusso tecnico: item troppo grandi, PR ferme, bug aperti, qualità della pipeline, debito che si sta accumulando, scelte architetturali che potrebbero influenzare roadmap o delivery. In una realtà come BuildTrust questa review aiuta a evitare che il dettaglio tecnico resti separato dalle scelte di capacità e di priorità.

Rituale	Domanda a cui risponde	Artefatti obbligatori
Weekly operating review	Dove il sistema sta uscendo dal controllo?	Dashboard KPI, capacity view, issue aging, milestone risk list
Product triage	Cosa entra davvero nel backlog di piattaforma?	Backlog candidato, richieste da progetto, capacità disponibile
Delivery review	Il progetto è pronto alla prossima decisione o milestone?	Project board, RAID, evidence status, status note draft
Tech review	Il flusso tecnico è sostenibile e di qualità?	Board tecnico, PR aperte, bug trend, pipeline status

Artefatti minimi e loro funzione

Uno dei rischi tipici di crescita è moltiplicare documenti senza chiarirne la funzione. In BuildTrust conviene invece pochi artefatti, ma molto chiari. Il Project Charter serve per allineare l'impianto del progetto. Il RAID serve per leggere rischio e dipendenze. Il backlog di prodotto serve per ordinare la domanda di piattaforma. Il board di progetto serve per governare l'esecuzione. Il runbook serve per supporto e rilascio. La Weekly Status Note serve per la narrativa verso cliente. Quando ogni artefatto ha un compito preciso, le persone smettono di duplicare contenuti in luoghi diversi.

La regola pratica è semplice: se una informazione viene aggiornata spesso per guidare il lavoro, deve vivere in uno strumento operativo. Se deve restare come riferimento stabile, deve vivere in un documento. Se serve per formalizzare una decisione verso il cliente, deve avere una forma condivisibile e recuperabile. Questa distinzione aiuta molto a evitare caos documentale.

Come leggere davvero velocity, flow e KPI senza usarli male

Le metriche di sviluppo sono spesso fraintese perché vengono usate come numeri da mostrare e non come segnali da interpretare. In BuildTrust è particolarmente importante evitare questo errore: il team lavorerà in parallelo su prodotto, delivery, supporto e integrazioni, quindi quasi nessun indicatore ha significato se letto da solo. La vera maturità sta nel collegare il dato al contesto e al tipo di decisione che deve produrre.

La velocity è un esempio perfetto. Se la si trasforma in obiettivo, il team tenderà a ottimizzare il numero e non il sistema. Se la si usa invece come indicatore di stabilità della capacità, allora diventa utile. Una velocity relativamente stabile suggerisce che il backlog è abbastanza pronto, il team è abbastanza protetto e il flusso non è eccessivamente disturbato. Una velocity che oscilla fortemente chiede invece al management di guardare interruzioni, focus, qualità della definizione degli item e dipendenze esterne.

La lettura corretta è quindi sempre comparativa e storica. Non bisogna chiedersi soltanto 'quanto vale la velocity questa settimana', ma 'come si muove rispetto alle settimane precedenti, insieme a quali altri segnali e con quali eventi organizzativi sullo sfondo'. Se la velocity cala nella stessa finestra in cui aumentano hotfix, supporto urgente e issue aperte lato progetto, il dato va interpretato in modo diverso rispetto a un calo che coincide con refactoring strutturale o preparazione di un rilascio complesso.

Le analisi dettagliate su velocity, throughput, cycle time, WIP e gestione degli indicatori in peggioramento sono sviluppate nel corpo principale del documento: **§21.6.1–21.6.4**.

Decisioni, escalation e comportamento atteso quando qualcosa non va

Ogni modello operativo viene realmente testato quando le cose non vanno come previsto. Per questo BuildTrust deve chiarire non solo il processo ideale, ma anche il comportamento atteso in caso di blocchi, ritardi, bug critici, dipendenze cliente non rispettate o richieste fuori perimetro. Una software factory matura non si riconosce dall'assenza di problemi, ma dalla velocità e dalla qualità con cui li rende leggibili e li governa.

Il primo principio è l'emersione precoce. Quando un rischio comincia a minacciare una milestone o una release, non dovrebbe restare in un canale tecnico ristretto. Va classificato, tracciato e portato rapidamente al livello giusto. Il secondo principio è la responsabilità di proposta: chi porta un problema non dovrebbe limitarsi a segnalarlo, ma dovrebbe già formulare una prima ipotesi di impatto e una possibile strada di gestione. Il terzo principio è la proporzione: non tutto richiede la stessa escalation. Un modello maturo distingue bene ciò che può essere gestito dal team, ciò che richiede il DM, ciò che richiede il PM e ciò che deve arrivare al COO o al CEO.

Il comportamento atteso nelle escalation — tecnica, di progetto e verso il cliente, e con intervento del COO — è sviluppato in dettaglio nel corpo del documento: **§18.3.1–18.3.3**.

Il valore organizzativo della precisione

Essere precisi, nel contesto BuildTrust, non significa riempire gli strumenti di micro-task o scrivere documenti lunghissimi. Significa rendere ogni passaggio abbastanza chiaro da non doverlo reinterpretare ogni volta. La precisione è un investimento in velocità futura: meno errori di comprensione, meno ricostruzioni manuali, meno conflitti di aspettativa, più possibilità di delegare e scalare.

Per questo il TOM dovrebbe essere applicato con rigore soprattutto nei punti in cui l'organizzazione oggi rischia di perdere valore: handover tra funzioni, distinzione tra prodotto e custom, preparazione delle milestone, uso disciplinato di Azure DevOps, lettura dei KPI e gestione delle escalation. Se questi punti diventano solidi, BuildTrust potrà crescere con molta più affidabilità senza appesantirsi inutilmente.

26. Conclusioni

Il presente documento ha definito il Target Operating Model di BuildTrust a partire dall'analisi dello stato organizzativo attuale, con l'obiettivo di fornire alla direzione aziendale un riferimento strutturato per governare la transizione verso un modello di software factory scalabile e industrializzabile.

L'analisi della struttura AS-IS ha evidenziato una discontinuità significativa tra il potenziale della pipeline commerciale 2026 e la capacità operativa disponibile. Tale discontinuità non è riconducibile a carenze di competenza individuale, ma a una configurazione organizzativa ancora centrata su logiche di startup, inadeguata rispetto al livello di parallelismo e complessità che i progetti in pipeline richiedono.

Il TOM proposto introduce cinque cambiamenti strutturali che costituiscono le condizioni necessarie per una crescita sostenibile: la centralità del COO come unico presidio dell'esecuzione operativa end-to-end; la separazione formale tra Product Management e Delivery Management; un setup di progetto disciplinato basato su handover, readiness, milestone, evidenze e acceptance; una toolchain integrata sull'ecosistema Microsoft con Azure DevOps come strumento primario di governo del lavoro; un sistema di KPI e governance operativa che rende visibile e misurabile l'avanzamento a ogni livello dell'organizzazione.

L'efficacia del modello è subordinata alla qualità e alla coerenza dell'adozione. Un Target Operating Model non produce valore per il solo fatto di essere approvato: produce valore quando i processi descritti vengono effettivamente seguiti, gli artefatti vengono prodotti e mantenuti aggiornati, le decisioni vengono tracciate e i rituali di governance vengono condotti con disciplina. La responsabilità primaria di questa adozione è in capo al COO, con il supporto esplicito e la legittimazione del CEO.

Il presente documento costituisce la versione 2.0 del Target Operating Model di BuildTrust. E da intendersi come documento vivente: soggetto a revisione periodica sulla base dell'esperienza operativa acquisita, delle evoluzioni della pipeline e delle decisioni strategiche della direzione. Ogni modifica rilevante al modello deve essere documentata nel Decision Log aziendale con indicazione della versione, della motivazione e dell'approvazione.